HAETAE Provable-Security Correspondence Catalog

Exhaustive mapping for counted EasyCrypt declarations in `haetae-security/provable-security/easycrypt`

Scope and Counts

This handout gives the mathematician's reading of the counted declaration surface used in the machine-checked provable-security proof for HAETAE. The source set is exactly the manifest consumed by `provable-security/verify-provable-security.sh`. The counted declaration classes extracted from that manifest are

$$\#\text{Types} = 63, \quad \#\text{Ops} = 678, \quad \#\text{Modules} = 113, \quad \#\text{Lemmas/Axioms} = 1091, \quad \#\text{Hoare/phoare/equiv lines} = 146.$$

The EasyCrypt code column records the exact declaration line as it appears in the source file. For declarations whose theorem statement spans several lines, the displayed line is the source declaration line; the mathematical column gives the object, predicate, theorem schema, program module, or semantic judgment denoted by the declaration.

Global Mathematical Notation

Let $\mathcal{M}$ be the message space, $\mathcal{PK}$ the public-key space, $\mathcal{SK}$ the secret-key space, and Σ the HAETAE signature space. A randomized HAETAE signing execution is written

$$(\sigma, T) \leftarrow \mathsf{Sign}^{\mathcal{H}}(sk, m; \rho),$$

where ρ denotes signing coins, T denotes the observable transcript, and $\mathcal{H}$ is the lazy random oracle. An adversary $\mathcal{A}$ interacts with signing and hash oracles, producing a forgery $(m^\star, \sigma^\star)$. The basic advantage is

$$\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A}) = \Pr[\mathsf{Verify}^{\mathcal{H}}(pk, m^\star, \sigma^\star) = \mathsf{true} \wedge m^\star \notin Q_S].$$

The EasyCrypt game hops are read as experiments $\mathcal{G}_0, \mathcal{G}_1, \ldots$ with bad events Bad_i, hash-query log Q_H, signing-query log Q_S, and budget parameters q_H, q_S. The two central semantic judgments are

$$\{P\}\ c\ \{Q\} \quad \equiv \quad \forall s.\ P(s) \Rightarrow Q([\![c]\!](s)),$$

$$\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q] \quad \equiv \quad P(s_1, s_2) \Rightarrow Q([\![c_1]\!](s_1), [\![c_2]\!](s_2)).$$

The ROM fundamental-lemma steps have the form

$$|\Pr[\mathcal{G}_i : \mathsf{Win}] - \Pr[\mathcal{G}_{i+1} : \mathsf{Win}]| \leq \Pr[\mathcal{G}_i : \mathsf{Bad}_i],$$

and the sampler-freshness steps instantiate $\Pr[\mathsf{sampler\_expand\_query}(\rho) \in Q_H] \leq q_H/|\mathcal{C}|$ for the appropriate challenge/seed domain $\mathcal{C}$.

Exact Declaration Correspondence

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Security.ec:22	op	`op correctness_obligation =`	Total mathematical function or predicate correctnessobligation in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:29	lemma	`lemma correctness_obligation_holds : correctness_obligation.`	Checked theorem correctnessobligationholds in the final EUF-CMA theorem and advantage composition; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Security.ec:41	lemma	`lemma correctness_perfect &m :`	Checked theorem correctnessperfect in the final EUF-CMA theorem and advantage composition; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Security.ec:45	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\ c\ \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_Security.ec:82	lemma	`lemma euf_cma_security_no_signing &m :`	Theorem eufcmasecuritynosigning contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:137	lemma	`lemma euf_cma_security &m :`	Theorem eufcmasecurity contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Security.ec:156	lemma	`lemma euf_cma_security_with_hop_interfaces &m :`	Theorem eufcmasecuritywithhopinterfaces contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:191	lemma	`lemma euf_cma_security_from_simulated_hop &m :`	Theorem eufcmasecurityfromsimulatedhop contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:246	lemma	`lemma euf_cma_security_from_explicit_boundaries &m :`	Theorem eufcmasecurityfromexplicitboundaries contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:287	lemma	`lemma euf_cma_security_from_rom_internal_transcript_hop &m :`	Theorem eufcmasecurityfromrominternaltranscripthop contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:312	lemma	`lemma rom_internal_transcript_hop_from_clear_site_reduction &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Security.ec:343	lemma	`lemma rom_internal_transcript_hop_from_clear_site_counted_reduction &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Security.ec:386	lemma	`lemma bimodal_to_module_sis_reduction_bound &m :`	Probability bound $\Pr[\text{bimodaltomodulesisreductionbound}] \le B$ or loss inequality controlling a bad event in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:402	lemma	`lemma euf_cma_security_from_mechanized_boundaries &m :`	Theorem eufcmasecurityfrommechanizedboundaries contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:441	lemma	`lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal &m :`	Theorem eufcmasecurityfromromtranscriptandmechanizedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:464	lemma	`lemma euf_cma_security_from_rom_transcript_and_counted_bimodal &m :`	Theorem eufcmasecurityfromromtranscriptandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:515	lemma	`lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal &m :`	Theorem eufcmasecurityfromromclearsiteandmechanizedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:539	lemma	`lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal &m :`	Theorem eufcmasecurityfromromclearsiteandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:601	lemma	`lemma euf_cma_security_from_structural_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfromstructuralnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:621	lemma	`lemma euf_cma_security_from_public_sim_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfrompublicsimnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:641	lemma	`lemma euf_cma_security_from_paper_sim_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfrompapersimnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:672	lemma	`lemma euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfromrompapersimnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:693	lemma	`lemma euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfromrealsigningpapersimnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:714	lemma	`lemma euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal &m :`	Theorem eufcmasecurityfromhaetaerejectionpapersimnmaandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:736	lemma	`lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim &m :`	Probability bound $\Pr[\text{exacthyperballhaetaerejectionnmaboundfrompapersim}] \le B$ or loss inequality controlling a bad event in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:755	lemma	`lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop &m :`	Probability bound $\Pr[\text{haetaerejectionnmaboundfromexacthyperballsamplinghop}] \le B$ or loss inequality controlling a bad event in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:787	lemma	`lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Security.ec:811	lemma	`lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Security.ec:833	lemma	`lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Security.ec:854	lemma	`lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss`	Probability bound $\Pr[\text{realsigningroexacthyperballhopfromrosigningattempthoploss}] \le B$ or loss inequality controlling a bad event in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:876	lemma	`lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_Security.ec:911	lemma	`lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_Security.ec:942	lemma	`lemma euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal &m :`	Theorem eufcmasecurityfromexacthyperballpapersimbridgeandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:975	lemma	`lemma euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal &m :`	Theorem eufcmasecurityfromexacthyperballsamplinghopandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1004	lemma	`lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal &m :`	Theorem eufcmasecurityfromroexacthyperballsamplinghopandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1040	lemma	`lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal &m :`	Theorem eufcmasecurityfromrosigningattempttoroexacthyperballhopandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1070	lemma	`lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m :`	Probability bound $\Pr[\text{realsigningroexacthyperballhopcheckedfromlossbudget}] \le B$ or loss inequality controlling a bad event in the final EUF-CMA theorem and advantage composition.
HAETAE_Security.ec:1084	lemma	`lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_Security.ec:1108	lemma	`lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal &m :`	Theorem eufcmasecurityfrompapersampleliftingandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1138	lemma	`lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal`	Theorem eufcmasecurityfrombudgetedpapersampleliftingandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1185	lemma	`lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m :`	Theorem eufcmasecurityfromcheckedrosamplinghopandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Security.ec:1207	lemma	`lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal &m :`	Theorem eufcmasecurityfromrealsigningexacthyperballhopandcountedbimodal contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
Sig_ROM.ec:5	type	`type pkey.`	Public-key space $\mathcal{PK}$; values of `pkey.` denote HAETAE verification keys.
Sig_ROM.ec:6	type	`type skey.`	Secret-key space $\mathcal{SK}$; values of `skey.` denote HAETAE signing keys.
Sig_ROM.ec:7	type	`type message.`	Carrier set $\mathcal{M}$ of HAETAE messages; the EasyCrypt type `message.` is read as a mathematical message object.
Sig_ROM.ec:8	type	`type context.`	Carrier set context used in the signature interface in the random-oracle model; EasyCrypt equality is the mathematical equality on this set.
Sig_ROM.ec:9	type	`type signature.`	Signature space Σ; values of `signature.` denote candidate HAETAE signatures.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
Sig_ROM.ec:10	type	`type ro_query.`	Transcript/query carrier used to define sets Q_H of random-oracle queries and logged signing transcripts.
Sig_ROM.ec:11	type	`type ro_output.`	Carrier set $\mathsf{rooutput}$ used in the signature interface in the random-oracle model; EasyCrypt equality is the mathematical equality on this set.
Sig_ROM.ec:13	op	`op [lossless] dro_output : ro_query -> ro_output distr.`	Numerical bound or budget parameter $\mathsf{lossless}$ used to upper-bound a probability loss in the signature interface in the random-oracle model.
Sig_ROM.ec:15	type	`type query = message * context.`	Transcript/query carrier used to define sets Q_H of random-oracle queries and logged signing transcripts.
Sig_ROM.ec:16	type	`type signed_query = query * signature.`	Signature space Σ; values of $\mathsf{signed\_query}$ denote candidate HAETAE signatures.
Sig_ROM.ec:18	op	`op fresh_msg (qs : query list) (m : message) (ctx : context) : bool =`	Predicate/event $\mathsf{freshmsg}$ on the game state; in probability expressions it denotes the indicator event 1_{freshmsg}. *Context:* signature interface in the random-oracle model.
Sig_ROM.ec:21	op	`op fresh_sig (qs : signed_query list) (m : message) (ctx : context)`	Predicate/event $\mathsf{freshsig}$ on the game state; in probability expressions it denotes the indicator event 1_{freshsig}. *Context:* signature interface in the random-oracle model.
Sig_ROM.ec:25	lemma	`lemma fresh_msg_nil m ctx : fresh_msg [] m ctx.`	Probability bound $\Pr[\mathsf{freshmsgnil}] \le B$ or loss inequality controlling a bad event in the signature interface in the random-oracle model.
Sig_ROM.ec:28	lemma	`lemma fresh_sig_nil m ctx sig : fresh_sig [] m ctx sig.`	Probability bound $\Pr[\mathsf{freshsignil}] \le B$ or loss inequality controlling a bad event in the signature interface in the random-oracle model.
Sig_ROM.ec:32	type	`type in_t <- ro_query,`	Carrier set int used in the signature interface in the random-oracle model; EasyCrypt equality is the mathematical equality on this set.
Sig_ROM.ec:33	type	`type out_t <- ro_output,`	Carrier set outt used in the signature interface in the random-oracle model; EasyCrypt equality is the mathematical equality on this set.
Sig_ROM.ec:34	op	`op dout <- dro_output.`	Total mathematical function or predicate dout in the signature interface in the random-oracle model.
Sig_ROM.ec:36	module	`module type Oracle = {`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
Sig_ROM.ec:40	module	`module type POracle = {`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
Sig_ROM.ec:44	module	`module type Scheme(O : POracle) = {`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
Sig_ROM.ec:51	module	`module type CORR_ADV(O : POracle) = {`	Adversary interface/module $\mathcal{A}$; its procedures are quantified in the security theorem subject to call budgets.
Sig_ROM.ec:55	module	`module Correctness(H : Oracle, S : Scheme, A : CORR_ADV) = {`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
Sig_ROM.ec:73	module	`module type SignOracle = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
Sig_ROM.ec:77	module	`module type Adversary(H : POracle, O : SignOracle) = {`	Adversary interface/module $\mathcal{A}$; its procedures are quantified in the security theorem subject to call budgets.
Sig_ROM.ec:81	module	`module type NMA_Adversary(H : POracle) = {`	Adversary interface/module $\mathcal{A}$; its procedures are quantified in the security theorem subject to call budgets.
Sig_ROM.ec:85	module	`module NMA_As_CMA(A : NMA_Adversary) (H : POracle, O : SignOracle) = {`	Experiment module $\mathcal{G}_{\mathsf{NMAAsCMA}}$ whose returned Boolean event defines an advantage probability.
Sig_ROM.ec:94	module	`module EUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {`	Experiment module $\mathcal{G}_{\mathsf{EUFCMA}}$ whose returned Boolean event defines an advantage probability.
Sig_ROM.ec:98	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
Sig_ROM.ec:108	module	`module A = A(H, O)`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
Sig_ROM.ec:126	module	`module UF_NMA(H : Oracle, S : Scheme, A : NMA_Adversary) = {`	Experiment module $\mathcal{G}_{\mathsf{UFNMA}}$ whose returned Boolean event defines an advantage probability.
Sig_ROM.ec:149	lemma	`lemma nma_as_cma_exact &m :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
Sig_ROM.ec:174	module	`module SUF_CMA(H : Oracle, S : Scheme, A : Adversary) = {`	Experiment module $\mathcal{G}_{\mathsf{SUFCMA}}$ whose returned Boolean event defines an advantage probability.
Sig_ROM.ec:178	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
Sig_ROM.ec:188	module	`module A = A(H, O)`	EasyCrypt program module for the signature interface in the random-oracle model; its procedures denote probabilistic state transformers.
HAETAE_GameHops.ec:16	lemma	`lemma euf_cma_no_signing_exact &m :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_GameHops.ec:34	lemma	`lemma euf_cma_game_hop_bound &m :`	Probability bound $\Pr[\mathsf{eufcmagamehopbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_GameHops.ec:75	lemma	`lemma euf_cma_concrete_bound &m :`	Probability bound $\Pr[\mathsf{eufcmaconcretebound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec:23	module	`module type SigningSimulator(H : SIG.POracle) = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec:28	module	`module RealSigningSimulator(S : SIG.Scheme) (H : SIG.POracle) = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec:43	module	`module EUF_CMA_SimulatedSign(`	Experiment module $\mathcal{G}_{\mathsf{EUFCMASimulatedSign}}$ whose returned Boolean event defines an advantage probability.
HAETAE_HopGames.ec:51	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec:61	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec:81	op	`op transcript_log_consistent`	Total mathematical function or predicate $\mathsf{transcriptlogconsistent}$ in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec:88	lemma	`lemma transcript_log_consistent_nil md pk :`	Checked theorem $\mathsf{transcriptlogconsistentnil}$ in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec:95	lemma	`lemma transcript_log_consistent_cons md pk qs trs rs m ctx sig tr :`	Checked theorem $\mathsf{transcriptlogconsistentcons}$ in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec:114	lemma	`lemma transcript_log_consistent_no_min_entropy md pk qs trs rs :`	Checked theorem $\mathsf{transcriptlogconsistentnominentropy}$ in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec:124	lemma	`lemma transcript_log_consistent_valid md pk qs trs rs :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_HopGames.ec:134	op	`op transcript_log_ready (md : mode) (trs : transcript list) : bool =`	Total mathematical function or predicate $\mathsf{transcriptlogready}$ in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec:138	op	`op budgeted_programmed_query_count_discipline`	Numerical bound or budget parameter $\mathsf{budgetedprogrammedquerycountdiscipline}$ used to upper-bound a probability loss in the game-hop sequence, transcript games, and bad-event conditioning.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 145	op	`op budgeted_paper_sim_signing_count_discipline`	Numerical bound or budget parameter budgetedpapersimsigningcountdiscipline used to upper-bound a probability loss in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 150	op	`op budgeted_paper_sim_sampler_freshness_discipline`	Predicate/event budgetedpapersimsamplerfreshnessdiscipline on the game state; in probability expressions it denotes the indicator event $1_{\text{budgetedpapersimsamplerfreshnessdiscipline}}$. *Context:* game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 160	op	`op sampler_rom_covered`	Predicate/event samplerromcovered on the game state; in probability expressions it denotes the indicator event $1_{\text{samplerromcovered}}$. *Context:* game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 168	lemma	`lemma sampler_rom_covered_fresh_after_clean_seed`	Probability bound $\Pr[\text{samplerromcoveredfreshaftercleanseed}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 179	lemma	`lemma sampler_rom_covered_self_log_preserves`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 188	lemma	`lemma sampler_rom_covered_old_log_clean_boundary`	Probability bound $\Pr[\text{samplerromcoveredoldlogcleanboundary}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 205	op	`op budgeted_programmed_reprogram_discipline`	Numerical bound or budget parameter budgetedprogrammedreprogramdiscipline used to upper-bound a probability loss in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 212	op	`op budgeted_programmed_challenge_query_discipline`	Numerical bound or budget parameter budgetedprogrammedchallengequerydiscipline used to upper-bound a probability loss in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 218	lemma	`lemma transcript_log_consistent_ready md pk qs trs rs :`	Checked theorem transcriptlogconsistentready in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 229	lemma	`lemma transcript_log_valid_member md trs tr :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_HopGames.ec: 239	lemma	`lemma transcript_log_min_entropy_clear_member md trs tr :`	Checked theorem transcriptlogminentropyclearmember in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 249	lemma	`lemma transcript_log_ready_member_valid md trs tr :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_HopGames.ec: 259	lemma	`lemma transcript_log_ready_member_no_min_entropy md trs tr :`	Checked theorem transcriptlogreadymembernominentropy in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 269	lemma	`lemma transcript_log_ready_no_failure_for_matching_site md trs tr site :`	Checked theorem transcriptlogreadynofailureformatchingsite in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 280	lemma	`lemma transcript_log_ready_no_failure_for_programming_site md trs tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 293	lemma	`lemma transcript_log_ready_no_failure_for_signature_programming_site`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 308	lemma	`lemma transcript_log_ready_actual_signature_site_clear_member md trs tr :`	Checked theorem transcriptlogreadyactualsignaturesiteclearmember in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 322	lemma	`lemma transcript_log_ready_signature_sites_clear md trs :`	Checked theorem transcriptlogreadysignaturesitesclear in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 332	op	`op internal_transcript_state_sound`	Total mathematical function or predicate internaltranscriptstatesound in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 338	lemma	`lemma internal_transcript_state_sound_nil md sk :`	Checked theorem internaltranscriptstatesoundnil in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 348	lemma	`lemma internal_transcript_state_sound_keygen md sd :`	Checked theorem internaltranscriptstatesoundkeygen in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 360	lemma	`lemma internal_transcript_state_sound_no_min_entropy`	Checked theorem internaltranscriptstatesoundnominentropy in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 370	lemma	`lemma internal_transcript_state_sound_valid md pk sk qs trs rs :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_HopGames.ec: 379	lemma	`lemma internal_transcript_state_sound_log_ready md pk sk qs trs rs :`	Checked theorem internaltranscriptstatesoundlogready in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 388	lemma	`lemma transcript_log_consistent_sign_internal_cons`	Checked theorem transcriptlogconsistentsigninternalcons in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 406	lemma	`lemma internal_transcript_state_sound_sign_internal_cons`	Checked theorem internaltranscriptstatesoundsigninternalcons in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 428	lemma	`lemma sign_internal_transcript_no_min_entropy md pk sk m ctx coins :`	Checked theorem signinternaltranscriptnominentropy in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 442	module	`module EUF_CMA_TranscriptSimulatedSign(`	Experiment module $\mathcal{G}_{\text{EUFCMATranscriptSimulatedSign}}$ whose returned Boolean event defines an advantage probability.
HAETAE_HopGames.ec: 453	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 467	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 490	module	`module EUF_CMA_InternalTranscriptSign(`	Experiment module $\mathcal{G}_{\text{EUFCMAInternalTranscriptSign}}$ whose returned Boolean event defines an advantage probability.
HAETAE_HopGames.ec: 500	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 517	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 544	module	`module EUF_CMA_ROMInternalTranscriptSign(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 554	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 584	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 613	module	`module ROMInternalTranscriptAsNMA(A : SIG.Adversary) (H : SIG.POracle) = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 620	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 650	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 665	op	`op public_sim_source`	Total mathematical function or predicate publicsimsource in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 670	op	`op public_sim_response_aux_vector :`	Deterministic algebraic map or predicate publicsimresponseauxvector over HAETAE module/ring objects.
HAETAE_HopGames.ec: 676	op	`op public_sim_response_vector :`	Deterministic algebraic map or predicate publicsimresponsevector over HAETAE module/ring objects.
HAETAE_HopGames.ec: 682	op	`op public_sim_hint_vector :`	Deterministic algebraic map or predicate publicsimhintvector over HAETAE module/ring objects.
HAETAE_HopGames.ec: 688	op	`op public_sim_commitment_lowbits :`	Total mathematical function or predicate publicsimcommitmentlowbits in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 700	op	`op public_sim_commitment_challenge :`	Probability law or sampler publicsimcommitmentchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 708	op	`op public_sim_commitment_highbits :`	Total mathematical function or predicate publicsimcommitmenthighbits in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 716	op	`op public_sim_signature_token :`	Total mathematical function or predicate publicsimsignaturetoken in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 723	op	`op public_sim_signature :`	Total mathematical function or predicate publicsimsignature in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 733	lemma	`lemma public_sim_commitment_lowbitsE md sk m ctx coins :`	Checked theorem publicsimcommitmentlowbitsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 742	lemma	`lemma public_sim_response_aux_vectorE md sk m ctx coins :`	Checked theorem publicsimresponseauxvectorE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 750	lemma	`lemma public_sim_commitment_challengeE md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_HopGames.ec: 758	lemma	`lemma public_sim_commitment_highbitsE md sk m ctx coins :`	Checked theorem publicsimcommitmenthighbitsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 767	lemma	`lemma public_sim_response_vectorE md sk m ctx coins :`	Checked theorem publicsimresponsevectorE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 775	lemma	`lemma public_sim_hint_vectorE md sk m ctx coins :`	Checked theorem publicsimhintvectorE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 783	lemma	`lemma public_sim_signature_tokenE md sk m ctx coins :`	Checked theorem publicsimsignaturetokenE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 793	lemma	`lemma public_sim_signatureE md sk m ctx coins :`	Checked theorem publicsimsignatureE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 804	module	`module ROMInternalTranscriptPublicSimAsNMA(A : SIG.Adversary)`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 811	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 841	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 855	type	`type paper_sim_signature_sample = sig_token * poly * int.`	Signature space Σ; values of paper_sim_signature_sample denote candidate HAETAE signatures.
HAETAE_HopGames.ec: 857	op	`op paper_sim_sample_token (s : paper_sim_signature_sample) : sig_token =`	Probability law or sampler papersimsampletoken; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 859	op	`op paper_sim_sample_lowbits_carrier`	Probability law or sampler papersimsamplelowbitscarrier; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 862	op	`op paper_sim_sample_entropy (s : paper_sim_signature_sample) : int =`	Probability law or sampler papersimsampleentropy; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 864	op	`op paper_sim_sample_response (s : paper_sim_signature_sample) : polyvecl =`	Probability law or sampler papersimsampleresponse; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 866	op	`op paper_sim_sample_response_aux`	Probability law or sampler papersimsampleresponseaux; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 869	op	`op paper_sim_sample_hint (s : paper_sim_signature_sample) : hint_t =`	Probability law or sampler papersimsamplehint; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 872	op	`op paper_sim_commitment_lowbits`	Total mathematical function or predicate papersimcommitmentlowbits in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 880	op	`op paper_sim_commitment_challenge`	Probability law or sampler papersimcommitmentchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 888	op	`op paper_sim_commitment_highbits`	Total mathematical function or predicate papersimcommitmenthighbits in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 896	op	`op paper_sim_signature`	Total mathematical function or predicate papersimsignature in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 905	op	`op paper_sim_signature_sample_wf`	Probability law or sampler papersimsignaturesamplewf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 912	lemma	`lemma paper_sim_commitment_lowbits_wf md s :`	Checked theorem papersimcommitmentlowbitswf in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 918	lemma	`lemma paper_sim_signature_valid md pk m ctx s :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_HopGames.ec: 931	op	`op public_sim_lowbits_carrier`	Total mathematical function or predicate publicsimlowbitscarrier in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 937	op	`op paper_sim_sample_from_public_coins`	Probability law or sampler papersimsamplefrompubliccoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 944	lemma	`lemma public_sim_lowbits_carrier_wf md pk m ctx coins :`	Checked theorem publicsimlowbitscarrierwf in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 955	lemma	`lemma paper_sim_sample_from_public_coins_wf md pk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 970	lemma	`lemma paper_sim_sample_response_public_coinsE md pk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 978	lemma	`lemma paper_sim_sample_response_aux_public_coinsE md pk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 986	lemma	`lemma paper_sim_sample_hint_public_coinsE md pk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 994	lemma	`lemma paper_sim_sample_token_public_coinsE md pk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1001	lemma	`lemma paper_sim_commitment_lowbits_public_coinsE md pk m ctx coins :`	Checked theorem papersimcommitmentlowbitspubliccoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1012	lemma	`lemma paper_sim_commitment_challenge_public_coinsE md pk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_HopGames.ec: 1022	lemma	`lemma paper_sim_commitment_highbits_public_coinsE md pk m ctx coins :`	Checked theorem papersimcommitmenthighbitspubliccoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1032	lemma	`lemma paper_sim_signature_public_coinsE md pk m ctx coins :`	Checked theorem papersimsignaturepubliccoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1044	op	`op real_signing_lowbits_carrier`	Total mathematical function or predicate realsigninglowbitscarrier in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1049	op	`op paper_sim_sample_from_real_signing_coins`	Probability law or sampler papersimsamplefromrealsigningcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 1056	lemma	`lemma real_signing_lowbits_carrier_wf md sk m ctx coins :`	Checked theorem realsigninglowbitscarrierwf in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1065	lemma	`lemma paper_sim_sample_from_real_signing_coins_wf md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1078	lemma	`lemma paper_sim_sample_response_real_signing_coinsE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1086	lemma	`lemma paper_sim_sample_response_aux_real_signing_coinsE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1094	lemma	`lemma paper_sim_sample_hint_real_signing_coinsE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1102	lemma	`lemma paper_sim_sample_token_real_signing_coinsE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1109	lemma	`lemma paper_sim_commitment_lowbits_real_signing_coinsE md sk m ctx coins :`	Checked theorem papersimcommitmentlowbitsrealsigningcoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1120	lemma	`lemma paper_sim_commitment_challenge_real_signing_coinsE`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_HopGames.ec: 1131	lemma	`lemma paper_sim_commitment_highbits_real_signing_coinsE`	Checked theorem papersimcommitmenthighbitsrealsigningcoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1142	lemma	`lemma paper_sim_signature_real_signing_coinsE md sk m ctx coins :`	Checked theorem papersimsignaturerealsigningcoinsE in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1154	op	`op paper_sim_sample_from_rejection_attempt`	Probability law or sampler papersimsamplefromrejectionattempt; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 1160	op	`op paper_sim_abort_fallback_sample`	Probability law or sampler papersimabortfallbacksample; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 1164	op	`op haetae_rejection_sample_from_attempt`	Probability law or sampler haetaerejectionsamplefromattempt; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_HopGames.ec: 1171	lemma	`lemma paper_sim_abort_fallback_sample_wf md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1182	lemma	`lemma paper_sim_sample_from_rejection_attempt_of_coinsE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1195	lemma	`lemma paper_sim_commitment_lowbits_rejection_attemptE md st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1207	lemma	`lemma paper_sim_commitment_challenge_rejection_attemptE md pk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1225	lemma	`lemma paper_sim_commitment_highbits_rejection_attemptE md pk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1249	lemma	`lemma paper_sim_signature_rejection_attempt_fieldsE md pk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1277	lemma	`lemma paper_sim_signature_rejection_attempt_matches_state md pk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1293	lemma	`lemma paper_sim_signature_rejection_attempt_of_coinsE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 1306	lemma	`lemma haetae_rejection_sample_from_attempt_signatureE md pk m ctx st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1323	lemma	`lemma haetae_rejection_sample_from_attempt_no_fallback md pk m ctx st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1332	lemma	`lemma haetae_rejection_sample_from_attempt_of_coinsE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1343	lemma	`lemma haetae_rejection_sample_from_attempt_of_coins_wf md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1352	module	`module type PaperSimSigningSampler = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1359	module	`module ROMPaperSimSigningSampler(H : SIG.POracle) : PaperSimSigningSampler = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 1389	module	`module RealSigningPaperSimSampler(H : SIG.POracle) : PaperSimSigningSampler = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 1421	module	`module ROSigningAttemptPaperSimSampler(H : SIG.POracle)`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1456	module	`module HAETAERejectionPaperSimSampler(H : SIG.POracle)`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1492	module	`module ExactHyperballHAETAERejectionPaperSimSampler(H : SIG.POracle)`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1523	module	`module ExactHyperballPaperSimSampler(H : SIG.POracle)`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1554	module	`module ROExactHyperballPaperSimSampler(H : SIG.POracle)`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_HopGames.ec: 1587	module	`module StructuralAttemptPaperSample = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 1599	module	`module ExactHyperballPaperSample = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 1611	module	`module ROMInternalTranscriptPaperSimAsNMA(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 1620	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 1646	module	`module A = A(H, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 1661	module	`module ROMInternalTranscriptBudgetedPaperSimAsNMA(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 1675	module	`module AH = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 1695	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 1746	module	`module A = A(AH, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 1774	lemma	`lemma budgeted_paper_sim_sign_oracle_preserves_signing_count :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1775	hoare/equiv	`hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 1795	lemma	`lemma adversary_budgeted_paper_sim_signing_count_preserved :`	Checked theorem adversarybudgetedpapersimsigningcountpreserved in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 1796	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 1816	lemma	`lemma budgeted_paper_sim_main_signing_count_preserved :`	Theorem budgetedpapersimmainsigningcountpreserved contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 1817	hoare/equiv	`hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 1836	lemma	`lemma sampler_expand_query_point_bound q :`	Probability bound $\Pr[\text{samplerexpandquerypointbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1859	lemma	`lemma sampler_expand_query_log_fset_bound (qs :  ro_query list) :`	Probability bound $\Pr[\text{samplerexpandquerylogfsetbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1877	lemma	`lemma sampler_expand_query_log_budget_bound (qs :  ro_query list) hc :`	Probability bound $\Pr[\text{samplerexpandquerylogbudgetbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1896	lemma	`lemma sampler_expand_query_joint_log_budget_bound`	Probability bound $\Pr[\text{samplerexpandqueryjointlogbudgetbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1929	lemma	`lemma concrete_lazy_rom_sampler_expand_query_fresh_le`	Probability bound $\Pr[\text{concretelazyromsamplerexpandqueryfreshle}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1949	lemma	`lemma concrete_lazy_rom_sampler_expand_query_fresh_eq`	Probability bound $\Pr[\text{concretelazyromsamplerexpandqueryfresheq}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 1951	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 1964	lemma	`lemma concrete_lazy_rom_get_preserves_sampler_rom_covered`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1966	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 1982	lemma	`lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 1984	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 1999	lemma	`lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2000	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2024	lemma	`lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2026	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2039	lemma	`lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2041	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2054	lemma	`lemma sampler_bad_prequery_one_step_bound_nonnegative :`	Probability bound $\Pr[\text{samplerbadprequeryonestepboundnonnegative}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2062	lemma	`lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound pk &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 2155	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2166	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2202	lemma	`lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 2203	hoare/equiv	`phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2214	lemma	`lemma budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 2239	lemma	`lemma concrete_lazy_rom_sampler_expand_query_fresh_phoare`	Probability bound $\Pr[\mathsf{concretelazyromsamplerexpandqueryfreshphoare}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2241	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2252	lemma	`lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare`	Probability bound $\Pr[\mathsf{concretelazyromsamplerexpandqueryfreshargphoare}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2254	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2266	lemma	`lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare`	Probability bound $\Pr[\mathsf{concretelazyromsamplerexpandqueryfreshargcleanphoare}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2268	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2285	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le`	Probability bound $\Pr[\mathsf{concreterosigningattemptsamplewithseedfreshstructuralle}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2287	hoare/equiv	`phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2307	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le`	Probability bound $\Pr[\mathsf{concreterosigningattemptsamplewithseedfreshstructuralargle}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2309	hoare/equiv	`phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2330	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le`	Probability bound $\Pr[\mathsf{concreterosigningattemptsamplewithseedfreshstructuralargcleanle}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2332	hoare/equiv	`phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2355	lemma	`lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2357	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2382	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 2384	hoare/equiv	`phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2407	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq`	Probability bound $\Pr[\mathsf{concreterosigningattemptsamplewithseedfreshstructuraleq}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2409	hoare/equiv	`phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2429	lemma	`lemma concrete_ro_exact_hyperball_sample_with_seed_exact`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 2431	hoare/equiv	`phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2448	lemma	`lemma concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 2450	hoare/equiv	`phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Probabilistic Hoare judgment **phoare**$[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 2468	lemma	`lemma concrete_ro_exact_hyperball_sample_with_seed_exact_pr`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 2480	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2482	hoare/equiv	`hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2494	lemma	`lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2496	hoare/equiv	`hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2510	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2511	hoare/equiv	`hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2529	lemma	`lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2530	hoare/equiv	`hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2550	lemma	`lemma concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2551	hoare/equiv	`hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2577	lemma	`lemma concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2578	hoare/equiv	`hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2604	lemma	`lemma concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh :`	Probability bound $\Pr[\mathsf{concretebudgetedrosigningattemptosigncleansamplerfresh}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 2605	hoare/equiv	`hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 2624	module	`module EUF_CMA_ROMInternalTranscriptCountedSign(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2628	module	`module C = CountedLazyROM(H)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2636	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2667	module	`module A = A(C, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 2696	module	`module EUF_CMA_ROMProgrammedTranscriptCountedSign(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2700	module	`module C = CountedLazyROM(H)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2708	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2741	module	`module A = A(C, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2770	module	`module EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2774	module	`module C = CountedLazyROM(H)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2784	module	`module AH = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2803	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2843	module	`module A = A(AH, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2874	module	`module EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2879	module	`module C = CountedLazyROM(H)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2889	module	`module AH = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2908	module	`module O = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_HopGames.ec: 2948	module	`module A = A(AH, O)`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 2986	lemma	`lemma internal_transcript_sign_oracle_preserves_state :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 2987	hoare/equiv	`hoare[EUF_CMA_InternalTranscriptSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3016	lemma	`lemma adversary_internal_transcript_state_preserved :`	Checked theorem adversaryinternaltranscriptstatepreserved in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 3017	hoare/equiv	`hoare[A(H, EUF_CMA_InternalTranscriptSign(H, A).O).forge :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3044	lemma	`lemma internal_transcript_sign_main_state_sound :`	Theorem internaltranscriptsignmainstatesound contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3045	hoare/equiv	`hoare[EUF_CMA_InternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3076	lemma	`lemma internal_transcript_sign_main_transcripts_valid :`	Theorem internaltranscriptsignmaintranscriptsvalid contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3077	hoare/equiv	`hoare[EUF_CMA_InternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3118	lemma	`lemma internal_transcript_sign_main_log_ready :`	Theorem internaltranscriptsignmainlogready contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3119	hoare/equiv	`hoare[EUF_CMA_InternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3160	lemma	`lemma internal_transcript_sign_main_no_min_entropy :`	Theorem internaltranscriptsignmainnominentropy contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3161	hoare/equiv	`hoare[EUF_CMA_InternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3202	lemma	`lemma rom_internal_transcript_sign_oracle_preserves_state :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3203	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3238	lemma	`lemma adversary_rom_internal_transcript_state_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3239	hoare/equiv	`hoare[A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).O).forge :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3266	lemma	`lemma rom_internal_transcript_sign_main_state_sound :`	Theorem rominternaltranscriptsignmainstatesound contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3267	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3300	lemma	`lemma rom_internal_transcript_sign_main_transcripts_valid :`	Theorem rominternaltranscriptsignmaintranscriptsvalid contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3301	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3344	lemma	`lemma rom_internal_transcript_sign_main_log_ready :`	Theorem rominternaltranscriptsignmainlogready contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3345	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3388	lemma	`lemma rom_internal_transcript_sign_main_no_min_entropy :`	Theorem rominternaltranscriptsignmainnominentropy contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3389	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3432	lemma	`lemma rom_internal_transcript_sign_main_no_failure_for_signature_site :`	Theorem rominternaltranscriptsignmainnofailureforsignaturesite contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3433	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 3485	lemma	`lemma rom_internal_transcript_sign_main_signature_sites_clear :`	Theorem rominternaltranscriptsignmainsignaturesitesclear contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3486	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3531	lemma	`lemma rom_internal_transcript_bad_forgery_zero &m :`	Probability bound $\Pr[\text{rominternaltranscriptbadforgeryzero}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3539	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3548	lemma	`lemma rom_internal_transcript_bad_forgery_bound &m :`	Probability bound $\Pr[\text{rominternaltranscriptbadforgerybound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3564	lemma	`lemma rom_internal_transcript_bad_forgery_counted_bound &m :`	Probability bound $\Pr[\text{rominternaltranscriptbadforgerycountedbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3583	lemma	`lemma counted_rom_internal_transcript_sign_oracle_preserves_budget_state :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3584	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3653	lemma	`lemma adversary_counted_rom_internal_transcript_budget_state_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3654	hoare/equiv	`hoare[A(EUF_CMA_ROMInternalTranscriptCountedSign(H, A).C,`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3694	lemma	`lemma counted_rom_internal_transcript_main_bad_clear :`	Theorem countedrominternaltranscriptmainbadclear contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3695	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3733	lemma	`lemma counted_rom_internal_transcript_main_bad_false :`	Theorem countedrominternaltranscriptmainbadfalse contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3734	hoare/equiv	`hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3740	lemma	`lemma counted_rom_internal_transcript_counted_bad_zero &m :`	Probability bound $\Pr[\text{countedrominternaltranscriptcountedbadzero}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3745	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3751	lemma	`lemma counted_rom_internal_transcript_budget_sound &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3761	lemma	`lemma counted_rom_internal_transcript_counted_bad_subunit &m :`	Probability bound $\Pr[\text{countedrominternaltranscriptcountedbadsubunit}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3778	lemma	`lemma programmed_counted_sign_oracle_preserves_min_entropy_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3779	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3818	lemma	`lemma adversary_programmed_counted_min_entropy_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3819	hoare/equiv	`hoare[A(EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C,`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3841	lemma	`lemma programmed_counted_main_min_entropy_clear :`	Theorem programmedcountedmainminentropyclear contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 3842	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3869	lemma	`lemma programmed_counted_min_entropy_bad_zero &m :`	Probability bound $\Pr[\text{programmedcountedminentropybadzero}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3875	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3879	lemma	`lemma programmed_counted_min_entropy_bad_bound &m :`	Probability bound $\Pr[\text{programmedcountedminentropybadbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3888	lemma	`lemma programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr &m :`	Probability bound $\Pr[\text{programmedcountedprequeryreprogrammingbadboundfrombudgetexpr}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3915	lemma	`lemma budgeted_counted_lazy_rom_get_true :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3916	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3925	lemma	`lemma budgeted_counted_lazy_rom_init_true :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3926	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.init :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3935	lemma	`lemma budgeted_counted_lazy_rom_bad_true :`	Probability bound $\Pr[\text{budgetedcountedlazyrombadtrue}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 3936	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.bad :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3942	lemma	`lemma budgeted_programmed_hash_get_preserves_challenge_query_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3943	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3963	lemma	`lemma budgeted_programmed_sign_oracle_preserves_challenge_query_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3964	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 3989	lemma	`lemma adversary_budgeted_programmed_challenge_query_discipline_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 3990	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4011	lemma	`lemma budgeted_programmed_main_challenge_query_discipline :`	Theorem budgetedprogrammedmainchallengequerydiscipline contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4012	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 4042	lemma	`lemma budgeted_programmed_main_challenge_query_count_bound :`	Theorem budgetedprogrammedmainchallengequerycountbound contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4043	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4054	lemma	`lemma budgeted_programmed_adaptive_prequery_lift`	Probability bound $\Pr[\text{budgetedprogrammedadaptiveprequerylift}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4070	lemma	`lemma budgeted_programmed_adaptive_prequery_lift_checked_31`	Probability bound $\Pr[\text{budgetedprogrammedadaptiveprequeryliftchecked31}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4089	lemma	`lemma budgeted_programmed_hash_get_preserves_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4090	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4119	lemma	`lemma budgeted_programmed_reprogram_discipline_after_actual_signature`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4157	lemma	`lemma budgeted_programmed_sign_oracle_preserves_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4158	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4223	lemma	`lemma adversary_budgeted_programmed_discipline_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4224	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4260	lemma	`lemma budgeted_programmed_main_reprogram_clear :`	Theorem budgetedprogrammedmainreprogramclear contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4261	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4310	lemma	`lemma budgeted_programmed_bad_reprogram_zero &m :`	Probability bound $\Pr[\text{budgetedprogrammedbadreprogramzero}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4317	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4321	lemma	`lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m :`	Probability bound $\Pr[\text{budgetedprogrammedprequeryreprogrambadboundfromprequery}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4345	lemma	`lemma budgeted_programmed_hash_get_preserves_query_budget :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4346	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4368	lemma	`lemma budgeted_programmed_sign_oracle_preserves_query_budget :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4369	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4399	lemma	`lemma adversary_budgeted_programmed_query_budget_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4400	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4430	lemma	`lemma budgeted_programmed_main_query_budget_preserved :`	Theorem budgetedprogrammedmainquerybudgetpreserved contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4431	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4467	lemma	`lemma budgeted_programmed_query_budget_certifies_branch_bound &m :`	Probability bound $\Pr[\text{budgetedprogrammedquerybudgetcertifiesbranchbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4483	lemma	`lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery`	Probability bound $\Pr[\text{budgetedprogrammedquerybudgetcertifiesbranchboundfromprequery}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4500	lemma	`lemma budgeted_programmed_query_budget_certifies_checked_31_bound`	Probability bound $\Pr[\text{budgetedprogrammedquerybudgetcertifieschecked31bound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4529	lemma	`lemma budgeted_sampled_direct_counted_lazy_rom_get_true :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4530	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4540	lemma	`lemma budgeted_sampled_direct_counted_lazy_rom_init_true :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4541	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4551	lemma	`lemma budgeted_sampled_direct_counted_lazy_rom_bad_true :`	Probability bound $\Pr[\text{budgetedsampleddirectcountedlazyrombadtrue}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4552	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4559	lemma	`lemma budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4560	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4582	lemma	`lemma budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4583	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4612	lemma	`lemma adversary_budgeted_sampled_direct_challenge_query_discipline_preserved :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4613	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4636	lemma	`lemma budgeted_sampled_direct_main_challenge_query_discipline :`	Theorem budgetedsampleddirectmainchallengequerydiscipline contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4637	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4668	lemma	`lemma budgeted_sampled_direct_hash_get_preserves_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4669	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 4699	lemma	`lemma budgeted_sampled_direct_sign_oracle_preserves_discipline :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 4700	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4769	lemma	`lemma adversary_budgeted_sampled_direct_discipline_preserved :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 4770	hoare/equiv	`hoare[`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4808	lemma	`lemma budgeted_sampled_direct_main_reprogram_clear :`	Theorem budgetedsampleddirectmainreprogramclear contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_HopGames.ec: 4809	hoare/equiv	`hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4859	lemma	`lemma budgeted_sampled_direct_bad_reprogram_zero &m :`	Probability bound $\Pr[\text{budgetedsampleddirectbadreprogramzero}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4867	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4871	lemma	`lemma budgeted_sampled_direct_one_step_bound_nonnegative :`	Probability bound $\Pr[\text{budgetedsampleddirectonestepboundnonnegative}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4881	lemma	`lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound`	Probability bound $\Pr[\text{budgetedsampleddirectsamplermessagehashprequerybound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4883	hoare/equiv	`phoare[DirectSigningCoinSampler.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 4920	lemma	`lemma budgeted_sampled_direct_bad_prequery_bound &m :`	Probability bound $\Pr[\text{budgetedsampleddirectbadprequerybound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 4993	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 4999	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 5130	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 5141	hoare/equiv	`hoare.`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 5182	lemma	`lemma budgeted_sampled_direct_prequery_reprogram_bad_bound &m :`	Probability bound $\Pr[\text{budgetedsampleddirectprequeryreprogrambadbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5213	lemma	`lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31 &m :`	Probability bound $\Pr[\text{budgetedsampleddirectprequeryreprogrambadchecked31}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5241	hoare/equiv	`equiv budgeted_programmed_counted_sampled_direct_hash_get_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5295	hoare/equiv	`equiv budgeted_programmed_counted_sampled_direct_sign_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5380	hoare/equiv	`equiv adversary_budgeted_programmed_counted_sampled_direct_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5480	hoare/equiv	`equiv budgeted_programmed_counted_sampled_direct_main_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5572	lemma	`lemma budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact`	Probability bound $\Pr[\text{budgetedprogrammedcountedsampleddirectprequeryreprogrambadexact}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5585	lemma	`lemma budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler`	Probability bound $\Pr[\text{budgetedprogrammedprequeryreprogrambadchecked31viadirectsampler}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5608	hoare/equiv	`equiv haetae_rom_internal_oracle_erasure :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5637	hoare/equiv	`equiv adversary_haetae_rom_internal_erasure :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5672	lemma	`lemma haetae_rom_internal_transcript_erasure_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 5713	hoare/equiv	`equiv rom_internal_oracle_structural_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5755	hoare/equiv	`equiv adversary_rom_internal_structural_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5811	lemma	`lemma rom_internal_transcript_structural_nma_bound &m :`	Probability bound $\Pr[\text{rominternaltranscriptstructuralnmabound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5858	lemma	`lemma rom_internal_transcript_clear_site_structural_nma_bound &m :`	Probability bound $\Pr[\text{rominternaltranscriptclearsitestructuralnmabound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 5881	hoare/equiv	`equiv rom_internal_public_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5927	hoare/equiv	`equiv adversary_rom_internal_public_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 5988	lemma	`lemma rom_internal_nma_public_sim_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 6050	hoare/equiv	`equiv rom_internal_public_sim_rom_paper_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6097	hoare/equiv	`equiv adversary_rom_internal_public_sim_rom_paper_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6158	lemma	`lemma rom_internal_nma_public_sim_rom_paper_sim_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 6221	hoare/equiv	`equiv rom_internal_real_signing_paper_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6278	hoare/equiv	`equiv adversary_rom_internal_real_signing_paper_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6354	lemma	`lemma rom_internal_nma_real_signing_paper_sim_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 6427	hoare/equiv	`equiv real_signing_ro_attempt_paper_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6485	hoare/equiv	`equiv adversary_real_signing_ro_attempt_paper_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 6548	lemma	`lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 6615	op	`op structural_to_exact_hyperball_paper_sample_loss_obligation`	Numerical bound or budget parameter structuraltoexacthyperballpapersamplelossobligation used to upper-bound a probability loss in the game-hop sequence, transcript games, and bad-event conditioning.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 6625	lemma	`lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss`	Probability bound $\Pr[\mathsf{concreterosigningattempttoexacthyperballsamplewithseedfreshloss}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6655	lemma	`lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss`	Probability bound $\Pr[\mathsf{concreterosigningattempttoexacthyperballsamplewithseedcleanloss}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6678	lemma	`lemma concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface`	Probability bound $\Pr[\mathsf{concretebudgetedosignsamplewithseedcleanlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6708	lemma	`lemma concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface`	Probability bound $\Pr[\mathsf{concretebudgetedosignoldlogsamplewithseedcleanlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6740	lemma	`lemma concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface`	Probability bound $\Pr[\mathsf{concretebudgetedosignoldlogsignaturefromsamplecleanlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6779	lemma	`lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface`	Probability bound $\Pr[\mathsf{concretebudgetedosignoldlogsignaturecleaneventlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6818	lemma	`lemma concrete_frozen_old_log_sample_clean_event_loss_surface`	Probability bound $\Pr[\mathsf{concretefrozenoldlogsamplecleaneventlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6845	lemma	`lemma concrete_frozen_old_log_signature_clean_event_loss_surface`	Probability bound $\Pr[\mathsf{concretefrozenoldlogsignaturecleaneventlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6875	lemma	`lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface`	Probability bound $\Pr[\mathsf{concretefrozenoldlogselfloggedsignaturecleaneventlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6913	lemma	`lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface`	Probability bound $\Pr[\mathsf{concretebudgetedselfloggedsignaturecleaneventlosssurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6956	lemma	`lemma concrete_lazy_rom_get_lossless :`	Probability bound $\Pr[\mathsf{concretelazyromgetlossless}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6965	lemma	`lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss`	Probability bound $\Pr[\mathsf{structuralattemptexacthyperballpapersampleonesteploss}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 6968	hoare/equiv	`phoare[StructuralAttemptPaperSample.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 6986	lemma	`lemma exact_hyperball_paper_sample_one_step_upper`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 6988	hoare/equiv	`phoare[ExactHyperballPaperSample.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7016	lemma	`lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7036	lemma	`lemma rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound &m :`	Probability bound $\Pr[\mathsf{rominternalbudgetedrosigningattemptsigningcalllossbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 7057	lemma	`lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7071	hoare/equiv	`phoare[StructuralAttemptPaperSample.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7097	lemma	`lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7154	lemma	`lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7183	lemma	`lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7205	lemma	`lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7243	lemma	`lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7291	lemma	`lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound &m :`	Probability bound $\Pr[\mathsf{rominternalnmarosigningattemptroexacthyperballlossbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 7326	lemma	`lemma concrete_lazy_rom_get_preserves_signature_event`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7328	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7338	lemma	`lemma concrete_lazy_rom_get_preserves_signature_event_with_pk`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7340	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7352	lemma	`lemma concrete_lazy_rom_get_preserves_current_signature_event`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7354	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7366	lemma	`lemma concrete_lazy_rom_get_true :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7367	hoare/equiv	`phoare[HAETAE_RO.FRO.get :  true ==> true] = 1%r.`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7372	lemma	`lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7374	hoare/equiv	`phoare[HAETAE_RO.FRO.get :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7388	lemma	`lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 7390	hoare/equiv	`hoare[HAETAE_RO.FRO.get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_HopGames.ec: 7404	lemma	`lemma concrete_ro_signing_attempt_sample_with_seed_lossless :`	Probability bound $\Pr[\mathsf{concreterosigningattemptsamplewithseedlossless}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 7413	lemma	`lemma concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le`	Checked theorem $\mathsf{concretebudgetedrosigningattemptosignactivecleansignaturestructuralle}$ in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 7415	hoare/equiv	`phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7487	lemma	`lemma concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 7489	hoare/equiv	`phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_HopGames.ec: 7541	lemma	`lemma concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 7585	lemma	`lemma concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 7673	lemma	`lemma concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface`	Probability bound $\Pr[\mathsf{concretebudgetedosigncleanonecalllossfromselfloggedsurface}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 7733	lemma	`lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7767	lemma	`lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7795	lemma	`lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :`	Game-equivalence/fundamental-lemma statement: two experiments have equal probabilities under the clean invariant, or differ only by a stated bad-event probability.
HAETAE_HopGames.ec: 7828	hoare/equiv	`equiv real_signing_rejection_paper_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 7886	hoare/equiv	`equiv adversary_real_signing_rejection_paper_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 7963	lemma	`lemma rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8039	hoare/equiv	`equiv exact_hyperball_haetae_rejection_paper_sim_nma_oracle :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8101	hoare/equiv	`equiv adversary_exact_hyperball_haetae_rejection_paper_sim_nma :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8164	lemma	`lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8241	lemma	`lemma public_sim_to_paper_sim_nma_hop_from_sampling_hop &m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8268	hoare/equiv	`equiv transcript_logging_oracle_erasure :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8286	hoare/equiv	`equiv adversary_transcript_logging_erasure :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8309	lemma	`lemma transcript_logging_erasure_exact &m :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8350	hoare/equiv	`equiv real_signing_oracle_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8369	hoare/equiv	`equiv adversary_real_signing_equiv :`	Relational judgment $\mathbf{equiv}[c_1 \sim c_2 : P \Rightarrow Q]$ proving preservation of a coupling between two games.
HAETAE_HopGames.ec: 8393	lemma	`lemma real_signing_simulator_exact &m :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8424	module	`module type ForkingAdversary(H : SIG.POracle) = {`	Adversary interface/module $\mathcal{A}$; its procedures are quantified in the security theorem subject to call budgets.
HAETAE_HopGames.ec: 8428	module	`module type TranscriptExtractor = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 8433	module	`module type BimodalTranscriptExtractor = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 8438	module	`module BimodalSpecialSoundness_Game(`	Experiment module $\mathcal{G}_{\mathsf{BimodalSpecialSoundnessGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_HopGames.ec: 8463	module	`module SpecialSoundness_Game(`	Experiment module $\mathcal{G}_{\mathsf{SpecialSoundnessGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_HopGames.ec: 8488	module	`module BimodalTranscriptExtractor_As_ModuleSIS(`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 8507	lemma	`lemma bimodal_special_soundness_lift_exact &m md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8535	module	`module type BimodalToModuleSIS = {`	EasyCrypt program module for the game-hop sequence, transcript games, and bad-event conditioning; its procedures denote probabilistic state transformers.
HAETAE_HopGames.ec: 8541	op	`op special_soundness_interfaces_ready (md : mode) : bool =`	Total mathematical function or predicate specialsoundnessinterfacesready in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8546	lemma	`lemma special_soundness_interfaces_ready_from_forking md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8559	lemma	`lemma special_soundness_interfaces_public_reconstruction md :`	Checked theorem specialsoundnessinterfacespublicreconstruction in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 8568	lemma	`lemma special_soundness_interfaces_ready_sound md :`	Checked theorem specialsoundnessinterfacesreadysound in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 8577	lemma	`lemma special_soundness_interfaces_ready_sound_with_public_reconstruction md :`	Checked theorem specialsoundnessinterfacesreadysoundwithpublicreconstruction in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 8587	op	`op signing_simulator_sound (md : mode) : bool =`	Total mathematical function or predicate signingsimulatorsound in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8592	lemma	`lemma signing_simulator_sound_current md :`	Checked theorem signingsimulatorsoundcurrent in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 8600	lemma	`lemma rejection_sampling_bound_from_fs_bound :`	Probability bound $\Pr[\mathsf{rejectionsamplingboundfromfsbound}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8615	op	`op fs_with_aborts_interfaces_ready (md : mode) : bool =`	Total mathematical function or predicate fswithabortsinterfacesready in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8620	lemma	`lemma fs_with_aborts_interfaces_ready_from_parts md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8634	lemma	`lemma fs_with_aborts_interfaces_ready_from_forking md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_HopGames.ec: 8644	lemma	`lemma fs_with_aborts_interfaces_ready_holds md :`	Checked theorem fswithabortsinterfacesreadyholds in the game-hop sequence, transcript games, and bad-event conditioning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_HopGames.ec: 8651	lemma	`lemma structural_to_exact_hyperball_paper_sample_loss_from_obligation`	Probability bound $\Pr[\mathsf{structuraltoexacthyperballpapersamplelossfromobligation}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8667	lemma	`lemma structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss`	Probability bound $\Pr[\mathsf{structuraltoexacthyperballpapersamplelossfromattemptloss}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8684	lemma	`lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss`	Probability bound $\Pr[\mathsf{structuraltoexacthyperballpapersamplelossobligationfromattemptloss}] \le B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_HopGames.ec: 8696	lemma	`lemma structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss`	Probability bound Pr[structuraltoexacthyperballpapersamplelossfromcoinsamplepairloss] $\leq B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8712	lemma	`lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss`	Probability bound Pr[structuraltoexacthyperballpapersamplelossobligationfromcoinsamplepairloss] $\leq B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8724	lemma	`lemma dexact_hyperball_reference_paper_sim_attempt_boundary_ready`	Probability bound Pr[dexacthyperballreferencepapersimattemptboundaryready] $\leq B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8732	lemma	`lemma dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready`	Probability bound Pr[dexacthyperballfullrejectionpapersimattemptboundaryready] $\leq B$ or loss inequality controlling a bad event in the game-hop sequence, transcript games, and bad-event conditioning.
HAETAE_HopGames.ec: 8741	lemma	`lemma dexact_hyperball_haetae_rejection_sample_no_fallback_mu`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8758	lemma	`lemma dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8777	lemma	`lemma dexact_hyperball_paper_sim_rejection_attempt_signature_mu`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_HopGames.ec: 8800	lemma	`lemma dexact_hyperball_haetae_rejection_sample_signature_mu`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Reductions.ec:9	op	`op mlwe_hardness_term : real.`	Total mathematical function or predicate mlwehardnessterm in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 10	op	`op module_sis_hardness_term : real.`	Total mathematical function or predicate modulesishardnessterm in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 11	op	`op bimodal_to_msis_reduction_loss_term : real = 0%r.`	Numerical bound or budget parameter bimodaltomsisreductionlossterm used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 13	op	`op bimodal_selftarget_msis_hardness_term =`	Total mathematical function or predicate bimodalselftargetmsishardnessterm in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 16	op	`op signature_query_count : real = 2%r.`	Oracle-related function signaturequerycount used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 17	op	`op hash_query_count : real = 2%r.`	Oracle-related function hashquerycount used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 18	op	`op signature_query_budget_count : int = 2.`	Numerical bound or budget parameter signaturequerybudgetcount used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 19	op	`op hash_query_budget_count : int = 2.`	Numerical bound or budget parameter hashquerybudgetcount used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 20	op	`op abort_probability : real = 0%r.`	Total mathematical function or predicate abortprobability in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 21	op	`op min_entropy_loss_factor : real = 2%r.`	Numerical bound or budget parameter minentropylossfactor used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 22	op	`op challenge_support_bits_lower_bound : int = 58.`	Numerical bound or budget parameter challengesupportbitslowerbound used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 23	op	`op challenge_support_cardinality_lower_bound : real =`	Numerical bound or budget parameter challengesupportcardinalitylowerbound used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 26	op	`op rom_hash_query_budget : real = hash_query_count.`	Numerical bound or budget parameter romhashquerybudget used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 27	op	`op rom_signature_query_budget : real = signature_query_count.`	Numerical bound or budget parameter romsignaturequerybudget used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 28	op	`op rom_total_query_budget =`	Numerical bound or budget parameter romtotalquerybudget used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 31	op	`op counted_rom_collision_term =`	Oracle-related function countedromcollisionterm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 35	op	`op counted_rom_prequery_term =`	Oracle-related function countedromprequeryterm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 39	op	`op counted_rom_min_entropy_term =`	Oracle-related function countedrommminentropyterm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 43	op	`op counted_rom_prequery_reprogramming_term =`	Oracle-related function countedromprequeryreprogrammingterm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 46	op	`op counted_rom_programming_loss_term =`	Numerical bound or budget parameter countedromprogramminglossterm used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 51	op	`op signing_query_scale =`	Oracle-related function signingqueryscale used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Reductions.ec: 54	op	`op fs_with_aborts_reprogramming_term =`	Total mathematical function or predicate fswithabortsreprogrammingterm in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 58	op	`op fs_with_aborts_min_entropy_term =`	Total mathematical function or predicate fswithabortsminentropyterm in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 62	op	`op rejection_sampling_loss_term =`	Numerical bound or budget parameter rejectionsamplinglossterm used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 65	op	`op haetae_euf_bound =`	Numerical bound or budget parameter haetaeeufbound used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 70	op	`op haetae_euf_counted_rom_bound =`	Numerical bound or budget parameter haetaeeufcountedrombound used to upper-bound a probability loss in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 75	lemma	`lemma haetae_euf_boundE :`	Probability bound Pr[haetaeeufboundE] $\leq B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 87	lemma	`lemma haetae_euf_bound_groupedE :`	Probability bound Pr[haetaeeufboundgroupedE] $\leq B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 98	lemma	`lemma haetae_euf_bound_rejection_groupedE :`	Probability bound Pr[haetaeeufboundrejectiongroupedE] $\leq B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 107	lemma	`lemma haetae_euf_counted_rom_bound_groupedE :`	Probability bound Pr[haetaeeufcountedromboundgroupedE] $\leq B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 117	lemma	`lemma signing_query_scaleE :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 123	lemma	`lemma signing_query_scale_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 129	lemma	`lemma fs_with_aborts_reprogramming_term_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 140	lemma	`lemma fs_with_aborts_min_entropy_term_nonnegative :`	Checked theorem fswithabortsminentropytermnonnegative in the reduction from a forgery event to the assumed hard problem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Reductions.ec: 151	lemma	`lemma rom_hash_query_budget_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Reductions.ec: 155	lemma	`lemma rom_signature_query_budget_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 159	lemma	`lemma hash_query_budget_countE :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 163	lemma	`lemma signature_query_budget_countE :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 167	lemma	`lemma hash_query_budget_count_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 171	lemma	`lemma signature_query_budget_count_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 175	lemma	`lemma min_entropy_loss_factor_nonnegative :`	Probability bound $\Pr[\mathsf{minentropylossfactornonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 179	lemma	`lemma challenge_support_cardinality_lower_bound_positive :`	Probability bound $\Pr[\mathsf{challengesupportcardinalitylowerboundpositive}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 183	lemma	`lemma challenge_support_cardinality_lower_bound_nonnegative :`	Probability bound $\Pr[\mathsf{challengesupportcardinalitylowerboundnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 189	lemma	`lemma rom_total_query_budget_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 200	lemma	`lemma counted_rom_collision_term_nonnegative :`	Probability bound $\Pr[\mathsf{countedromcollisiontermnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 209	lemma	`lemma counted_rom_prequery_term_nonnegative :`	Probability bound $\Pr[\mathsf{countedromprequerytermnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 220	lemma	`lemma counted_rom_min_entropy_term_nonnegative :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 231	lemma	`lemma counted_rom_prequery_reprogramming_term_nonnegative :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingtermnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 240	lemma	`lemma counted_rom_programming_loss_term_nonnegative :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 251	lemma	`lemma counted_rom_collision_termE :`	Probability bound $\Pr[\mathsf{countedromcollisiontermE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 263	lemma	`lemma counted_rom_prequery_termE :`	Probability bound $\Pr[\mathsf{countedromprequerytermE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 274	lemma	`lemma counted_rom_min_entropy_termE :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 285	lemma	`lemma counted_rom_prequery_reprogramming_termE :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingtermE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 294	lemma	`lemma counted_rom_programming_loss_termE :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 304	lemma	`lemma counted_rom_programming_loss_term_subunit :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermsubunit}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 313	lemma	`lemma counted_rom_programming_loss_term_positive :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermpositive}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 322	lemma	`lemma counted_rom_programming_loss_term_le1 :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermle1}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 328	lemma	`lemma counted_rom_prequery_reprogramming_budget_numeratorE :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingbudgetnumeratorE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 340	lemma	`lemma counted_rom_budget_numeratorE :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 353	lemma	`lemma counted_rom_programming_loss_term_cardinalityE :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermcardinalityE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 364	lemma	`lemma counted_rom_prequery_reprogramming_term_cardinalityE :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingtermcardinalityE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 374	lemma	`lemma counted_rom_programming_loss_term_concreteE :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermconcreteE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 382	lemma	`lemma counted_rom_prequery_reprogramming_term_concreteE :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingtermconcreteE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 390	lemma	`lemma counted_rom_programming_loss_term_checked_subunit :`	Probability bound $\Pr[\mathsf{countedromprogramminglosstermcheckedsubunit}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 403	lemma	`lemma counted_rom_support_lower_bound_summary :`	Probability bound $\Pr[\mathsf{countedromsupportlowerboundsummary}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 413	lemma	`lemma counted_rom_support_dominates_budget :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Reductions.ec: 417	lemma	`lemma counted_rom_prequery_reprogramming_support_dominates_budget :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingsupportdominatesbudget}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 421	lemma	`lemma counted_rom_prequery_reprogramming_term_subunit :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingtermsubunit}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 430	lemma	`lemma counted_rom_prequery_reprogramming_from_query_budgets_and_support :`	Probability bound $\Pr[\mathsf{countedromprequeryreprogrammingfromquerybudgetsandsupport}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 445	lemma	`lemma counted_rom_programming_loss_splitE :`	Probability bound $\Pr[\mathsf{countedromprogramminglosssplitE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 454	lemma	`lemma counted_rom_loss_from_query_budgets_and_support :`	Probability bound $\Pr[\mathsf{countedromlossfromquerybudgetsandsupport}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 471	lemma	`lemma rejection_sampling_loss_term_nonnegative :`	Probability bound $\Pr[\mathsf{rejectionsamplinglosstermnonnegative}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 479	lemma	`lemma fs_with_aborts_min_entropy_termE :`	Checked theorem $\mathsf{fswithabortsminentropytermE}$ in the reduction from a forgery event to the assumed hard problem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Reductions.ec: 486	lemma	`lemma fs_with_aborts_min_entropy_term_ge1 :`	Checked theorem $\mathsf{fswithabortsminentropytermge1}$ in the reduction from a forgery event to the assumed hard problem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Reductions.ec: 492	lemma	`lemma rejection_sampling_loss_term_ge1 :`	Probability bound $\Pr[\mathsf{rejectionsamplinglosstermge1}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_Reductions.ec: 500	lemma	`lemma bimodal_to_msis_reduction_loss_termE :`	Probability bound $\Pr[\mathsf{bimodaltomsisreductionlosstermE}] \le B$ or loss inequality controlling a bad event in the reduction from a forgery event to the assumed hard problem.
HAETAE_ROM.ec:10	type	`type ro_query =`	Transcript/query carrier used to define sets Q_H of random-oracle queries and logged signing transcripts.
HAETAE_ROM.ec:15	type	`type ro_output = crh * challenge * matrix * random_coins.`	Carrier set $\mathsf{rooutput}$ used in the concrete lazy random-oracle state and oracle semantics; EasyCrypt equality is the mathematical equality on this set.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_ROM.ec:17	op	`op message_hash_query (pk : pkey) (ctx : context) (m : message) : ro_query =`	Oracle-related function messagehashquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM.ec:19	op	`op challenge_hash_query`	Probability law or sampler challengehashquery; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:22	op	`op matrix_expand_query (md : mode) (sd : seed) : ro_query =`	Oracle-related function matrixexpandquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM.ec:24	op	`op sampler_expand_query (coins : random_coins) : ro_query =`	Probability law or sampler samplerexpandquery; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:27	op	`op ro_crh_zero : crh = nseq crhbytes 0.`	Total mathematical function or predicate rocrhzero in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:28	op	`op ro_challenge_zero : challenge = poly_zero.`	Probability law or sampler rochallengezero; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:29	op	`op ro_matrix_zero : matrix = [].`	Deterministic algebraic map or predicate romatrixzero over HAETAE module/ring objects.
HAETAE_ROM.ec:30	op	`op ro_random_coins_zero : random_coins = nseq seedbytes 0.`	Probability law or sampler rorandomcoinszero; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:31	op	`op ro_output_zero : ro_output =`	Total mathematical function or predicate rooutputzero in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:34	op	`op ro_output_to_crh (y : ro_output) : crh = y.‘1.`	Total mathematical function or predicate rooutputtocrh in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:35	op	`op ro_output_to_challenge (y : ro_output) : challenge = y.‘2.`	Probability law or sampler rooutputtochallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:36	op	`op ro_output_to_matrix (y : ro_output) : matrix = y.‘3.`	Deterministic algebraic map or predicate rooutputtomatrix over HAETAE module/ring objects.
HAETAE_ROM.ec:37	op	`op ro_output_to_random_coins (y : ro_output) : random_coins = y.‘4.`	Probability law or sampler rooutputtorandomcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:39	op	`op ro_output_of_crh (mu : crh) : ro_output =`	Total mathematical function or predicate rooutputofcrh in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:41	op	`op ro_output_of_challenge (ch : challenge) : ro_output =`	Probability law or sampler rooutputofchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:43	op	`op ro_output_of_matrix (a : matrix) : ro_output =`	Deterministic algebraic map or predicate rooutputofmatrix over HAETAE module/ring objects.
HAETAE_ROM.ec:45	op	`op ro_output_of_random_coins (coins : random_coins) : ro_output =`	Probability law or sampler rooutputofrandomcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:48	op	`op dmatrix_for_query (x : mode * seed) : matrix distr = dmatrix x.‘1.`	Oracle-related function dmatrixforquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM.ec:50	op	`op dro_output (q : ro_query) : ro_output distr =`	Total mathematical function or predicate drooutput in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:59	lemma	`lemma matrix_query_distribution_lossless x :`	Probability bound $\Pr[\mathsf{matrixquerydistributionlossless}] \le B$ or loss inequality controlling a bad event in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:67	lemma	`lemma ro_output_distribution_lossless q :`	Probability bound $\Pr[\mathsf{rooutputdistributionlossless}] \le B$ or loss inequality controlling a bad event in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:81	type	`type in_t <- ro_query,`	Carrier set int used in the concrete lazy random-oracle state and oracle semantics; EasyCrypt equality is the mathematical equality on this set.
HAETAE_ROM.ec:82	type	`type out_t <- ro_output,`	Carrier set outt used in the concrete lazy random-oracle state and oracle semantics; EasyCrypt equality is the mathematical equality on this set.
HAETAE_ROM.ec:83	op	`op dout <- dro_output.`	Total mathematical function or predicate dout in the concrete lazy random-oracle state and oracle semantics.
HAETAE_ROM.ec:85	module	`module type Oracle = {`	EasyCrypt program module for the concrete lazy random-oracle state and oracle semantics; its procedures denote probabilistic state transformers.
HAETAE_ROM.ec:89	module	`module type POracle = {`	EasyCrypt program module for the concrete lazy random-oracle state and oracle semantics; its procedures denote probabilistic state transformers.
HAETAE_ROM.ec:93	op	`op ro_message_hash (y : ro_output) : crh = ro_output_to_crh y.`	Oracle-related function romessagehash used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM.ec:94	op	`op ro_challenge_hash (y : ro_output) : challenge = ro_output_to_challenge y.`	Probability law or sampler rochallengehash; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:95	op	`op ro_matrix (y : ro_output) : matrix = ro_output_to_matrix y.`	Deterministic algebraic map or predicate romatrix over HAETAE module/ring objects.
HAETAE_ROM.ec:96	op	`op ro_signing_coins (y : ro_output) : random_coins =`	Probability law or sampler rosigningcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM.ec:99	lemma	`lemma sampler_expand_query_dro_output_ro_signing_coins`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:18	type	`type programming_site = ro_query * ro_output.`	Carrier set programmingsite used in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; EasyCrypt equality is the mathematical equality on this set.
HAETAE_ROM_Programming.ec:20	op	`op programming_site_query (site : programming_site) : ro_query = site.‘1.`	Oracle-related function programmingsitequery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM_Programming.ec:21	op	`op programming_site_output (site : programming_site) : ro_output = site.‘2.`	Total mathematical function or predicate programmingsiteoutput in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:23	op	`op programming_site_of_transcript`	Total mathematical function or predicate programmingsiteoftranscript in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:27	op	`op signature_programming_site_of_transcript`	Total mathematical function or predicate signatureprogrammingsiteoftranscript in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:31	op	`op transcript_signature_challenge_output (tr : transcript) : ro_output =`	Probability law or sampler transcriptsignaturechallengeoutput; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM_Programming.ec:34	op	`op actual_signature_programming_site`	Total mathematical function or predicate actualsignatureprogrammingsite in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:39	op	`op honest_signing_programming_site`	Total mathematical function or predicate honestsigningprogrammingsite in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:45	op	`op honest_signing_programming_site_query`	Oracle-related function honestsigningprogrammingsitequery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM_Programming.ec:51	op	`op programmed_site_query_min_entropy_bound_for`	Numerical bound or budget parameter programmedsitequeryminentropyboundfor used to upper-bound a probability loss in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:60	op	`op programmed_site_query_min_entropy_bound`	Numerical bound or budget parameter programmedsitequeryminentropybound used to upper-bound a probability loss in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:65	op	`op programming_site_matches_transcript`	Total mathematical function or predicate programmingsitematchestranscript in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:70	op	`op reprogramming_failure`	Total mathematical function or predicate reprogrammingfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:74	op	`op challenge_entropy_support_ok (md : mode) (tr : transcript) : bool =`	Probability law or sampler challengeentropysupportok; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_ROM_Programming.ec:77	op	`op min_entropy_failure (md : mode) (tr : transcript) : bool =`	Total mathematical function or predicate minentropyfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:80	op	`op transcript_log_min_entropy_clear (md : mode) (trs : transcript list) : bool =`	Total mathematical function or predicate transcriptlogminentropyclear in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:83	op	`op signing_record_log_min_entropy_clear`	Total mathematical function or predicate signingrecordlogminentropyclear in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:87	op	`op fs_with_aborts_failure`	Total mathematical function or predicate fswithabortsfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:91	op	`op transcript_signature_programming_site_clear`	Total mathematical function or predicate transcriptsignatureprogrammingsiteclear in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:95	op	`op transcript_log_signature_programming_sites_clear`	Total mathematical function or predicate transcriptlogsignatureprogrammingsitesclear in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:99	op	`op fs_with_aborts_bound_obligation : bool =`	Numerical bound or budget parameter fswithabortsboundobligation used to upper-bound a probability loss in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:103	type	`type counted_rom_event =`	Carrier set countedromevent used in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; EasyCrypt equality is the mathematical equality on this set.
HAETAE_ROM_Programming.ec:109	op	`op counted_event_is_hash_query (ev : counted_rom_event) : bool =`	Oracle-related function countedeventishashquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM_Programming.ec:115	op	`op counted_event_is_signature_site (ev : counted_rom_event) : bool =`	Total mathematical function or predicate countedeventissignaturesite in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:121	op	`op counted_event_is_programmed_site (ev : counted_rom_event) : bool =`	Total mathematical function or predicate countedeventisprogrammedsite in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:127	op	`op counted_event_is_min_entropy_check (ev : counted_rom_event) : bool =`	Total mathematical function or predicate countedeventisminentropycheck in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:133	op	`op ro_query_is_challenge (q : ro_query) : bool =`	Probability law or sampler roqueryischallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM_Programming.ec:139	op	`op challenge_query_log_count (qs : ro_query list) : int =`	Probability law or sampler challengequerylogcount; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_ROM_Programming.ec:142	lemma	`lemma challenge_query_log_count_cons_le q qs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:156	lemma	`lemma challenge_query_log_count_nonchallenge_cons q qs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:169	lemma	`lemma challenge_query_log_count_message_cons pk ctx m qs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:177	lemma	`lemma challenge_query_log_count_matrix_cons md sd qs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:185	lemma	`lemma challenge_query_log_count_sampler_cons coins qs :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:193	op	`op counted_trace_hash_queries (evs : counted_rom_event list) : int =`	Numerical bound or budget parameter countedtracehashqueries used to upper-bound a probability loss in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:196	op	`op counted_trace_signature_sites (evs : counted_rom_event list) : int =`	Total mathematical function or predicate countedtracesignaturesites in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:199	op	`op counted_trace_programmed_sites (evs : counted_rom_event list) : int =`	Total mathematical function or predicate countedtraceprogrammedsites in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:202	op	`op counted_trace_min_entropy_checks (evs : counted_rom_event list) : int =`	Total mathematical function or predicate countedtraceminentropychecks in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:205	op	`op programming_sites_same_query`	Oracle-related function programmingsitessamequery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM_Programming.ec:209	op	`op programming_sites_conflict`	Total mathematical function or predicate programmingsitesconflict in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:214	op	`op programming_site_prequeried`	Total mathematical function or predicate programmingsiteprequeried in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:218	op	`op programming_site_reprograms`	Total mathematical function or predicate programmingsitereprograms in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:222	op	`op programming_site_fresh`	Predicate/event programmingsitefresh on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{programmingsitefresh}}$. *Context:* lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:228	op	`op programming_transcript_fresh`	Predicate/event programmingtranscriptfresh on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{programmingtranscriptfresh}}$. *Context:* lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:235	op	`op rom_counted_programming_bad`	Predicate/event romcountedprogrammingbad on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{romcountedprogrammingbad}}$. *Context:* lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:242	op	`op rom_counted_state_bad`	Predicate/event romcountedstatebad on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{romcountedstatebad}}$. *Context:* lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:247	op	`op counted_rom_programming_bound_obligation : bool =`	Numerical bound or budget parameter countedromprogrammingboundobligation used to upper-bound a probability loss in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:250	module	`module CountedLazyROM(O : Oracle) = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_ROM_Programming.ec:302	module	`module type SigningCoinSampler = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_ROM_Programming.ec:306	module	`module DirectSigningCoinSampler : SigningCoinSampler = {`	Signing/sampling oracle module; mathematically it realizes the distributional kernel used in the simulated signing experiment.
HAETAE_ROM_Programming.ec:319	lemma	`lemma counted_lazy_rom_init_clears :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:320	hoare/equiv	`hoare[CountedLazyROM(O).init :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:333	lemma	`lemma counted_lazy_rom_init_clears_hash_queries :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:334	hoare/equiv	`hoare[CountedLazyROM(O).init :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:343	lemma	`lemma counted_lazy_rom_get_preserves_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:344	hoare/equiv	`hoare[CountedLazyROM(O).get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_ROM_Programming.ec:360	lemma	`lemma counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count n :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:361	hoare/equiv	`hoare[CountedLazyROM(O).get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:372	lemma	`lemma counted_lazy_rom_get_preserves_bad_clear :`	Probability bound $\Pr[\mathsf{countedlazyromgetpreservesbadclear}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:373	hoare/equiv	`hoare[CountedLazyROM(O).get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:387	lemma	`lemma counted_lazy_rom_get_preserves_min_entropy_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:388	hoare/equiv	`hoare[CountedLazyROM(O).get :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:398	lemma	`lemma counted_lazy_rom_observe_transcript_preserves_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:399	hoare/equiv	`hoare[CountedLazyROM(O).observe_transcript :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:417	lemma	`lemma counted_lazy_rom_observe_transcript_preserves_min_entropy_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:418	hoare/equiv	`hoare[CountedLazyROM(O).observe_transcript :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:430	lemma	`lemma counted_lazy_rom_observe_transcript_preserves_bad_clear :`	Probability bound $\Pr[\mathsf{countedlazyromobservetranscriptpreservesbadclear}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:431	hoare/equiv	`hoare[CountedLazyROM(O).observe_transcript :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:447	lemma	`lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh :`	Probability bound $\Pr[\mathsf{countedlazyromprogrampreservesbadclearifffresh}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:448	hoare/equiv	`hoare[CountedLazyROM(O).program :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:465	lemma	`lemma counted_lazy_rom_program_preserves_min_entropy_clear :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:466	hoare/equiv	`hoare[CountedLazyROM(O).program :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:476	lemma	`lemma counted_lazy_rom_bad_preserves_min_entropy_clear :`	Probability bound $\Pr[\mathsf{countedlazyrombadpreservesminentropyclear}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:477	hoare/equiv	`hoare[CountedLazyROM(O).bad :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:486	lemma	`lemma counted_lazy_rom_bad_false_when_clear :`	Probability bound $\Pr[\mathsf{countedlazyrombadfalsewhenclear}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:487	hoare/equiv	`hoare[CountedLazyROM(O).bad :`	Hoare judgment $\{P\}\, c\, \{Q\}$ for a deterministic/probabilistic EasyCrypt procedure used in the proof.
HAETAE_ROM_Programming.ec:501	module	`module type CountedROMInterface = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_ROM_Programming.ec:508	module	`module type CountedROMClient(C : CountedROMInterface) = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_ROM_Programming.ec:517	module	`module CountedLazyROMBadGame(`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_ROM_Programming.ec:521	module	`module C = CountedLazyROM(O)`	EasyCrypt program module for the lazy-ROM programming, freshness, and fundamental-lemma reasoning; its procedures denote probabilistic state transformers.
HAETAE_ROM_Programming.ec:522	module	`module A = A(C)`	EasyCrypt program module for the lazy-ROM programming, freshness, and fundamental-lemma reasoning; its procedures denote probabilistic state transformers.
HAETAE_ROM_Programming.ec:534	module	`module ProgramChallenge(O : Oracle) = {`	EasyCrypt program module for the lazy-ROM programming, freshness, and fundamental-lemma reasoning; its procedures denote probabilistic state transformers.
HAETAE_ROM_Programming.ec:540	module	`module type TranscriptProgrammer(O : Oracle) = {`	EasyCrypt program module for the lazy-ROM programming, freshness, and fundamental-lemma reasoning; its procedures denote probabilistic state transformers.
HAETAE_ROM_Programming.ec:544	module	`module TranscriptProgrammerFromSite(O : Oracle) = {`	Program module implementing a lazy random oracle $\mathcal{H}$; module state is the finite map/log used in ROM games.
HAETAE_ROM_Programming.ec:550	lemma	`lemma programming_site_self_matches md tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:560	lemma	`lemma programming_site_self_no_reprogramming_failure md tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:569	lemma	`lemma fs_with_aborts_failureE md tr site :`	Checked theorem fswithabortsfailureE in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:574	lemma	`lemma programming_site_prequeried_cons qs site :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:578	lemma	`lemma programming_site_prequeried_cons_preserve q qs site :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:583	lemma	`lemma programming_site_prequeried_output_irrelevant qs q y1 y2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:588	lemma	`lemma programming_site_prequeried_witness qs q y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:593	lemma	`lemma honest_signing_programming_site_queryE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:611	op	`op programmed_site_entropy_token_from_query (q :  ro_query) :  int =`	Oracle-related function programmedsiteentropytokenfromquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_ROM_Programming.ec:617	lemma	`lemma honest_signing_programming_site_query_entropy_token`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:629	lemma	`lemma honest_signing_programming_site_query_token_injective`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:644	lemma	`lemma signing_distribution_programming_site_query_spread`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:673	lemma	`lemma honest_signing_programming_site_query_spread`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:684	lemma	`lemma honest_signing_programming_site_query_spread_bound`	Probability bound $\Pr[\mathsf{honestsigningprogrammingsitequeryspreadbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:694	lemma	`lemma honest_signing_programming_site_query_spread_bound_for`	Probability bound $\Pr[\mathsf{honestsigningprogrammingsitequeryspreadboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_ROM_Programming.ec:704	lemma	`lemma honest_signing_programming_site_query_is_challenge`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:713	lemma	`lemma programming_site_prequeried_honest_challenge_filter`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:726	lemma	`lemma programming_site_prequeried_honest_message_consE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:745	lemma	`lemma honest_signing_programming_site_outputE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:764	lemma	`lemma honest_signing_programming_site_functional_output`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:778	lemma	`lemma honest_signing_programming_sites_no_conflict`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:795	lemma	`lemma actual_signature_programming_site_sign_internalE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:808	lemma	`lemma actual_signature_programming_site_sign_internal_any_pkE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:823	lemma	`lemma actual_signature_programming_site_prequeried_after_cons`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:839	lemma	`lemma actual_signature_programming_site_prequeried_after_message_hash`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:854	lemma	`lemma actual_signature_programming_site_prequeried_message_hashE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:870	lemma	`lemma actual_signature_programming_site_prequeried_message_hash_any_pkE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:884	op	`op programming_site_conflict_free`	Total mathematical function or predicate programmingsiteconflictfree in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:888	op	`op programming_site_log_conflict_free_for_honest`	Total mathematical function or predicate programmingsitelogconflictfreeforhonest in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:894	lemma	`lemma programming_site_log_conflict_free_for_honest_nil md sk :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:900	lemma	`lemma programming_site_log_conflict_free_for_honest_cons`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:914	lemma	`lemma programming_site_conflict_free_no_reprograms site sites :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:924	lemma	`lemma actual_signature_programming_site_sign_internal_conflict_step`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:949	lemma	`lemma programmed_site_reprogram_one_step_conflict_free_zero`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:969	lemma	`lemma programmed_site_prequery_one_step_fset_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepfsetboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:999	lemma	`lemma programmed_site_prequery_one_step_fset_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepfsetbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1012	lemma	`lemma programmed_site_prequery_one_step_budget_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepbudgetboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1044	lemma	`lemma programmed_site_prequery_one_step_budget_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepbudgetbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1061	lemma	`lemma programmed_site_prequery_one_step_challenge_fset_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengefsetboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1081	lemma	`lemma programmed_site_prequery_one_step_challenge_fset_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengefsetbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1095	lemma	`lemma programmed_site_prequery_one_step_challenge_budget_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengebudgetboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1129	lemma	`lemma programmed_site_prequery_one_step_challenge_budget_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengebudgetbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1148	lemma	`lemma programmed_site_prequery_one_step_challenge_concrete_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengeconcretebound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1165	lemma	`lemma programmed_site_prequery_one_step_challenge_concrete_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengeconcreteboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1183	lemma	`lemma challenge_query_log_count_budget_card_bound hc qs :`	Probability bound $\Pr[\mathsf{challengequerylogcountbudgetcardbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1195	lemma	`lemma programmed_site_prequery_one_step_challenge_counter_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengecounterbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1212	lemma	`lemma programmed_site_prequery_one_step_challenge_counter_bound_for`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepchallengecounterboundfor}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1231	lemma	`lemma programmed_site_prequery_one_step_message_hash_counter_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepmessagehashcounterbound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1257	lemma	`lemma direct_signing_coin_sampler_prequery_bound`	Probability bound $\Pr[\mathsf{directsigningcoinsamplerprequerybound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1262	hoare/equiv	`phoare[DirectSigningCoinSampler.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_ROM_Programming.ec:1283	lemma	`lemma direct_signing_coin_sampler_budgeted_prequery_bound`	Probability bound $\Pr[\mathsf{directsigningcoinsamplerbudgetedprequerybound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1286	hoare/equiv	`phoare[DirectSigningCoinSampler.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_ROM_Programming.ec:1301	lemma	`lemma direct_signing_coin_sampler_message_hash_prequery_bound`	Probability bound $\Pr[\mathsf{directsigningcoinsamplermessagehashprequerybound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1306	hoare/equiv	`phoare[DirectSigningCoinSampler.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_ROM_Programming.ec:1333	lemma	`lemma direct_signing_coin_sampler_message_hash_budgeted_prequery_bound`	Probability bound $\Pr[\mathsf{directsigningcoinsamplermessagehashbudgetedprequerybound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1336	hoare/equiv	`phoare[DirectSigningCoinSampler.sample :`	Probabilistic Hoare judgment $\mathbf{phoare}[c : P \Rightarrow Q] = p$ for the procedure-level event or postcondition.
HAETAE_ROM_Programming.ec:1353	lemma	`lemma programmed_site_prequery_one_step_concrete_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryonestepconcretebound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1369	lemma	`lemma programmed_site_prequery_reprogram_one_step_concrete_bound`	Probability bound $\Pr[\mathsf{programmedsitepreQueryreprogramonestepconcretebound}] \le B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_ROM_Programming.ec:1404	lemma	`lemma programmed_site_query_min_entropy_bound_constant_impossible`	Probability bound Pr[programmedsitequeryminentropyboundconstantimpossible] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1426	lemma	`lemma programming_site_reprograms_conflict_cons site old programmed :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1434	lemma	`lemma rom_counted_programming_bad_min_entropy queries programmed :`	Probability bound Pr[romcountedprogrammingbadminentropy] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1438	lemma	`lemma rom_counted_programming_bad_prequery_cons queries programmed site`	Probability bound Pr[romcountedprogrammingbadprequerycons] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1448	lemma	`lemma counted_rom_programming_bound_obligation_holds :`	Probability bound Pr[countedromprogrammingboundobligationholds] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1460	op	`op counted_lazy_rom_bad_flags_budget_sound (p : real) : bool =`	Predicate/event countedlazyrombadflagsbudgetsound on the game state; in probability expressions it denotes the indicator event $1_{\text{countedlazyrombadflagsbudgetsound}}$. *Context:* lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1463	lemma	`lemma counted_lazy_rom_bad_flags_universal_bound &m :`	Probability bound Pr[countedlazyrombadflagsuniversalbound] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1469	lemma	`lemma counted_lazy_rom_bad_flags_query_budget_bound &m :`	Probability bound Pr[countedlazyrombadflagsquerybudgetbound] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1478	lemma	`lemma counted_lazy_rom_bad_flags_query_budget_bound_subunit &m :`	Probability bound Pr[countedlazyrombadflagsquerybudgetboundsubunit] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1491	lemma	`lemma fs_with_aborts_no_programming_failure md tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1501	lemma	`lemma transcript_signature_challenge_output_matches tr :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_ROM_Programming.ec:1511	lemma	`lemma honest_transcript_challenge_entropy_support md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_ROM_Programming.ec:1521	lemma	`lemma honest_transcript_no_min_entropy_failure md sk m ctx coins :`	Checked theorem honesttranscriptnominentropyfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1529	lemma	`lemma signing_transcript_relation_challenge_entropy_support`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_ROM_Programming.ec:1549	lemma	`lemma signing_transcript_relation_no_min_entropy_failure`	Checked theorem signingtranscriptrelationnominentropyfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1560	lemma	`lemma signing_record_relation_no_min_entropy_failure md pk r :`	Checked theorem signingrecordrelationnominentropyfailure in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1572	lemma	`lemma signing_record_log_sound_no_min_entropy md pk rs :`	Checked theorem signingrecordlogsoundnominentropy in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1584	lemma	`lemma signing_record_log_sound_transcripts_no_min_entropy md pk rs :`	Checked theorem signingrecordlogsoundtranscriptsnominentropy in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1597	lemma	`lemma fs_with_aborts_no_failure_for_signing_relation`	Checked theorem fswithabortsnofailureforsigningrelation in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1609	lemma	`lemma fs_with_aborts_no_failure_for_signing_record md pk r site :`	Checked theorem fswithabortsnofailureforsigningrecord in the lazy-ROM programming, freshness, and fundamental-lemma reasoning; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_ROM_Programming.ec:1624	lemma	`lemma fs_with_aborts_no_failure_for_honest_programming md sk m ctx coins y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1639	lemma	`lemma transcript_valid_programming_site_signature_query md tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1650	lemma	`lemma transcript_valid_signature_programming_site_matches md tr y :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1666	lemma	`lemma transcript_valid_signature_programming_site_no_reprogramming`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1679	lemma	`lemma transcript_valid_actual_signature_programming_site_matches md tr :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1691	lemma	`lemma transcript_valid_actual_signature_programming_site_no_reprogramming`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1702	lemma	`lemma valid_forking_pair_signature_programming_site_query md tr1 tr2 y1 y2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1713	lemma	`lemma valid_forking_pair_programming_site_query md tr1 tr2 y1 y2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1723	lemma	`lemma valid_forking_pair_self_programming_sites_no_reprogramming`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_ROM_Programming.ec:1739	lemma	`lemma fs_bound_parts_nonnegative :`	Probability bound Pr[fsboundpartsnonnegative] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_ROM_Programming.ec:1745	lemma	`lemma fs_with_aborts_bound_obligation_holds :`	Probability bound Pr[fswithabortsboundobligationholds] $\leq B$ or loss inequality controlling a bad event in the lazy-ROM programming, freshness, and fundamental-lemma reasoning.
HAETAE_Scheme.ec:13	type	`type pkey <- pkey,`	Public-key space $\mathcal{PK}$; values of pkey denote HAETAE verification keys.
HAETAE_Scheme.ec:14	type	`type skey <- skey,`	Secret-key space $\mathcal{SK}$; values of skey denote HAETAE signing keys.
HAETAE_Scheme.ec:15	type	`type message <- message,`	Carrier set $\mathcal{M}$ of HAETAE messages; the EasyCrypt type message is read as a mathematical message object.
HAETAE_Scheme.ec:16	type	`type context <- context,`	Carrier set context used in the HAETAE key generation, signing, verification, and sampling algorithms; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Scheme.ec:17	type	`type signature <- signature,`	Signature space Σ; values of signature denote candidate HAETAE signatures.
HAETAE_Scheme.ec:18	type	`type ro_query <- ro_query,`	Transcript/query carrier used to define sets Q_H of random-oracle queries and logged signing transcripts.
HAETAE_Scheme.ec:19	type	`type ro_output <- ro_output,`	Carrier set roоutput used in the HAETAE key generation, signing, verification, and sampling algorithms; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Scheme.ec:20	op	`op dro_output <- dro_output.`	Total mathematical function or predicate droоutput in the HAETAE key generation, signing, verification, and sampling algorithms.
HAETAE_Scheme.ec:22	op	`op haetae_mode : mode.`	Total mathematical function or predicate haetaemode in the HAETAE key generation, signing, verification, and sampling algorithms.
HAETAE_Scheme.ec:24	module	`module HAETAE(H : SIG.POracle) = {`	EasyCrypt program module for the HAETAE key generation, signing, verification, and sampling algorithms; its procedures denote probabilistic state transformers.
HAETAE_Transcript.ec:11	type	`type transcript =`	Transcript/query carrier used to define sets Q_H of random-oracle queries and logged signing transcripts.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Transcript.ec: 14	op	`op signature_commitment_highbits :`	Total mathematical function or predicate signaturecommitmenthighbits in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 18	op	`op signature_commitment_lowbits :`	Total mathematical function or predicate signaturecommitmentlowbits in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 22	op	`op signature_message_hash :`	Oracle-related function signaturemessagehash used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Transcript.ec: 26	op	`op signature_challenge :`	Probability law or sampler signaturechallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 31	op	`op transcript_pk (tr : transcript) : pkey = tr.‘1.`	Total mathematical function or predicate transcriptpk in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 32	op	`op transcript_message (tr : transcript) : message = tr.‘2.`	Total mathematical function or predicate transcriptmessage in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 33	op	`op transcript_context (tr : transcript) : context = tr.‘3.`	Total mathematical function or predicate transcriptcontext in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 34	op	`op transcript_commitment_highbits (tr : transcript) : polyveck = tr.‘4.`	Total mathematical function or predicate transcriptcommitmenthighbits in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 35	op	`op transcript_commitment_lowbits (tr : transcript) : poly = tr.‘5.`	Total mathematical function or predicate transcriptcommitmentlowbits in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 36	op	`op transcript_message_hash (tr : transcript) : crh = tr.‘6.`	Oracle-related function transcriptmessagehash used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Transcript.ec: 37	op	`op transcript_challenge (tr : transcript) : challenge = tr.‘7.`	Probability law or sampler transcriptchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 38	op	`op transcript_signature (tr : transcript) : signature = tr.‘8.`	Total mathematical function or predicate transcriptsignature in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 40	op	`op transcript_of_signature :`	Total mathematical function or predicate transcriptofsignature in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 50	op	`op transcript_challenge_query (md : mode) (tr : transcript) : ro_query =`	Probability law or sampler transcriptchallengequery; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 57	op	`op transcript_signature_challenge_query (md : mode) (tr : transcript) :`	Probability law or sampler transcriptsignaturechallengequery; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 65	op	`op transcript_verifies (md : mode) (tr : transcript) : bool =`	Total mathematical function or predicate transcriptverifies in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 72	op	`op transcript_challenge_matches (tr : transcript) (y : ro_output) : bool =`	Probability law or sampler transcriptchallengematches; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 75	op	`op transcript_signature_challenge_matches`	Probability law or sampler transcriptsignaturechallengematches; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 79	op	`op transcript_fields_match_signature (tr : transcript) : bool =`	Total mathematical function or predicate transcriptfieldsmatchsignature in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 89	op	`op transcript_valid (md : mode) (tr : transcript) : bool =`	Predicate/event transcriptvalid on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{transcriptvalid}}$. *Context:* observable transcripts and transcript projections.
HAETAE_Transcript.ec: 94	op	`op transcript_public_equation (md : mode) (tr : transcript) : bool =`	Total mathematical function or predicate transcriptpublicequation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 97	op	`op same_fork_target (tr1 tr2 : transcript) : bool =`	Total mathematical function or predicate sameforktarget in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 105	op	`op distinct_fork_challenges (tr1 tr2 : transcript) : bool =`	Probability law or sampler distinctforkchallenges; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 108	op	`op valid_forking_pair (md : mode) (tr1 tr2 : transcript) : bool =`	Predicate/event validforkingpair on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{validforkingpair}}$. *Context:* observable transcripts and transcript projections.
HAETAE_Transcript.ec: 114	op	`op transcript_response (tr : transcript) : polyvecl =`	Total mathematical function or predicate transcriptresponse in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 116	op	`op transcript_response_aux (tr : transcript) : polyveck =`	Total mathematical function or predicate transcriptresponseaux in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 119	op	`op transcript_public_key_challenge_term (md : mode) (tr : transcript) :`	Probability law or sampler transcriptpublickeychallengeterm; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 125	op	`op active_public_reconstruction (md : mode) (tr : transcript) : bool =`	Total mathematical function or predicate activepublicreconstruction in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 128	op	`op inactive_public_reconstruction (md : mode) (tr : transcript) : bool =`	Total mathematical function or predicate inactivepublicreconstruction in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 131	op	`op forking_difference_solution`	Total mathematical function or predicate forkingdifferencesolution in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 135	op	`op fork_response_aux_difference (md : mode) (tr1 tr2 : transcript) :`	Total mathematical function or predicate forkresponseauxdifference in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 139	op	`op fork_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool =`	Probability law or sampler forkpublicchallengerelation; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 147	op	`op fork_response_aux_relation (tr1 tr2 : transcript) : bool =`	Total mathematical function or predicate forkresponseauxrelation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 150	op	`op fork_left_active_public_challenge_relation`	Probability law or sampler forkleftactivepublicchallengerelation; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 157	op	`op fork_right_active_public_challenge_relation`	Probability law or sampler forkrightactivepublicchallengerelation; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Transcript.ec: 164	op	`op fork_public_reconstruction_cases`	Total mathematical function or predicate forkpublicreconstructioncases in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 168	op	`op fork_matrix_source (tr1 tr2 : transcript) : byte list =`	Deterministic algebraic map or predicate forkmatrixsource over HAETAE module/ring objects.
HAETAE_Transcript.ec: 178	op	`op fork_module_matrix (md : mode) (tr1 tr2 : transcript) : matrix =`	Deterministic algebraic map or predicate forkmodulematrix over HAETAE module/ring objects.
HAETAE_Transcript.ec: 184	op	`op fork_module_target (md : mode) (tr1 tr2 : transcript) : polyveck =`	Total mathematical function or predicate forkmoduletarget in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 189	op	`op bimodal_selftarget_msis_instance_of_fork :`	Total mathematical function or predicate bimodalselftargetmsisinstanceoffork in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 194	op	`op extract_bimodal_selftarget_msis_solution :`	Total mathematical function or predicate extractbimodalselftargetmsissolution in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 202	op	`op module_sis_instance_of_fork :`	Total mathematical function or predicate modulesisinstanceoffork in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 208	op	`op extract_module_sis_solution :`	Total mathematical function or predicate extractmodulesissolution in the observable transcripts and transcript projections.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Transcript.ec: 216	op	`op bimodal_extraction_success (md : mode) (tr1 tr2 : transcript) : bool =`	Total mathematical function or predicate bimodalextractionsuccess in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 222	lemma	`lemma extract_bimodal_forking_some md tr1 tr2 :`	Checked theorem extractbimodalforkingsome in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 231	lemma	`lemma extract_bimodal_forking_nonempty md tr1 tr2 :`	Checked theorem extractbimodalforkingnonempty in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 240	lemma	`lemma transcript_fields_match_signature_challenge_query md tr :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 254	lemma	`lemma transcript_fields_match_signature_challenge_matches tr y :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Transcript.ec: 268	lemma	`lemma transcript_valid_signature_challenge_query md tr :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 278	lemma	`lemma transcript_valid_signature_challenge_matches md tr y :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Transcript.ec: 288	lemma	`lemma transcript_valid_signature_challengeE md tr :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Transcript.ec: 299	lemma	`lemma transcript_valid_public_equation md tr :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 313	lemma	`lemma transcript_valid_commitment_reconstruction md tr :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 333	lemma	`lemma valid_forking_pair_public_equations md tr1 tr2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 345	lemma	`lemma same_fork_target_challenge_query md tr1 tr2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 354	lemma	`lemma valid_forking_pair_challenge_query md tr1 tr2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 363	lemma	`lemma valid_forking_pair_signature_challenge_query md tr1 tr2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 378	lemma	`lemma valid_forking_pair_distinct_signature_challenges md tr1 tr2 :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Transcript.ec: 392	lemma	`lemma same_fork_target_commitment_highbits tr1 tr2 :`	Checked theorem sameforktargetcommitmenthighbits in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 397	lemma	`lemma valid_forking_pair_commitment_highbits md tr1 tr2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 406	lemma	`lemma valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 423	lemma	`lemma transcript_reconstructionE md tr :`	Checked theorem transcriptreconstructionE in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 436	lemma	`lemma transcript_active_reconstructionE md tr :`	Checked theorem transcriptactivereconstructionE in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 448	lemma	`lemma valid_forking_pair_public_challenge_relation md tr1 tr2 :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Transcript.ec: 461	lemma	`lemma valid_forking_pair_public_reconstruction_cases md tr1 tr2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 470	op	`op extraction_success (md : mode) (tr1 tr2 : transcript) : bool =`	Total mathematical function or predicate extractionsuccess in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 476	op	`op forking_extraction_obligation (md : mode) : bool =`	Total mathematical function or predicate forkingextractionobligation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 481	op	`op fork_public_reconstruction_obligation (md : mode) : bool =`	Total mathematical function or predicate forkpublicreconstructionobligation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 486	lemma	`lemma forking_difference_solution_wf md tr1 tr2 :`	Checked theorem forkingdifferencesolutionwf in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 493	lemma	`lemma fork_module_matrix_rows_wf md tr1 tr2 :`	Checked theorem forkmodulematrixrowswf in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 501	lemma	`lemma fork_module_matrix_size md tr1 tr2 :`	Checked theorem forkmodulematrixsize in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 505	lemma	`lemma bimodal_forking_solution_valid md tr1 tr2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Transcript.ec: 516	lemma	`lemma forking_extraction_obligation_holds md :`	Checked theorem forkingextractionobligationholds in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 527	lemma	`lemma fork_public_reconstruction_obligation_holds md :`	Checked theorem forkpublicreconstructionobligationholds in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 535	op	`op bimodal_to_module_sis_lift_obligation (md : mode) : bool =`	Total mathematical function or predicate bimodaltomodulesisliftobligation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 543	lemma	`lemma bimodal_to_module_sis_lift_obligation_holds md :`	Checked theorem bimodaltomodulesisliftobligationholds in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec: 551	op	`op special_soundness_obligation (md : mode) : bool =`	Total mathematical function or predicate specialsoundnessobligation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 556	op	`op special_soundness_with_public_reconstruction_obligation (md : mode) : bool =`	Total mathematical function or predicate specialsoundnesswithpublicreconstructionobligation in the observable transcripts and transcript projections.
HAETAE_Transcript.ec: 562	lemma	`lemma module_sis_extraction_from_bimodal md tr1 tr2 :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 582	lemma	`lemma special_soundness_from_bimodal_lift md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 592	lemma	`lemma special_soundness_with_public_reconstruction_from_bimodal_lift md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Transcript.ec: 607	op	`op transcript_from_honest_signing`	Oracle-related function transcriptfromhonestsigning used to form or interpret a random-oracle query in the lazy-ROM proof.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Transcript.ec:615	lemma	`lemma honest_transcript_verifies md sk m ctx coins :`	Checked theorem honesttranscriptverifies in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec:625	lemma	`lemma transcript_of_signature_fields md pk m ctx sig :`	Checked theorem transcriptofsignaturefields in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec:640	lemma	`lemma transcript_of_signature_matches_signature md pk m ctx sig :`	Checked theorem transcriptofsignaturematchessignature in the observable transcripts and transcript projections; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Transcript.ec:647	lemma	`lemma honest_transcript_challenge md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Events.ec:9	op	`op keygen_rejection_failure (md : mode) (pk : pkey) (sk : skey) : bool =`	Total mathematical function or predicate keygenrejectionfailure in the security events, bad events, and predicates over executions.
HAETAE_Events.ec:12	op	`op signing_rejection_failure (md : mode) (sig : signature) : bool =`	Total mathematical function or predicate signingrejectionfailure in the security events, bad events, and predicates over executions.
HAETAE_Events.ec:15	op	`op verification_rejection_failure (md : mode) (sig : signature) : bool =`	Total mathematical function or predicate verificationrejectionfailure in the security events, bad events, and predicates over executions.
HAETAE_Events.ec:18	op	`op any_rejection_failure (md : mode) (pk : pkey) (sk : skey)`	Total mathematical function or predicate anyrejectionfailure in the security events, bad events, and predicates over executions.
HAETAE_Events.ec:24	lemma	`lemma keygen_rejection_free_current md sd :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Events.ec:32	lemma	`lemma signing_rejection_free_current md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Events.ec:40	lemma	`lemma verification_rejection_free_current md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Events.ec:48	lemma	`lemma accepted_transcript_rejection_free_current md sd m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Assumptions.ec:9	type	`type mlwe_instance.`	Carrier set mlweinstance used in the cryptographic assumptions used by the reduction theorem; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Assumptions.ec:10	type	`type module_sis_instance = matrix * polyveck.`	Carrier set modulesisinstance used in the cryptographic assumptions used by the reduction theorem; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Assumptions.ec:11	type	`type module_sis_solution = polyvecl.`	Carrier set modulesissolution used in the cryptographic assumptions used by the reduction theorem; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Assumptions.ec:12	type	`type bimodal_selftarget_msis_instance = module_sis_instance.`	Carrier set bimodalselftargetmsisinstance used in the cryptographic assumptions used by the reduction theorem; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Assumptions.ec:13	type	`type bimodal_selftarget_msis_solution = module_sis_solution.`	Carrier set bimodalselftargetmsissolution used in the cryptographic assumptions used by the reduction theorem; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Assumptions.ec:15	op	`op [lossless] dmlwe_real : mode -> mlwe_instance distr.`	Numerical bound or budget parameter lossless used to upper-bound a probability loss in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:16	op	`op [lossless] dmlwe_random : mode -> mlwe_instance distr.`	Numerical bound or budget parameter lossless used to upper-bound a probability loss in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:17	op	`op [lossless] dmodule_sis_instance : mode -> module_sis_instance distr.`	Numerical bound or budget parameter lossless used to upper-bound a probability loss in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:18	op	`op [lossless] dbimodal_selftarget_msis_instance :`	Numerical bound or budget parameter lossless used to upper-bound a probability loss in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:21	op	`op module_sis_eval`	Total mathematical function or predicate modulesiseval in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:26	op	`op module_sis_zero_instance (md : mode) : module_sis_instance =`	Total mathematical function or predicate modulesiszeroinstance in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:29	op	`op module_sis_solution_wf (md : mode) (sol : module_sis_solution) : bool =`	Total mathematical function or predicate modulesissolutionwf in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:32	op	`op module_sis_valid`	Predicate/event modulesisvalid on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{modulesisvalid}}$. *Context:* cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:38	op	`op bimodal_selftarget_msis_valid`	Predicate/event bimodalselftargetmsisvalid on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{bimodalselftargetmsisvalid}}$. *Context:* cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:43	lemma	`lemma module_sis_eval_zero_instance md sol :`	Checked theorem modulesisevalzeroinstance in the cryptographic assumptions used by the reduction theorem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Assumptions.ec:50	lemma	`lemma module_sis_zero_valid md :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Assumptions.ec:59	lemma	`lemma module_sis_eval_wf md x sol :`	Checked theorem modulesisevalwf in the cryptographic assumptions used by the reduction theorem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Assumptions.ec:63	lemma	`lemma module_sis_valid_target_wf md x sol :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Assumptions.ec:72	op	`op bimodal_to_module_sis_instance :`	Total mathematical function or predicate bimodaltomodulesisinstance in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:75	op	`op bimodal_to_module_sis_solution :`	Total mathematical function or predicate bimodaltomodulesissolution in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:80	lemma	`lemma bimodal_to_module_sis_valid md x sol :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Assumptions.ec:91	module	`module type MLWE_Distinguisher = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:95	module	`module MLWE_Real_Game(B : MLWE_Distinguisher) = {`	Experiment module $\mathcal{G}_{\mathsf{MLWERealGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:106	module	`module MLWE_Random_Game(B : MLWE_Distinguisher) = {`	Experiment module $\mathcal{G}_{\mathsf{MLWERandomGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:117	module	`module type ModuleSIS_Solver = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:121	module	`module ModuleSIS_Relation_Game(B : ModuleSIS_Solver) = {`	Experiment module $\mathcal{G}_{\mathsf{ModuleSISRelationGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:132	module	`module type BimodalSelfTargetMSIS_Solver = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:137	module	`module BimodalSelfTargetMSIS_Relation_Game(`	Experiment module $\mathcal{G}_{\mathsf{BimodalSelfTargetMSISRelationGame}}$ whose returned Boolean event defines an advantage probability.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Assumptions.ec:150	op	`op lift_bimodal_solution_option`	Total mathematical function or predicate liftbimodalsolutionoption in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:157	op	`op dmodule_sis_from_bimodal (md : mode) : module_sis_instance distr =`	Oracle-related function dmodulesisfrombimodal used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Assumptions.ec:161	lemma	`lemma dmodule_sis_from_bimodalE md :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Assumptions.ec:167	lemma	`lemma dmodule_sis_from_bimodal_lossless md :`	Probability bound $\Pr[\mathsf{dmodulesisfrombimodallossless}] \le B$ or loss inequality controlling a bad event in the cryptographic assumptions used by the reduction theorem.
HAETAE_Assumptions.ec:174	module	`module BimodalToModuleSIS_Lift_Game(`	Experiment module $\mathcal{G}_{\mathsf{BimodalToModuleSISLiftGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:191	lemma	`lemma bimodal_lift_successE md x sol :`	Checked theorem bimodalliftsuccessE in the cryptographic assumptions used by the reduction theorem; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Assumptions.ec:209	lemma	`lemma bimodal_solver_lift_exact &m md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Assumptions.ec:227	module	`module type ModuleSIS_ModeSolver = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:232	module	`module type BimodalSelfTargetMSIS_ModeSolver = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:237	module	`module ModuleSIS_ModeRelation_Game(`	Experiment module $\mathcal{G}_{\mathsf{ModuleSISModeRelationGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:250	module	`module BimodalSelfTargetMSIS_ModeRelation_Game(`	Experiment module $\mathcal{G}_{\mathsf{BimodalSelfTargetMSISModeRelationGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:263	module	`module ModuleSIS_LiftedDistribution_Relation_Game(`	Experiment module $\mathcal{G}_{\mathsf{ModuleSISLiftedDistributionRelationGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:276	module	`module BimodalModeSolver_As_ModuleSIS(`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:292	lemma	`lemma bimodal_mode_solver_lifted_distribution_exact &m md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Assumptions.ec:310	module	`module type MLWE_Reduction = {`	Reduction module $\mathcal{R}^{\mathcal{A}}$ that converts a successful forgery/transcript event into an instance of the underlying hardness problem.
HAETAE_Assumptions.ec:314	module	`module type BimodalSelfTargetMSIS_Reduction = {`	Reduction module $\mathcal{R}^{\mathcal{A}}$ that converts a successful forgery/transcript event into an instance of the underlying hardness problem.
HAETAE_Assumptions.ec:318	module	`module type ModuleSIS_Reduction = {`	Reduction module $\mathcal{R}^{\mathcal{A}}$ that converts a successful forgery/transcript event into an instance of the underlying hardness problem.
HAETAE_Assumptions.ec:322	module	`module MLWE_Game(B : MLWE_Reduction) = {`	Experiment module $\mathcal{G}_{\mathsf{MLWEGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:331	module	`module BimodalSelfTargetMSIS_Game(B : BimodalSelfTargetMSIS_Reduction) = {`	Experiment module $\mathcal{G}_{\mathsf{BimodalSelfTargetMSISGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:340	module	`module ModuleSIS_Game(B : ModuleSIS_Reduction) = {`	Experiment module $\mathcal{G}_{\mathsf{ModuleSISGame}}$ whose returned Boolean event defines an advantage probability.
HAETAE_Assumptions.ec:349	module	`module BimodalMSIS_As_ModuleSIS(B : BimodalSelfTargetMSIS_Reduction) = {`	EasyCrypt program module for the cryptographic assumptions used by the reduction theorem; its procedures denote probabilistic state transformers.
HAETAE_Assumptions.ec:362	lemma	`lemma bimodal_msis_as_module_sis_exact &m :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:10	type	`type signing_randomness_source = int.`	Signature space Σ; values of signing_randomness_source denote candidate HAETAE signatures.
HAETAE_Distributions.ec:12	op	`op dbyte : byte distr = dinter 0 255.`	Total mathematical function or predicate dbyte in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:13	op	`op dseed : seed distr = dlist dbyte seedbytes.`	Probability law or sampler dseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:14	op	`op dcrh : crh distr = dlist dbyte crhbytes.`	Total mathematical function or predicate dcrh in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:15	op	`op signing_entropy_token_cardinality : int = 288230376151711744.`	Total mathematical function or predicate signingentropytokencardinality in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:16	op	`op signing_entropy_token_in_range (x : int) : bool =`	Total mathematical function or predicate signingentropytokeninrange in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:18	op	`op signing_randomness_byte_ok (b : byte) : bool = 0 <= b /\ b < 256.`	Probability law or sampler signingrandomnessbyteok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:19	op	`op signing_entropy_token_to_coins (x : int) : random_coins =`	Probability law or sampler signingentropytokentocoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:29	op	`op signing_randomness_domain (coins : random_coins) : bool =`	Probability law or sampler signingrandomnessdomain; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:33	op	`op dsigning_entropy_token : int distr =`	Total mathematical function or predicate dsigningentropytoken in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:35	op	`op dsigning_randomness_source : signing_randomness_source distr =`	Probability law or sampler dsigningrandomnesssource; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:37	op	`op signing_randomness_source_valid (src : signing_randomness_source) : bool =`	Predicate/event signingrandomnesssourcevalid on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{signingrandomnesssourcevalid}}$. *Context:* probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:39	op	`op signing_randomness_from_source (src : signing_randomness_source) :`	Probability law or sampler signingrandomnessfromsource; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:42	op	`op dsigning_randomness : random_coins distr =`	Probability law or sampler dsigningrandomness; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:44	op	`op drandom_coins : random_coins distr = dsigning_randomness.`	Probability law or sampler drandomcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:45	op	`op signing_randomness_point_bound : real =`	Numerical bound or budget parameter signingrandomnesspointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:47	op	`op signing_randomness_token_spread (d : random_coins distr) : bool =`	Probability law or sampler signingrandomnesstokenspread; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:51	op	`op signing_randomness_distribution_ok (d : random_coins distr) : bool =`	Probability law or sampler signingrandomnessdistributionok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:55	op	`op dcoeff : coeff distr = dinter 0 (q - 1).`	Deterministic algebraic map or predicate dcoeff over HAETAE module/ring objects.
HAETAE_Distributions.ec:56	op	`op dpoly : poly distr = dlist dcoeff n.`	Deterministic algebraic map or predicate dpoly over HAETAE module/ring objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Distributions.ec:57	op	`op dpolyveck (md : mode) : polyveck distr = dlist dpoly (mode_k md).`	Deterministic algebraic map or predicate dpolyveck over HAETAE module/ring objects.
HAETAE_Distributions.ec:58	op	`op dpolyvecl (md : mode) : polyvecl distr = dlist dpoly (mode_l md).`	Deterministic algebraic map or predicate dpolyvecl over HAETAE module/ring objects.
HAETAE_Distributions.ec:59	op	`op dmatrix (md : mode) : matrix distr = dlist (dpolyvecl md) (mode_k md).`	Deterministic algebraic map or predicate dmatrix over HAETAE module/ring objects.
HAETAE_Distributions.ec:65	op	`op dchallenge (md : mode) : challenge distr =`	Probability law or sampler dchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:68	type	`type signing_sample_pair = polyvecl * polyveck.`	Signature space Σ; values of signing_sample_pair denote candidate HAETAE signatures.
HAETAE_Distributions.ec:70	op	`op signing_sample_pair_y1 (s : signing_sample_pair) : polyvecl = s.‘1.`	Probability law or sampler signingsamplepairy1; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:71	op	`op signing_sample_pair_y2 (s : signing_sample_pair) : polyveck = s.‘2.`	Probability law or sampler signingsamplepairy2; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:73	op	`op signing_sample_pair_wf (md : mode) (s : signing_sample_pair) : bool =`	Probability law or sampler signingsamplepairwf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:77	op	`op signing_sample_pair_unit (md : mode) (s : signing_sample_pair) : bool =`	Probability law or sampler signingsamplepairunit; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:81	op	`op signing_sample_bound (md : mode) : int = mode_ln md.`	Numerical bound or budget parameter signingsamplebound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:83	op	`op coeff_bounded_by (b : int) (x : coeff) : bool =`	Numerical bound or budget parameter coeffboundedby used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:86	op	`op poly_bounded_by (b : int) (p : poly) : bool =`	Numerical bound or budget parameter polyboundedby used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:91	op	`op polyvecl_bounded_by (md : mode) (b : int) (xs : polyvecl) : bool =`	Numerical bound or budget parameter polyveclboundedby used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:96	op	`op polyveck_bounded_by (md : mode) (b : int) (xs : polyveck) : bool =`	Numerical bound or budget parameter polyveckboundedby used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:101	op	`op signing_sample_pair_bounded (md : mode) (s : signing_sample_pair) : bool =`	Numerical bound or budget parameter signingsamplepairbounded used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:107	op	`op signing_sample_pair_sample_ok (md : mode) (s : signing_sample_pair) : bool =`	Probability law or sampler signingsamplepairsampleok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:111	op	`op signing_sample_pair_of_coins`	Probability law or sampler signingsamplepairofcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:117	op	`op dstructural_signing_sample_pair`	Probability law or sampler dstructuralsigningsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:122	op	`op dsigning_unit_coeff : coeff distr = duniform [-1; 1].`	Deterministic algebraic map or predicate dsigningunitcoeff over HAETAE module/ring objects.
HAETAE_Distributions.ec:123	op	`op dsigning_unit_poly : poly distr = dlist dsigning_unit_coeff n.`	Deterministic algebraic map or predicate dsigningunitpoly over HAETAE module/ring objects.
HAETAE_Distributions.ec:124	op	`op dsigning_unit_polyvecl (md : mode) : polyvecl distr =`	Deterministic algebraic map or predicate dsigningunitpolyvecl over HAETAE module/ring objects.
HAETAE_Distributions.ec:126	op	`op dsigning_unit_polyveck (md : mode) : polyveck distr =`	Deterministic algebraic map or predicate dsigningunitpolyveck over HAETAE module/ring objects.
HAETAE_Distributions.ec:129	op	`op signing_bounded_coeff_cardinality (md : mode) : int =`	Numerical bound or budget parameter signingboundedcoeffcardinality used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:131	op	`op signing_bounded_coeff_point_bound (md : mode) : real =`	Numerical bound or budget parameter signingboundedcoeffpointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:133	op	`op dsigning_bounded_coeff (md : mode) : coeff distr =`	Numerical bound or budget parameter dsigningboundedcoeff used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:135	op	`op dsigning_bounded_poly (md : mode) : poly distr =`	Numerical bound or budget parameter dsigningboundedpoly used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:137	op	`op dsigning_bounded_polyvecl (md : mode) : polyvecl distr =`	Numerical bound or budget parameter dsigningboundedpolyvecl used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:139	op	`op dsigning_bounded_polyveck (md : mode) : polyveck distr =`	Numerical bound or budget parameter dsigningboundedpolyveck used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:145	op	`op dbounded_unit_signing_sample_pair (md : mode) :`	Numerical bound or budget parameter dboundedunitsigningsamplepair used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:156	op	`op dchecked_hyperball_signing_sample_pair (md : mode) :`	Probability law or sampler dcheckedhyperballsigningsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:163	op	`op hyperball_coeff_ok (md : mode) (c : coeff) : bool =`	Deterministic algebraic map or predicate hyperballcoeffok over HAETAE module/ring objects.
HAETAE_Distributions.ec:165	op	`op hyperball_poly_ok (md : mode) (p : poly) : bool =`	Deterministic algebraic map or predicate hyperballpolyok over HAETAE module/ring objects.
HAETAE_Distributions.ec:167	op	`op hyperball_polyvecl_ok (md : mode) (y1 : polyvecl) : bool =`	Deterministic algebraic map or predicate hyperballpolyveclok over HAETAE module/ring objects.
HAETAE_Distributions.ec:169	op	`op hyperball_polyveck_ok (md : mode) (y2 : polyveck) : bool =`	Deterministic algebraic map or predicate hyperballpolyveckok over HAETAE module/ring objects.
HAETAE_Distributions.ec:171	op	`op hyperball_sample_ok (md : mode) (s : signing_sample_pair) : bool =`	Probability law or sampler hyperballsampleok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:175	op	`op dhyperball_coeff (md : mode) : coeff distr =`	Deterministic algebraic map or predicate dhyperballcoeff over HAETAE module/ring objects.
HAETAE_Distributions.ec:177	op	`op dhyperball_poly (md : mode) : poly distr =`	Deterministic algebraic map or predicate dhyperballpoly over HAETAE module/ring objects.
HAETAE_Distributions.ec:179	op	`op dhyperball_polyvecl (md : mode) : polyvecl distr =`	Deterministic algebraic map or predicate dhyperballpolyvecl over HAETAE module/ring objects.
HAETAE_Distributions.ec:181	op	`op dhyperball_polyveck (md : mode) : polyveck distr =`	Deterministic algebraic map or predicate dhyperballpolyveck over HAETAE module/ring objects.
HAETAE_Distributions.ec:184	op	`op hyperball_coeff_point_bound (md : mode) : real =`	Numerical bound or budget parameter hyperballcoeffpointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:186	op	`op hyperball_poly_point_bound (md : mode) : real =`	Numerical bound or budget parameter hyperballpolypointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:188	op	`op hyperball_polyvecl_point_bound (md : mode) : real =`	Numerical bound or budget parameter hyperballpolyveclpointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:190	op	`op hyperball_polyveck_point_bound (md : mode) : real =`	Numerical bound or budget parameter hyperballpolyveckpointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Distributions.ec:192	op	`op hyperball_sample_pair_point_bound (md : mode) : real =`	Numerical bound or budget parameter hyperballsamplepairpointbound used to upper-bound a probability loss in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:201	op	`op dexact_hyperball_signing_sample_pair (md : mode) :`	Probability law or sampler dexacthyperballsigningsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:205	op	`op signing_sample_pair_distribution_ok`	Probability law or sampler signingsamplepairdistributionok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Distributions.ec:211	lemma	`lemma byte_distribution_lossless : is_lossless dbyte.`	Probability bound $\Pr[\text{bytedistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:214	lemma	`lemma seed_distribution_lossless : is_lossless dseed.`	Probability bound $\Pr[\text{seeddistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:220	lemma	`lemma crh_distribution_lossless : is_lossless dcrh.`	Probability bound $\Pr[\text{crhdistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:226	lemma	`lemma signing_entropy_token_cardinality_positive :`	Checked theorem signingentropytokencardinalitypositive in the probability distributions for coins, challenges, and sampled objects; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Distributions.ec:230	lemma	`lemma signing_entropy_token_distribution_lossless :`	Probability bound $\Pr[\text{signingentropytokendistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:238	lemma	`lemma signing_randomness_source_distribution_lossless :`	Probability bound $\Pr[\text{signingrandomnesssourcedistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:245	lemma	`lemma signing_coin_distribution_lossless : is_lossless drandom_coins.`	Probability bound $\Pr[\text{signingcoindistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:251	lemma	`lemma signing_entropy_token_to_coins_tokenK x :`	Checked theorem signingentropytokentocoinstokenK in the probability distributions for coins, challenges, and sampled objects; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Distributions.ec:260	lemma	`lemma signing_randomness_from_source_tokenK src :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Distributions.ec:267	lemma	`lemma signing_entropy_token_to_coins_size x :`	Checked theorem signingentropytokentocoinssize in the probability distributions for coins, challenges, and sampled objects; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Distributions.ec:279	lemma	`lemma signing_entropy_token_to_coins_byte_domain x :`	Theorem signingentropytokentocoinsbytedomain contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Distributions.ec:292	lemma	`lemma signing_entropy_token_to_coins_domain x :`	Theorem signingentropytokentocoinsdomain contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Distributions.ec:305	lemma	`lemma signing_randomness_from_source_domain src :`	Theorem signingrandomnessfromsourcedomain contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Distributions.ec:314	lemma	`lemma signing_entropy_token_distribution_point_bound x :`	Probability bound $\Pr[\text{signingentropytokendistributionpointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:326	lemma	`lemma signing_randomness_source_distribution_point_bound src :`	Probability bound $\Pr[\text{signingrandomnesssourcedistributionpointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:334	lemma	`lemma signing_coin_distribution_token_point_bound token :`	Probability bound $\Pr[\text{signingcoindistributiontokenpointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:348	lemma	`lemma signing_coin_distribution_token_spread :`	Checked theorem signingcoindistributiontokenspread in the probability distributions for coins, challenges, and sampled objects; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Distributions.ec:355	lemma	`lemma signing_coin_distribution_domain coins :`	Theorem signingcoindistributiondomain contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Distributions.ec:371	lemma	`lemma signing_coin_distribution_domain_full :`	Theorem signingcoindistributiondomainfull contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Distributions.ec:379	lemma	`lemma signing_coin_distribution_ok :`	Checked theorem signingcoindistributionok in the probability distributions for coins, challenges, and sampled objects; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Distributions.ec:390	lemma	`lemma coeff_distribution_lossless : is_lossless dcoeff.`	Probability bound $\Pr[\text{coeffdistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:393	lemma	`lemma poly_distribution_lossless : is_lossless dpoly.`	Probability bound $\Pr[\text{polydistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:399	lemma	`lemma polyveck_distribution_lossless md : is_lossless (dpolyveck md).`	Probability bound $\Pr[\text{polyveckdistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:405	lemma	`lemma polyvecl_distribution_lossless md : is_lossless (dpolyvecl md).`	Probability bound $\Pr[\text{polyvecldistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:411	lemma	`lemma matrix_distribution_lossless md : is_lossless (dmatrix md).`	Probability bound $\Pr[\text{matrixdistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:417	lemma	`lemma challenge_distribution_lossless md : is_lossless (dchallenge md).`	Probability bound $\Pr[\text{challengedistributionlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:423	lemma	`lemma signing_sample_pair_of_coins_wf md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:434	lemma	`lemma signing_sample_pair_of_coins_unit md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:445	lemma	`lemma signing_sample_bound_gt0 md :`	Probability bound $\Pr[\text{signingsampleboundgt0}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:449	lemma	`lemma unit_coeff_bounded_by_signing_sample_bound md c :`	Probability bound $\Pr[\text{unitcoeffboundedbysigningsamplebound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:457	lemma	`lemma poly_unit_bounded_by_signing_sample_bound md p :`	Probability bound $\Pr[\text{polyunitboundedbysigningsamplebound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:470	lemma	`lemma polyvecl_unit_bounded_by_signing_sample_bound md y1 :`	Probability bound $\Pr[\text{polyveclunitboundedbysigningsamplebound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:482	lemma	`lemma polyveck_unit_bounded_by_signing_sample_bound md y2 :`	Probability bound $\Pr[\text{polyveckunitboundedbysigningsamplebound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:494	lemma	`lemma signing_sample_pair_unit_bounded md s :`	Probability bound $\Pr[\text{signingsamplepairunitbounded}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:505	lemma	`lemma signing_sample_pair_of_coins_sample_ok md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:516	lemma	`lemma structural_signing_sample_pair_lossless md sk m ctx coins :`	Probability bound $\Pr[\text{structuralsigningsamplepairlossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:523	lemma	`lemma structural_signing_sample_pair_support md sk m ctx coins s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:534	lemma	`lemma structural_signing_sample_pair_sample_ok md sk m ctx coins s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Distributions.ec:543	lemma	`lemma structural_signing_sample_pair_distribution_ok md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:553	lemma	`lemma signing_unit_coeff_lossless :`	Probability bound $\Pr[\mathsf{signingunitcoefflossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:560	lemma	`lemma signing_unit_coeff_support c :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Distributions.ec:568	lemma	`lemma signing_bounded_coeff_cardinality_positive md :`	Probability bound $\Pr[\mathsf{signingboundedcoeffcardinalitypositive}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:575	lemma	`lemma signing_bounded_coeff_lossless md :`	Probability bound $\Pr[\mathsf{signingboundedcoefflossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:583	lemma	`lemma signing_bounded_coeff_support md c :`	Probability bound $\Pr[\mathsf{signingboundedcoeffsupport}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:590	lemma	`lemma signing_bounded_coeff_supportP md c :`	Probability bound $\Pr[\mathsf{signingboundedcoeffsupportP}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:597	lemma	`lemma signing_bounded_coeff_point_bound md c :`	Probability bound $\Pr[\mathsf{signingboundedcoeffpointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:616	lemma	`lemma signing_bounded_coeff_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{signingboundedcoeffpointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:625	lemma	`lemma dlist_size_point_bound ['a] (d : 'a distr) bd xs :`	Probability bound $\Pr[\mathsf{dlistsizepointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:647	lemma	`lemma dlist_point_bound ['a] (d : 'a distr) len bd xs :`	Probability bound $\Pr[\mathsf{dlistpointbound}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:661	lemma	`lemma hyperball_coeff_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{hyperballcoeffpointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:668	lemma	`lemma hyperball_coeff_point_boundE md c :`	Probability bound $\Pr[\mathsf{hyperballcoeffpointboundE}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:676	lemma	`lemma signing_unit_poly_lossless :`	Probability bound $\Pr[\mathsf{signingunitpolylossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:683	lemma	`lemma signing_unit_poly_support p :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Distributions.ec:695	lemma	`lemma signing_bounded_poly_lossless md :`	Probability bound $\Pr[\mathsf{signingboundedpolylossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:702	lemma	`lemma signing_bounded_poly_support md p :`	Probability bound $\Pr[\mathsf{signingboundedpolysupport}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:717	lemma	`lemma signing_bounded_poly_supportP md p :`	Probability bound $\Pr[\mathsf{signingboundedpolysupportP}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:738	lemma	`lemma hyperball_poly_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{hyperballpolypointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:745	lemma	`lemma hyperball_poly_lossless md :`	Probability bound $\Pr[\mathsf{hyperballpolylossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:751	lemma	`lemma hyperball_poly_support md p :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:759	lemma	`lemma hyperball_poly_supportP md p :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:766	lemma	`lemma hyperball_poly_point_boundE md p :`	Probability bound $\Pr[\mathsf{hyperballpolypointboundE}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:778	lemma	`lemma signing_unit_polyvecl_lossless md :`	Probability bound $\Pr[\mathsf{signingunitpolyvecllossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:785	lemma	`lemma signing_unit_polyveck_lossless md :`	Probability bound $\Pr[\mathsf{signingunitpolyvecklossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:792	lemma	`lemma signing_bounded_polyvecl_lossless md :`	Probability bound $\Pr[\mathsf{signingboundedpolyvecllossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:799	lemma	`lemma signing_bounded_polyveck_lossless md :`	Probability bound $\Pr[\mathsf{signingboundedpolyvecklossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:806	lemma	`lemma signing_unit_polyvecl_support md y1 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Distributions.ec:825	lemma	`lemma signing_unit_polyveck_support md y2 :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Distributions.ec:844	lemma	`lemma signing_bounded_polyvecl_support md y1 :`	Probability bound $\Pr[\mathsf{signingboundedpolyveclsupport}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:865	lemma	`lemma signing_bounded_polyvecl_supportP md y1 :`	Probability bound $\Pr[\mathsf{signingboundedpolyveclsupportP}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:892	lemma	`lemma signing_bounded_polyveck_support md y2 :`	Probability bound $\Pr[\mathsf{signingboundedpolyvecksupport}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:913	lemma	`lemma signing_bounded_polyveck_supportP md y2 :`	Probability bound $\Pr[\mathsf{signingboundedpolyvecksupportP}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:940	lemma	`lemma hyperball_polyvecl_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{hyperballpolyveclpointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:947	lemma	`lemma hyperball_polyveck_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{hyperballpolyveckpointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:954	lemma	`lemma hyperball_sample_pair_point_bound_nonnegative md :`	Probability bound $\Pr[\mathsf{hyperballsamplepairpointboundnonnegative}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:963	lemma	`lemma hyperball_polyvecl_lossless md :`	Probability bound $\Pr[\mathsf{hyperballpolyvecllossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:969	lemma	`lemma hyperball_polyveck_lossless md :`	Probability bound $\Pr[\mathsf{hyperballpolyvecklossless}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:975	lemma	`lemma hyperball_polyvecl_support md y1 :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:983	lemma	`lemma hyperball_polyvecl_supportP md y1 :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:990	lemma	`lemma hyperball_polyveck_support md y2 :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:998	lemma	`lemma hyperball_polyveck_supportP md y2 :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1005	lemma	`lemma hyperball_polyvecl_point_boundE md y1 :`	Probability bound $\Pr[\mathsf{hyperballpolyveclpointboundE}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1017	lemma	`lemma hyperball_polyveck_point_boundE md y2 :`	Probability bound $\Pr[\mathsf{hyperballpolyveckpointboundE}] \le B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Distributions.ec:1029	lemma	`lemma bounded_unit_signing_sample_pair_lossless md :`	Probability bound Pr[boundedunitsigningsamplepairlossless] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1039	lemma	`lemma bounded_unit_signing_sample_pair_support md s :`	Probability bound Pr[boundedunitsigningsamplepairsupport] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1060	lemma	`lemma bounded_unit_signing_sample_pair_sample_ok md s :`	Probability bound Pr[boundedunitsigningsamplepairsampleok] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1072	lemma	`lemma bounded_unit_signing_sample_pair_distribution_ok md :`	Probability bound Pr[boundedunitsigningsamplepairdistributionok] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1082	lemma	`lemma checked_hyperball_signing_sample_pair_lossless md :`	Probability bound Pr[checkedhyperballsigningsamplepairlossless] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1092	lemma	`lemma checked_hyperball_signing_sample_pair_support md s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1114	lemma	`lemma checked_hyperball_signing_sample_pair_distribution_ok md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1124	lemma	`lemma hyperball_sample_ok_sample_ok md s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1139	lemma	`lemma exact_hyperball_signing_sample_pair_lossless md :`	Probability bound Pr[exacthyperballsigningsamplepairlossless] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1149	lemma	`lemma exact_hyperball_signing_sample_pair_support md s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1163	lemma	`lemma exact_hyperball_signing_sample_pair_supportP md s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1173	lemma	`lemma exact_hyperball_signing_sample_pair_sample_ok md s :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1182	lemma	`lemma exact_hyperball_signing_sample_pair_distribution_ok md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Distributions.ec:1192	lemma	`lemma exact_hyperball_signing_sample_pair_point_bound md s :`	Probability bound Pr[exacthyperballsigningsamplepairpointbound] $\leq B$ or loss inequality controlling a bad event in the probability distributions for coins, challenges, and sampled objects.
HAETAE_Distributions.ec:1206	lemma	`lemma exact_hyperball_signing_sample_pair_checkedE md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:16	type	`type signing_transcript_record = message * context * signature * transcript.`	Signature space Σ; values of `signing_transcript_record` denote candidate HAETAE signatures.
HAETAE_Rejection.ec:17	type	`type signing_attempt_sample = polyvecl * polyveck * polyveck.`	Signature space Σ; values of `signing_attempt_sample` denote candidate HAETAE signatures.
HAETAE_Rejection.ec:18	type	`type signing_attempt_state =`	Signature space Σ; values of `signing_attempt_state` denote candidate HAETAE signatures.
HAETAE_Rejection.ec:22	op	`op signing_attempt_state_coins`	Probability law or sampler signingattemptstatecoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:25	op	`op signing_attempt_state_sample`	Probability law or sampler signingattemptstatesample; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:28	op	`op signing_attempt_sample_y1 (s : signing_attempt_sample) : polyvecl =`	Probability law or sampler signingattemptsampley1; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:30	op	`op signing_attempt_sample_y2 (s : signing_attempt_sample) : polyveck =`	Probability law or sampler signingattemptsampley2; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:32	op	`op signing_attempt_sample_commitment_raw`	Probability law or sampler signingattemptsamplecommitmentraw; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:35	op	`op signing_attempt_state_sampled_y1`	Probability law or sampler signingattemptstatesampledy1; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:38	op	`op signing_attempt_state_sampled_y2`	Probability law or sampler signingattemptstatesampledy2; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:41	op	`op signing_attempt_state_sampled_y`	Probability law or sampler signingattemptstatesampledy; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:44	op	`op signing_attempt_state_highbits`	Total mathematical function or predicate signingattemptstatehighbits in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:47	op	`op signing_attempt_state_lowbits`	Total mathematical function or predicate signingattemptstatelowbits in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:50	op	`op signing_attempt_state_challenge`	Probability law or sampler signingattemptstatechallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:53	op	`op signing_attempt_state_response`	Total mathematical function or predicate signingattemptstateresponse in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:56	op	`op signing_attempt_state_response_aux`	Total mathematical function or predicate signingattemptstateresponseaux in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:59	op	`op signing_attempt_state_hint`	Total mathematical function or predicate signingattemptstatehint in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:62	op	`op signing_attempt_state_signature`	Total mathematical function or predicate signingattemptstatesignature in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:65	op	`op signature_rejection_branch_byte (sig : signature) : int =`	Total mathematical function or predicate signaturerejectionbranchbyte in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:67	op	`op signing_attempt_state_reject2_branch_byte`	Total mathematical function or predicate signingattemptstatereject2branchbyte in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:70	op	`op signing_attempt_state_reject2_branch_bit`	Total mathematical function or predicate signingattemptstatereject2branchbit in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:73	op	`op polyvecl_double (md : mode) (xs : polyvecl) : polyvecl =`	Deterministic algebraic map or predicate polyvecldouble over HAETAE module/ring objects.
HAETAE_Rejection.ec:77	op	`op signing_attempt_state_reject2_sampled_y1`	Probability law or sampler signingattemptstatereject2sampledy1; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:80	op	`op signing_attempt_state_reject2_sampled_y2`	Probability law or sampler signingattemptstatereject2sampledy2; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec:83	op	`op signing_attempt_state_reject2_balance_response`	Total mathematical function or predicate signingattemptstatereject2balanceresponse in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:88	op	`op signing_attempt_state_reject2_balance_aux`	Total mathematical function or predicate signingattemptstatereject2balanceaux in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:93	op	`op signing_attempt_state_lowbits_carrier`	Total mathematical function or predicate signingattemptstatelowbitscarrier in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:96	op	`op signing_attempt_state_token`	Total mathematical function or predicate signingattemptstatetoken in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:102	op	`op signing_attempt_state_programmed_lowbits`	Total mathematical function or predicate signingattemptstateprogrammedlowbits in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:111	op	`op signing_attempt_state_lowbits_consistent`	Total mathematical function or predicate signingattemptstatelowbitsconsistent in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec:116	op	`op signing_attempt_state_programmed_challenge`	Probability law or sampler signingattemptstateprogrammedchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec: 124	op	`op signing_attempt_state_challenge_consistent`	Probability law or sampler signingattemptstatechallengeconsistent; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 130	op	`op signing_attempt_state_reconstructed_highbits`	Total mathematical function or predicate signingattemptstatereconstructedhighbits in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 136	op	`op signing_attempt_state_public_equation_consistent`	Total mathematical function or predicate signingattemptstatepublicequationconsistent in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 141	op	`op signing_attempt_state_signature_of_fields`	Total mathematical function or predicate signingattemptstatesignatureoffields in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 152	op	`op signing_attempt_state_fields_match_signature`	Total mathematical function or predicate signingattemptstatefieldsmatchsignature in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 158	op	`op signing_attempt_state_field_accepts`	Predicate/event signingattemptstatefieldaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatefieldaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 166	op	`op signing_attempt_state_of_coins`	Probability law or sampler signingattemptstateofcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 181	op	`op signing_attempt_sample_of_pair`	Probability law or sampler signingattemptsampleofpair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 189	op	`op signing_attempt_state_of_sample_pair`	Probability law or sampler signingattemptstateofsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 203	op	`op signing_attempt_state_valid_signature_accepts`	Predicate/event signingattemptstatevalidsignatureaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatevalidsignatureaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 207	op	`op signing_attempt_state_response_norm_accepts`	Predicate/event signingattemptstateresponsenormaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstateresponsenormaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 211	op	`op signing_attempt_state_hint_norm_accepts`	Predicate/event signingattemptstatehintnormaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatehintnormaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 215	op	`op signing_attempt_state_accepts`	Predicate/event signingattemptstateaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstateaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 221	op	`op signing_attempt_reject1_bound (md : mode) : int =`	Numerical bound or budget parameter signingattemptreject1bound used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 224	op	`op signing_attempt_reject2_bound (md : mode) : int =`	Numerical bound or budget parameter signingattemptreject2bound used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 227	op	`op signing_attempt_reject2_balance_norm_sq`	Deterministic algebraic map or predicate signingattemptreject2balancenormsq over HAETAE module/ring objects.
HAETAE_Rejection.ec: 233	op	`op signing_attempt_reject2_concrete_aborts`	Total mathematical function or predicate signingattemptreject2concreteaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 240	op	`op signing_attempt_state_reject1_norm_sq`	Deterministic algebraic map or predicate signingattemptstatereject1normsq over HAETAE module/ring objects.
HAETAE_Rejection.ec: 244	op	`op signing_attempt_state_reject1_accepts`	Predicate/event signingattemptstatereject1accepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatereject1accepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 250	op	`op signing_attempt_state_balance_norm_sq`	Deterministic algebraic map or predicate signingattemptstatebalancenormsq over HAETAE module/ring objects.
HAETAE_Rejection.ec: 258	op	`op signing_attempt_state_bimodal_reject2_branch`	Total mathematical function or predicate signingattemptstatebimodalreject2branch in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 262	op	`op signing_attempt_state_reject2_under_bound`	Numerical bound or budget parameter signingattemptstatereject2underbound used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 267	op	`op signing_attempt_state_reject2_aborts`	Total mathematical function or predicate signingattemptstatereject2aborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 276	op	`op signing_attempt_state_reject2_accepts`	Predicate/event signingattemptstatereject2accepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatereject2accepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 280	op	`op signing_attempt_state_pack_accepts`	Predicate/event signingattemptstatepackaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatepackaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 285	op	`op signing_attempt_state_reference_rejection_accepts`	Predicate/event signingattemptstatereferencerejectionaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstatereferencerejectionaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 292	op	`op signing_attempt_state_reference_rejection_aborts`	Total mathematical function or predicate signingattemptstatereferencerejectionaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 296	op	`op signing_attempt_state_reject1_aborts`	Total mathematical function or predicate signingattemptstatereject1aborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 300	op	`op signing_attempt_state_pack_aborts`	Total mathematical function or predicate signingattemptstatepackaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 304	op	`op signing_attempt_state_verification_accepts`	Predicate/event signingattemptstateverificationaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstateverificationaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 310	op	`op signing_attempt_state_rejection_accepts`	Predicate/event signingattemptstaterejectionaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingattemptstaterejectionaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 316	op	`op signing_attempt_state_rejection_aborts`	Total mathematical function or predicate signingattemptstaterejectionaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 321	op	`op signing_attempt_state_response_norm_aborts`	Deterministic algebraic map or predicate signingattemptstateresponsenormaborts over HAETAE module/ring objects.
HAETAE_Rejection.ec: 325	op	`op signing_attempt_state_hint_norm_aborts`	Deterministic algebraic map or predicate signingattemptstatehintnormaborts over HAETAE module/ring objects.
HAETAE_Rejection.ec: 329	op	`op signing_attempt_state_rejection_sampling_aborts`	Total mathematical function or predicate signingattemptstaterejectionsamplingaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 333	op	`op signing_attempt_state_aborts`	Total mathematical function or predicate signingattemptstateaborts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 339	op	`op signing_accepts`	Predicate/event signingaccepts on the game state; in probability expressions it denotes the indicator event $1_{\text{signingaccepts}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 345	op	`op dsigning_attempt_state`	Total mathematical function or predicate dsigningattemptstate in the rejection-sampling predicates and distributional loss bounds.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec: 350	op	`op signing_sample_pair_sampler_ok`	Probability law or sampler signingsamplepairsamplerok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 357	op	`op dsigning_attempt_state_from_sample_pair_sampler`	Probability law or sampler dsigningattemptstatefromsamplepairsampler; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 366	type	`type signing_attempt_coin_sample_pair = random_coins * signing_sample_pair.`	Signature space Σ; values of signing_attempt_coin_sample_pair denote candidate HAETAE signatures.
HAETAE_Rejection.ec: 368	op	`op signing_attempt_state_of_coin_sample_pair`	Probability law or sampler signingattemptstateofcoinsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 373	op	`op dstructural_signing_attempt_coin_sample_pair`	Probability law or sampler dstructuralsigningattemptcoinsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 381	op	`op dexact_hyperball_signing_attempt_coin_sample_pair`	Probability law or sampler dexacthyperballsigningattemptcoinsamplepair; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 389	op	`op dstructural_signing_attempt_state`	Total mathematical function or predicate dstructuralsigningattemptstate in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 397	op	`op dstructural_signing_attempt_state_from_sampler`	Probability law or sampler dstructuralsigningattemptstatefromsampler; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 403	op	`op dbounded_unit_signing_attempt_state`	Numerical bound or budget parameter dboundedunitsigningattemptstate used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 409	op	`op dchecked_hyperball_signing_attempt_state`	Total mathematical function or predicate dcheckedhyperballsigningattemptstate in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 415	op	`op dexact_hyperball_signing_attempt_state`	Total mathematical function or predicate dexacthyperballsigningattemptstate in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 421	op	`op signing_attempt_state_sample_pair_wf`	Probability law or sampler signingattemptstatesamplepairwf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 426	op	`op signing_attempt_state_sample_pair_unit`	Probability law or sampler signingattemptstatesamplepairunit; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Rejection.ec: 431	op	`op signing_attempt_state_sample_pair_bounded`	Numerical bound or budget parameter signingattemptstatesamplepairbounded used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 438	op	`op signing_attempt_relation`	Total mathematical function or predicate signingattemptrelation in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 445	op	`op signing_simulation_relation`	Total mathematical function or predicate signingsimulationrelation in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 452	op	`op signing_transcript_relation`	Total mathematical function or predicate signingtranscriptrelation in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 459	op	`op signing_record_message (r : signing_transcript_record) : message = r.‘1.`	Total mathematical function or predicate signingrecordmessage in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 460	op	`op signing_record_context (r : signing_transcript_record) : context = r.‘2.`	Total mathematical function or predicate signingrecordcontext in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 461	op	`op signing_record_signature (r : signing_transcript_record) : signature = r.‘3.`	Total mathematical function or predicate signingrecordsignature in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 462	op	`op signing_record_transcript (r : signing_transcript_record) : transcript =`	Total mathematical function or predicate signingrecordtranscript in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 465	op	`op signing_record_query (r : signing_transcript_record) : message * context =`	Oracle-related function signingrecordquery used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Rejection.ec: 468	op	`op signing_record_relation`	Total mathematical function or predicate signingrecordrelation in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 476	op	`op signing_record_queries (rs : signing_transcript_record list) :`	Numerical bound or budget parameter signingrecordqueries used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 480	op	`op signing_record_transcripts (rs : signing_transcript_record list) :`	Total mathematical function or predicate signingrecordtranscripts in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 484	op	`op signing_record_log_sound`	Total mathematical function or predicate signingrecordlogsound in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 488	op	`op transcript_log_valid (md : mode) (trs : transcript list) : bool =`	Predicate/event transcriptlogvalid on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{transcriptlogvalid}}$. *Context:* rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 491	op	`op structural_to_exact_hyperball_sample_pair_point_loss_obligation : bool =`	Numerical bound or budget parameter structuraltoexacthyperballsamplepairpointlossobligation used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 499	op	`op structural_to_exact_hyperball_sample_pair_loss_obligation : bool =`	Numerical bound or budget parameter structuraltoexacthyperballsamplepairlossobligation used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 506	op	`op structural_to_exact_hyperball_coin_sample_pair_loss_obligation : bool =`	Numerical bound or budget parameter structuraltoexacthyperballcoinsamplepairlossobligation used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 513	op	`op structural_to_exact_hyperball_attempt_loss_obligation : bool =`	Numerical bound or budget parameter structuraltoexacthyperballattemptlossobligation used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 520	op	`op rejection_sampling_bound_obligation : bool =`	Numerical bound or budget parameter rejectionsamplingboundobligation used to upper-bound a probability loss in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 525	lemma	`lemma signing_attempt_verifies md pk sk m ctx coins sig :`	Checked theorem signingattemptverifies in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 535	lemma	`lemma signing_simulation_verifies md pk m ctx sig :`	Checked theorem signingsimulationverifies in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 542	lemma	`lemma signing_attempt_simulates md pk sk m ctx coins sig :`	Checked theorem signingattemptsimulates in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 554	lemma	`lemma sign_internal_attempt_relation md sk m ctx coins :`	Checked theorem signinternalattemptrelation in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 571	lemma	`lemma signing_attempt_sample_of_pair_currentE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 583	lemma	`lemma signing_attempt_state_of_sample_pair_currentE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 593	lemma	`lemma structural_signing_sample_pair_sampler_ok md sk m ctx :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 601	lemma	`lemma bounded_unit_signing_sample_pair_sampler_ok md :`	Probability bound $\Pr[\mathsf{boundedunitsigningsamplepairsamplerok}] \le B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 609	lemma	`lemma checked_hyperball_signing_sample_pair_sampler_ok md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 617	lemma	`lemma exact_hyperball_signing_sample_pair_sampler_ok md :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec: 625	lemma	`lemma dsigning_attempt_state_from_sample_pair_sampler_lossless`	Probability bound Pr[dsigningattemptstatefromsamplepairsamplerlossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 643	lemma	`lemma dsigning_attempt_state_from_sample_pair_sampler_sample_ok`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 678	lemma	`lemma dstructural_signing_attempt_state_lossless md sk m ctx :`	Probability bound Pr[dstructuralsigningattemptstatelossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 685	lemma	`lemma dstructural_signing_attempt_state_from_sampler_lossless`	Probability bound Pr[dstructuralsigningattemptstatefromsamplerlossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 695	lemma	`lemma dbounded_unit_signing_attempt_state_lossless md sk m ctx :`	Probability bound Pr[dboundedunitsigningattemptstatelossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 703	lemma	`lemma dbounded_unit_signing_attempt_state_sample_ok md sk m ctx st :`	Probability bound Pr[dboundedunitsigningattemptstatesampleok] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 713	lemma	`lemma dchecked_hyperball_signing_attempt_state_lossless md sk m ctx :`	Probability bound Pr[dcheckedhyperballsigningattemptstatelossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 721	lemma	`lemma dchecked_hyperball_signing_attempt_state_sample_ok md sk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 731	lemma	`lemma dexact_hyperball_signing_attempt_state_lossless md sk m ctx :`	Probability bound Pr[dexacthyperballsigningattemptstatelossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 739	lemma	`lemma dexact_hyperball_signing_attempt_state_sample_ok md sk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 749	lemma	`lemma dexact_hyperball_signing_attempt_state_supportP md sk m ctx st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 778	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1`	Probability bound Pr[dexacthyperballsigningattemptstatereferencerejectionabortbound1] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 786	lemma	`lemma dexact_hyperball_signing_attempt_state_mu_pullback`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 801	lemma	`lemma signing_attempt_state_reference_rejection_aborts_componentE md st :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 817	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_split`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 834	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound`	Probability bound Pr[dexacthyperballsigningattemptstatereferencerejectionabortunionbound] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 852	lemma	`lemma signing_attempt_state_of_sample_pair_signatureE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 862	lemma	`lemma signing_attempt_state_of_sample_pair_pack_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 875	lemma	`lemma signing_attempt_state_of_sample_pair_pack_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 884	lemma	`lemma dexact_hyperball_signing_attempt_state_pack_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 904	lemma	`lemma dstructural_signing_attempt_state_from_samplerE md sk m ctx :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 925	lemma	`lemma dstructural_signing_attempt_stateE md sk m ctx :`	Checked theorem dstructuralsigningattemptstateE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 934	lemma	`lemma dstructural_signing_attempt_state_from_sampler_coin_sample_pairE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 950	lemma	`lemma dexact_hyperball_signing_attempt_state_coin_sample_pairE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 966	lemma	`lemma mu_dlet_le_add_all ['a 'b 'c]`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Rejection.ec: 1004	lemma	`lemma structural_to_exact_hyperball_sample_pair_loss_from_point_loss :`	Probability bound Pr[structuraltoexacthyperballsamplepairlossfrompointloss] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1023	lemma	`lemma structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss :`	Probability bound Pr[structuraltoexacthyperballcoinsamplepairlossfromsamplepairloss] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1039	lemma	`lemma structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss :`	Probability bound Pr[structuraltoexacthyperballattemptlossfromcoinsamplepairloss] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1055	lemma	`lemma signing_attempt_state_of_coins_signatureE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinssignatureE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1062	lemma	`lemma signing_attempt_state_of_coins_sampled_y1E md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1073	lemma	`lemma signing_attempt_state_of_coins_sampled_y2E md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1084	lemma	`lemma signing_attempt_state_of_coins_sampled_yE md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1095	lemma	`lemma signing_attempt_state_of_coins_highbitsE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinshighbitsE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1102	lemma	`lemma signing_attempt_state_of_coins_lowbitsE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinslowbitsE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1109	lemma	`lemma signing_attempt_state_of_coins_challengeE md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Rejection.ec: 1116	lemma	`lemma signing_attempt_state_of_coins_responseE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsresponseE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1123	lemma	`lemma signing_attempt_state_of_coins_response_auxE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsresponseauxE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1130	lemma	`lemma signing_attempt_state_of_coins_hintE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinshintE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1137	lemma	`lemma signing_attempt_state_of_coins_lowbits_carrierE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinslowbitscarrierE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1146	lemma	`lemma signing_attempt_state_of_coins_tokenE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinstokenE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec: 1158	lemma	`lemma signing_attempt_state_of_coins_programmed_lowbitsE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1169	lemma	`lemma signing_attempt_state_of_coins_lowbits_consistent md sk m ctx coins :`	Checked theorem signingattemptstateofcoinslowbitsconsistent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1178	lemma	`lemma signing_attempt_state_of_coins_programmed_challengeE md sk m ctx coins :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1189	lemma	`lemma signing_attempt_state_of_coins_challenge_consistent md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Rejection.ec: 1198	lemma	`lemma signing_attempt_state_of_coins_reconstructed_highbitsE`	Checked theorem signingattemptstateofcoinsreconstructedhighbitsE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1210	lemma	`lemma signing_attempt_state_of_coins_public_equation_consistent`	Checked theorem signingattemptstateofcoinspublicequationconsistent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1221	lemma	`lemma signing_attempt_state_of_coins_signature_of_fieldsE md sk m ctx coins :`	Checked theorem signingattemptstateofcoinssignatureoffieldsE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1234	lemma	`lemma signing_attempt_state_of_coins_fields_match_signature md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsfieldsmatchsignature in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1244	lemma	`lemma signing_attempt_state_of_coins_field_accepts md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsfieldaccepts in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1258	lemma	`lemma signing_attempt_state_of_coins_accepts_current md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsacceptscurrent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1272	lemma	`lemma signing_attempt_reject1_bound_ge_response_bound md :`	Probability bound $\Pr[\text{signingattemptreject1boundgeresponsebound}] \le B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1276	lemma	`lemma signing_attempt_state_reject1_accepts_from_response_norm_ok md st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1289	lemma	`lemma signing_attempt_state_balance_norm_sq_concreteE md st :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Rejection.ec: 1298	lemma	`lemma signing_attempt_state_reject2_under_bound_concreteE md st :`	Probability bound $\Pr[\text{signingattemptstaterejct2underboundconcreteE}] \le B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1311	lemma	`lemma signing_attempt_state_reject2_aborts_concreteE md st :`	Checked theorem signingattemptstatereject2abortsconcreteE in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1321	lemma	`lemma signing_attempt_state_reject2_accepts_from_branch_clear md st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1332	lemma	`lemma signing_attempt_state_of_sample_pair_reject2_branch_clear`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1345	lemma	`lemma signing_attempt_state_of_sample_pair_reject2_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1354	lemma	`lemma signing_attempt_state_of_sample_pair_reject2_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1364	lemma	`lemma dexact_hyperball_signing_attempt_state_reject2_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1384	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound`	Probability bound $\Pr[\text{dexacthyperballsigningattemptstatereferencerejectionabortreducedbound}] \le B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1401	lemma	`lemma signing_attempt_state_of_sample_pair_response_norm_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1411	lemma	`lemma signing_attempt_state_of_sample_pair_reject1_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1420	lemma	`lemma signing_attempt_state_of_sample_pair_reject1_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1429	lemma	`lemma dexact_hyperball_signing_attempt_state_reject1_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1449	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound`	Probability bound $\Pr[\text{dexacthyperballsigningattemptstatereferencerejectionaborthintbound}] \le B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1463	lemma	`lemma signing_attempt_state_of_sample_pair_hint_norm_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1473	lemma	`lemma signing_attempt_state_of_sample_pair_hint_norm_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1482	lemma	`lemma dexact_hyperball_signing_attempt_state_hint_norm_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1502	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1515	lemma	`lemma signing_attempt_state_of_coins_reject2_branch_clear_current`	Checked theorem signingattemptstateofcoinsreject2branchclearcurrent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1528	lemma	`lemma signing_attempt_state_of_coins_reject2_sampled_y1E md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1537	lemma	`lemma signing_attempt_state_of_coins_reject2_sampled_y2E md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1546	lemma	`lemma signing_attempt_state_of_coins_reject2_balance_norm_sqE`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Rejection.ec: 1563	lemma	`lemma signing_attempt_state_of_coins_balance_norm_sq_current`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Rejection.ec: 1580	lemma	`lemma signing_attempt_state_of_coins_reject2_accepts_current`	Checked theorem signingattemptstateofcoinsreject2acceptscurrent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1589	lemma	`lemma signing_attempt_state_pack_accepts_from_valid md st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1599	lemma	`lemma signing_attempt_state_reference_rejection_accepts_from_accepts md st :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1616	lemma	`lemma signing_attempt_state_of_coins_reference_rejection_accepts_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec: 1626	lemma	`lemma signing_attempt_state_of_coins_verification_accepts_current`	Checked theorem signingattemptstateofcoinsverificationacceptscurrent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1637	lemma	`lemma signing_attempt_state_of_coins_rejection_accepts_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1648	lemma	`lemma signing_attempt_state_of_coins_rejection_no_abort_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1657	lemma	`lemma signing_attempt_state_of_coins_no_abort_current md sk m ctx coins :`	Checked theorem signingattemptstateofcoinsnoabortcurrent in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1674	lemma	`lemma signing_attempt_state_distribution_lossless md sk m ctx :`	Probability bound Pr[signingattemptstatedistributionlossless] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1681	lemma	`lemma signing_attempt_state_distribution_token_spread`	Checked theorem signingattemptstatedistributiontokenspread in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1698	lemma	`lemma signing_attempt_state_distribution_reference_rejection_abort_zero_ current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1712	lemma	`lemma signing_attempt_state_distribution_rejection_abort_zero_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1726	lemma	`lemma sign_internal_simulation_relation md sk m ctx coins :`	Checked theorem signinternalsimulationrelation in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1736	lemma	`lemma signing_attempt_transcript_valid md pk sk m ctx coins sig :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Rejection.ec: 1753	lemma	`lemma signing_attempt_transcript_relation md pk sk m ctx coins sig :`	Checked theorem signingattempttranscriptrelation in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1766	lemma	`lemma sign_internal_transcript_relation md sk m ctx coins :`	Checked theorem signinternaltranscriptrelation in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1779	lemma	`lemma sign_internal_record_relation md sk m ctx coins :`	Checked theorem signinternalrecordrelation in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1792	lemma	`lemma signing_record_relation_sound md pk r :`	Checked theorem signingrecordrelationsound in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1809	lemma	`lemma signing_record_log_sound_cons md pk r rs :`	Checked theorem signingrecordlogsoundcons in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1817	lemma	`lemma signing_record_log_sound_transcripts_valid md pk rs :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Rejection.ec: 1830	lemma	`lemma signing_record_queries_cons r rs :`	Checked theorem signingrecordqueriescons in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1835	lemma	`lemma signing_record_transcripts_cons r rs :`	Checked theorem signingrecordtranscriptscons in the rejection-sampling predicates and distributional loss bounds; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Rejection.ec: 1840	lemma	`lemma rejection_bound_parts_nonnegative :`	Probability bound Pr[rejectionboundpartsnonnegative] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1845	lemma	`lemma rejection_sampling_bound_obligation_holds :`	Probability bound Pr[rejectionsamplingboundobligationholds] $\leq B$ or loss inequality controlling a bad event in the rejection-sampling predicates and distributional loss bounds.
HAETAE_Rejection.ec: 1856	lemma	`lemma signing_attempt_state_of_sample_pair_sampled_yE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1868	lemma	`lemma signing_attempt_state_of_sample_pair_highbitsE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1877	lemma	`lemma signing_attempt_state_of_sample_pair_lowbitsE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1886	lemma	`lemma signing_attempt_state_of_sample_pair_challengeE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1895	lemma	`lemma signing_attempt_state_of_sample_pair_responseE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1904	lemma	`lemma signing_attempt_state_of_sample_pair_response_auxE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1914	lemma	`lemma signing_attempt_state_of_sample_pair_hintE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1923	lemma	`lemma signing_attempt_state_of_sample_pair_lowbits_carrierE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1933	lemma	`lemma signing_attempt_state_of_sample_pair_tokenE md sk m ctx coins sample :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1945	lemma	`lemma signing_attempt_state_of_sample_pair_programmed_lowbitsE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1957	lemma	`lemma signing_attempt_state_of_sample_pair_lowbits_consistent`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1967	lemma	`lemma signing_attempt_state_of_sample_pair_programmed_challengeE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Rejection.ec: 1979	lemma	`lemma signing_attempt_state_of_sample_pair_challenge_consistent`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 1989	lemma	`lemma signing_attempt_state_of_sample_pair_reconstructed_highbitsE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 2001	lemma	`lemma signing_attempt_state_of_sample_pair_public_equation_consistent`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 2012	lemma	`lemma signing_attempt_state_of_sample_pair_signature_of_fieldsE`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 2026	lemma	`lemma signing_attempt_state_of_sample_pair_fields_match_signature`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 2037	lemma	`lemma signing_attempt_state_of_sample_pair_field_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec: 2052	lemma	`lemma signing_attempt_state_of_sample_pair_valid_signature_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Rejection.ec:2062	lemma	`lemma signing_attempt_state_of_sample_pair_accepts_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2075	lemma	`lemma signing_attempt_state_of_sample_pair_verification_accepts_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2086	lemma	`lemma signing_attempt_state_of_sample_pair_reference_rejection_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2102	lemma	`lemma signing_attempt_state_of_sample_pair_reference_rejection_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2112	lemma	`lemma signing_attempt_state_of_sample_pair_rejection_accepts_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2123	lemma	`lemma signing_attempt_state_of_sample_pair_rejection_no_abort_current`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2132	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2144	lemma	`lemma dexact_hyperball_signing_attempt_state_reference_rejection_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2156	lemma	`lemma dexact_hyperball_signing_attempt_state_rejection_accepts`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2169	lemma	`lemma dexact_hyperball_signing_attempt_state_rejection_no_abort`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Rejection.ec:2182	lemma	`lemma dexact_hyperball_signing_attempt_state_rejection_abort_zero`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:9	type	`type byte = int.`	Carrier set byte used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:10	type	`type seed = byte list.`	Carrier set seed used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:11	type	`type crh = byte list.`	Carrier set crh used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:12	type	`type random_coins = byte list.`	Carrier set randomcoins used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:13	type	`type xof256_state = byte list.`	Carrier set xof256state used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:14	type	`type message = byte list.`	Carrier set $\mathcal{M}$ of HAETAE messages; the EasyCrypt type message is read as a mathematical message object.
HAETAE_Algebra.ec:15	type	`type context = byte list.`	Carrier set context used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:16	type	`type coeff = int.`	Carrier set coeff used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:17	type	`type poly = coeff list.`	Carrier set poly used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:18	type	`type challenge = poly.`	Carrier set challenge used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:19	type	`type polyveck = poly list.`	Carrier set polyveck used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:20	type	`type polyvecl = poly list.`	Carrier set polyvecl used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:21	type	`type polyvecm = poly list.`	Carrier set polyvecm used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:22	type	`type matrix = poly list list.`	Carrier set matrix used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:23	type	`type hint_t = polyveck.`	Carrier set hintt used in the module/ring algebra, norms, and deterministic arithmetic relations; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Algebra.ec:24	type	`type sig_token = polyvecl * polyveck * hint_t.`	Signature space Σ; values of sig_token denote candidate HAETAE signatures.
HAETAE_Algebra.ec:26	type	`type pkey = seed * polyveck.`	Public-key space $\mathcal{PK}$; values of pkey denote HAETAE verification keys.
HAETAE_Algebra.ec:27	type	`type encoded_pkey = byte list.`	Public-key space $\mathcal{PK}$; values of encoded_pkey denote HAETAE verification keys.
HAETAE_Algebra.ec:28	type	`type skey = seed * int.`	Secret-key space $\mathcal{SK}$; values of skey denote HAETAE signing keys.
HAETAE_Algebra.ec:29	type	`type signature = polyveck * poly * crh * challenge * sig_token * int * int.`	Signature space Σ; values of signature denote candidate HAETAE signatures.
HAETAE_Algebra.ec:31	op	`op secret_key_of_seed (_ : mode) (sd : seed) : skey = (sd, 0).`	Probability law or sampler secretkeyofseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:32	op	`op poly_zero : poly = nseq n 0.`	Deterministic algebraic map or predicate polyzero over HAETAE module/ring objects.
HAETAE_Algebra.ec:33	op	`op polyveck_zero (md : mode) : polyveck = nseq (mode_k md) poly_zero.`	Deterministic algebraic map or predicate polyveckzero over HAETAE module/ring objects.
HAETAE_Algebra.ec:34	op	`op polyvecl_zero (md : mode) : polyvecl = nseq (mode_l md) poly_zero.`	Deterministic algebraic map or predicate polyveclzero over HAETAE module/ring objects.
HAETAE_Algebra.ec:35	op	`op polyvecm_zero (md : mode) : polyvecm = nseq (mode_m md) poly_zero.`	Deterministic algebraic map or predicate polyvecmzero over HAETAE module/ring objects.
HAETAE_Algebra.ec:36	op	`op matrix_zero (md : mode) : matrix =`	Deterministic algebraic map or predicate matrixzero over HAETAE module/ring objects.
HAETAE_Algebra.ec:39	lemma	`lemma poly_zero_size :`	Checked theorem polyzerosize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:43	lemma	`lemma polyveck_zero_size md :`	Checked theorem polyveckzerosize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:47	lemma	`lemma polyvecl_zero_size md :`	Checked theorem polyveclzerosize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:51	lemma	`lemma polyvecm_zero_size md :`	Checked theorem polyvecmzerosize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:55	op	`op poly_wf (p : poly) : bool = size p = n.`	Deterministic algebraic map or predicate polywf over HAETAE module/ring objects.
HAETAE_Algebra.ec:56	op	`op polyveck_wf (md : mode) (xs : polyveck) : bool =`	Deterministic algebraic map or predicate polyveckwf over HAETAE module/ring objects.
HAETAE_Algebra.ec:58	op	`op polyvecl_wf (md : mode) (xs : polyvecl) : bool =`	Deterministic algebraic map or predicate polyveclwf over HAETAE module/ring objects.
HAETAE_Algebra.ec:61	lemma	`lemma polyveck_wf_nth md b row :`	Checked theorem polyveckwfnth in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:72	op	`op crh_wf (mu : crh) : bool = size mu = crhbytes.`	Total mathematical function or predicate crhwf in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:73	op	`op challenge_coeff_ok (x : coeff) : bool = x = -1 \/ x = 0 \/ x = 1.`	Probability law or sampler challengecoeffok; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:74	op	`op challenge_wf (ch : challenge) : bool =`	Probability law or sampler challengewf; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:76	op	`op unit_coeff_ok (x : coeff) : bool = x = -1 \/ x = 1.`	Deterministic algebraic map or predicate unitcoeffok over HAETAE module/ring objects.
HAETAE_Algebra.ec:77	op	`op challenge_sparse_at (md : mode) (i : int) (x : coeff) : bool =`	Probability law or sampler challengesparseat; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:79	op	`op challenge_sparse (md : mode) (ch : challenge) : bool =`	Probability law or sampler challengesparse; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:83	op	`op poly_unit (p : poly) : bool = poly_wf p /\ all unit_coeff_ok p.`	Deterministic algebraic map or predicate polyunit over HAETAE module/ring objects.
HAETAE_Algebra.ec:84	op	`op polyveck_unit (md : mode) (xs : polyveck) : bool =`	Deterministic algebraic map or predicate polyveckunit over HAETAE module/ring objects.
HAETAE_Algebra.ec:86	op	`op polyvecl_unit (md : mode) (xs : polyvecl) : bool =`	Deterministic algebraic map or predicate polyveclunit over HAETAE module/ring objects.
HAETAE_Algebra.ec:89	lemma	`lemma poly_zero_wf : poly_wf poly_zero.`	Checked theorem polyzerowf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:92	lemma	`lemma polyveck_zero_wf md : polyveck_wf md (polyveck_zero md).`	Checked theorem polyveckzerowf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:100	lemma	`lemma polyvecl_zero_wf md : polyvecl_wf md (polyvecl_zero md).`	Checked theorem polyveclzerowf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:108	op	`op coeff_mod (x : coeff) : coeff = x %% q.`	Deterministic algebraic map or predicate coeffmod over HAETAE module/ring objects.
HAETAE_Algebra.ec:109	op	`op coeff_add (x y : coeff) : coeff = coeff_mod (x + y).`	Deterministic algebraic map or predicate coeffadd over HAETAE module/ring objects.
HAETAE_Algebra.ec:110	op	`op coeff_neg (x : coeff) : coeff = coeff_mod (-x).`	Deterministic algebraic map or predicate coeffneg over HAETAE module/ring objects.
HAETAE_Algebra.ec:111	op	`op coeff_sub (x y : coeff) : coeff = coeff_add x (coeff_neg y).`	Deterministic algebraic map or predicate coeffsub over HAETAE module/ring objects.
HAETAE_Algebra.ec:112	op	`op coeff_mul (x y : coeff) : coeff = coeff_mod (x * y).`	Deterministic algebraic map or predicate coeffmul over HAETAE module/ring objects.
HAETAE_Algebra.ec:113	op	`op coeff_double (x : coeff) : coeff = coeff_add x x.`	Deterministic algebraic map or predicate coeffdouble over HAETAE module/ring objects.
HAETAE_Algebra.ec:115	op	`op poly_coeff (p : poly) (i : int) : coeff = nth 0 p i.`	Deterministic algebraic map or predicate polycoeff over HAETAE module/ring objects.
HAETAE_Algebra.ec:116	op	`op poly_normalize (p : poly) : poly =`	Deterministic algebraic map or predicate polynormalize over HAETAE module/ring objects.
HAETAE_Algebra.ec:118	op	`op poly_add (p r : poly) : poly =`	Deterministic algebraic map or predicate polyadd over HAETAE module/ring objects.
HAETAE_Algebra.ec:122	op	`op poly_neg (p : poly) : poly =`	Deterministic algebraic map or predicate polyneg over HAETAE module/ring objects.
HAETAE_Algebra.ec:125	op	`op poly_sub (p r : poly) : poly = poly_add p (poly_neg r).`	Deterministic algebraic map or predicate polysub over HAETAE module/ring objects.
HAETAE_Algebra.ec:126	op	`op poly_double (p : poly) : poly =`	Deterministic algebraic map or predicate polydouble over HAETAE module/ring objects.
HAETAE_Algebra.ec:130	op	`op poly_mul_term (p r : poly) (i j : int) : coeff =`	Deterministic algebraic map or predicate polymulterm over HAETAE module/ring objects.
HAETAE_Algebra.ec:133	op	`op poly_mul (p r : poly) : poly =`	Deterministic algebraic map or predicate polymul over HAETAE module/ring objects.
HAETAE_Algebra.ec:140	op	`op poly_dot (row : poly list) (v : poly list) : poly =`	Deterministic algebraic map or predicate polydot over HAETAE module/ring objects.
HAETAE_Algebra.ec:147	op	`op matrix_vec_mul (md : mode) (a : matrix) (v : polyvecl) : polyveck =`	Deterministic algebraic map or predicate matrixvecmul over HAETAE module/ring objects.
HAETAE_Algebra.ec:151	op	`op polyveck_add (md : mode) (xs ys : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckadd over HAETAE module/ring objects.
HAETAE_Algebra.ec:156	op	`op polyveck_neg (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckneg over HAETAE module/ring objects.
HAETAE_Algebra.ec:159	op	`op polyveck_sub (md : mode) (xs ys : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyvecksub over HAETAE module/ring objects.
HAETAE_Algebra.ec:161	op	`op polyveck_double (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckdouble over HAETAE module/ring objects.
HAETAE_Algebra.ec:165	op	`op polyvecl_sub (md : mode) (xs ys : polyvecl) : polyvecl =`	Deterministic algebraic map or predicate polyveclsub over HAETAE module/ring objects.
HAETAE_Algebra.ec:170	lemma	`lemma polyvecl_sub_size md xs ys :`	Checked theorem polyveclsubsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:174	lemma	`lemma poly_add_wf p r : poly_wf (poly_add p r).`	Checked theorem polyaddwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:182	lemma	`lemma poly_normalize_wf p : poly_wf (poly_normalize p).`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:185	lemma	`lemma poly_neg_wf p : poly_wf (poly_neg p).`	Checked theorem polynegwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:193	lemma	`lemma poly_sub_wf p r : poly_wf (poly_sub p r).`	Checked theorem polysubwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:196	lemma	`lemma poly_double_wf p : poly_wf (poly_double p).`	Checked theorem polydoublewf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:204	lemma	`lemma poly_mul_wf p r : poly_wf (poly_mul p r).`	Checked theorem polymulwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:212	lemma	`lemma poly_dot_wf row v : poly_wf (poly_dot row v).`	Checked theorem polydotwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:231	lemma	`lemma matrix_vec_mul_wf md a v : polyveck_wf md (matrix_vec_mul md a v).`	Checked theorem matrixvecmulwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:243	lemma	`lemma polyveck_add_wf md xs ys : polyveck_wf md (polyveck_add md xs ys).`	Checked theorem polyveckaddwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:255	lemma	`lemma polyveck_neg_wf md xs : polyveck_wf md (polyveck_neg md xs).`	Checked theorem polyvecknegwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:267	lemma	`lemma polyveck_sub_wf md xs ys : polyveck_wf md (polyveck_sub md xs ys).`	Checked theorem polyvecksubwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:270	lemma	`lemma polyveck_double_wf md xs : polyveck_wf md (polyveck_double md xs).`	Checked theorem polyveckdoublewf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:282	lemma	`lemma polyvecl_sub_wf md xs ys : polyvecl_wf md (polyvecl_sub md xs ys).`	Checked theorem polyveclsubwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:291	op	`op coeff_q_bound (x : coeff) : bool = 0 <= x < q.`	Numerical bound or budget parameter coeffqbound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:292	op	`op poly_coeffs_q_bound (p : poly) : bool =`	Numerical bound or budget parameter polycoeffsqbound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:294	op	`op polyveck_coeffs_q_bound (md : mode) (xs : polyveck) : bool =`	Numerical bound or budget parameter polyveckcoeffsqbound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:298	lemma	`lemma coeff_mod_q_bound x : coeff_q_bound (coeff_mod x).`	Probability bound Pr[coeffmodqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:301	lemma	`lemma poly_zero_coeffs_q_bound : poly_coeffs_q_bound poly_zero.`	Probability bound Pr[polyzerocoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:307	lemma	`lemma poly_add_coeffs_q_bound p r :`	Probability bound Pr[polyaddcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:317	lemma	`lemma poly_neg_coeffs_q_bound p :`	Probability bound Pr[polynegcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:327	lemma	`lemma poly_sub_coeffs_q_bound p r :`	Probability bound Pr[polysubcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:331	lemma	`lemma poly_double_coeffs_q_bound p :`	Probability bound Pr[polydoublecoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:341	lemma	`lemma poly_mul_coeffs_q_bound p r :`	Probability bound Pr[polymulcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:351	lemma	`lemma poly_dot_coeffs_q_bound row v :`	Probability bound Pr[polydotcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:371	lemma	`lemma matrix_vec_mul_coeffs_q_bound md a v :`	Probability bound Pr[matrixvecmulcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:384	lemma	`lemma polyveck_zero_coeffs_q_bound md :`	Probability bound Pr[polyveckzerocoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:393	lemma	`lemma polyveck_add_coeffs_q_bound md xs ys :`	Probability bound Pr[polyveckaddcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:403	lemma	`lemma polyveck_neg_coeffs_q_bound md xs :`	Probability bound Pr[polyvecknegcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:413	lemma	`lemma polyveck_sub_coeffs_q_bound md xs ys :`	Probability bound Pr[polyvecksubcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:417	lemma	`lemma polyveck_double_coeffs_q_bound md xs :`	Probability bound Pr[polyveckdoublecoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:427	op	`op z1_alpha : int = 256.`	Total mathematical function or predicate z1alpha in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:429	op	`op coeff_decompose_z1_low (x : coeff) : coeff =`	Deterministic algebraic map or predicate coeffdecomposez1low over HAETAE module/ring objects.
HAETAE_Algebra.ec:432	op	`op coeff_decompose_z1_high (x : coeff) : coeff =`	Deterministic algebraic map or predicate coeffdecomposez1high over HAETAE module/ring objects.
HAETAE_Algebra.ec:434	op	`op coeff_lsb (x : coeff) : coeff = x %% 2.`	Deterministic algebraic map or predicate coefflsb over HAETAE module/ring objects.
HAETAE_Algebra.ec:435	op	`op coeff_decompose_vk_low (x : coeff) : coeff =`	Deterministic algebraic map or predicate coeffdecomposevklow over HAETAE module/ring objects.
HAETAE_Algebra.ec:439	op	`op coeff_decompose_vk_high (x : coeff) : coeff =`	Deterministic algebraic map or predicate coeffdecomposevkhigh over HAETAE module/ring objects.
HAETAE_Algebra.ec:441	op	`op coeff_decompose_hint_high (md : mode) (x : coeff) : coeff =`	Deterministic algebraic map or predicate coeffdecomposehinthigh over HAETAE module/ring objects.
HAETAE_Algebra.ec:446	op	`op coeff_compose_z1 (hi lo : coeff) : coeff = hi * z1_alpha + lo.`	Deterministic algebraic map or predicate coeffcomposez1 over HAETAE module/ring objects.
HAETAE_Algebra.ec:448	op	`op poly_highbits (p : poly) : poly =`	Deterministic algebraic map or predicate polyhighbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:450	op	`op poly_lowbits (p : poly) : poly =`	Deterministic algebraic map or predicate polylowbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:452	op	`op poly_lsb (p : poly) : poly =`	Deterministic algebraic map or predicate polylsb over HAETAE module/ring objects.
HAETAE_Algebra.ec:454	op	`op poly_vk_lowbits (p : poly) : poly =`	Deterministic algebraic map or predicate polyvklowbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:456	op	`op poly_vk_highbits (p : poly) : poly =`	Deterministic algebraic map or predicate polyvkhighbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:458	op	`op poly_compose_z1 (hi lo : poly) : poly =`	Deterministic algebraic map or predicate polycomposez1 over HAETAE module/ring objects.
HAETAE_Algebra.ec:462	op	`op poly_hint_highbits (md : mode) (p : poly) : poly =`	Deterministic algebraic map or predicate polyhinthighbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:465	op	`op polyvecl_highbits (md : mode) (xs : polyvecl) : polyvecl =`	Deterministic algebraic map or predicate polyveclhighbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:467	op	`op polyvecl_lowbits (md : mode) (xs : polyvecl) : polyvecl =`	Deterministic algebraic map or predicate polyvecllowbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:469	op	`op polyveck_highbits_hint (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckhighbitshint over HAETAE module/ring objects.
HAETAE_Algebra.ec:471	op	`op polyveck_vk_lowbits (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckvklowbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:473	op	`op polyveck_vk_highbits (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckvkhighbits over HAETAE module/ring objects.
HAETAE_Algebra.ec:475	op	`op polyveck_mul_alpha (md : mode) (xs : polyveck) : polyveck =`	Deterministic algebraic map or predicate polyveckmulalpha over HAETAE module/ring objects.
HAETAE_Algebra.ec:483	lemma	`lemma poly_highbits_wf p : poly_wf (poly_highbits p).`	Checked theorem polyhighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:486	lemma	`lemma poly_lowbits_wf p : poly_wf (poly_lowbits p).`	Checked theorem polylowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:489	lemma	`lemma poly_lsb_wf p : poly_wf (poly_lsb p).`	Checked theorem polylsbwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:492	lemma	`lemma poly_vk_lowbits_wf p : poly_wf (poly_vk_lowbits p).`	Checked theorem polyvklowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:495	lemma	`lemma poly_vk_highbits_wf p : poly_wf (poly_vk_highbits p).`	Checked theorem polyvkhighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:498	lemma	`lemma poly_compose_z1_wf hi lo : poly_wf (poly_compose_z1 hi lo).`	Checked theorem polycomposez1wf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:501	lemma	`lemma poly_hint_highbits_wf md p : poly_wf (poly_hint_highbits md p).`	Checked theorem polyhinthighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:504	lemma	`lemma polyvecl_highbits_wf md xs :`	Checked theorem polyveclhighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:514	lemma	`lemma polyvecl_lowbits_wf md xs :`	Checked theorem polyvecllowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:524	lemma	`lemma polyveck_highbits_hint_wf md xs :`	Checked theorem polyveckhighbitshintwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:534	lemma	`lemma polyveck_vk_lowbits_wf md xs :`	Checked theorem polyveckvklowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:544	lemma	`lemma polyveck_vk_highbits_wf md xs :`	Checked theorem polyveckvkhighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:554	lemma	`lemma polyveck_mul_alpha_wf md xs :`	Checked theorem polyveckmulalphawf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:564	op	`op public_key_seed_poly (tag : int) (src : byte list) : poly =`	Probability law or sampler publickeyseedpoly; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:566	op	`op public_key_seed_polyvecl`	Probability law or sampler publickeyseedpolyvecl; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:571	op	`op public_key_seed_polyveck`	Probability law or sampler publickeyseedpolyveck; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:576	op	`op haetae_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.`	Probability law or sampler haetaekeygenseedbufbytes; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:577	op	`op haetae_reference_keygen_seedbuf_bytes : int =`	Probability law or sampler haetaereferencekeygenseedbufbytes; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:579	op	`op haetae_reference_keygen_xof_absorb_bytes : int = seedbytes.`	Total mathematical function or predicate haetaereferencekeygenxofabsorbbytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:580	op	`op haetae_reference_keygen_xof_squeeze_bytes : int =`	Total mathematical function or predicate haetaereferencekeygenxofsqueezebytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:582	op	`op haetae_keygen_rhoprime_offset : int = 0.`	Total mathematical function or predicate haetaekeygenrhoprimeoffset in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:583	op	`op haetae_keygen_sigma_offset : int = seedbytes.`	Total mathematical function or predicate haetaekeygensigmaoffset in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:584	op	`op haetae_keygen_key_offset : int = seedbytes + crhbytes.`	Total mathematical function or predicate haetaekeygenkeyoffset in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:585	op	`op haetae_shake256_rate_bytes : int = shake256_rate_bytes.`	Oracle-related function haetaeshake256ratebytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:586	op	`op haetae_shake256_domain_separator : byte = shake256_domain_separator.`	Oracle-related function haetaeshake256domainseparator used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:587	op	`op haetae_shake256_bytes (input : byte list) (outlen : int) : byte list =`	Oracle-related function haetaeshake256bytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:589	op	`op haetae_seed_slice (src : byte list) (offset len : int) : byte list =`	Probability law or sampler haetaeseedslice; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:591	op	`op haetae_xof256_absorb_input`	Total mathematical function or predicate haetaexof256absorbinput in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:594	op	`op haetae_xof256_absorb_once`	Total mathematical function or predicate haetaexof256absorbonce in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:597	op	`op haetae_xof256_squeeze`	Total mathematical function or predicate haetaexof256squeeze in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:600	op	`op haetae_xof256_absorb_once_squeeze`	Total mathematical function or predicate haetaexof256absorboncesqueeze in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:605	op	`op haetae_reference_keygen_seedbuf_after_memcpy (sd : seed) : byte list =`	Probability law or sampler haetaereferencekeygenseedbufaftermemcpy; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:609	op	`op haetae_reference_keygen_xof_input (sd : seed) : byte list =`	Total mathematical function or predicate haetaereferencekeygenxofinput in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:613	op	`op haetae_keygen_xof_seedbuf (sd : seed) : byte list =`	Probability law or sampler haetaekeygenxofseedbuf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:618	op	`op haetae_reference_keygen_xof256_seedbuf (sd : seed) : byte list =`	Probability law or sampler haetaereferencekeygenxof256seedbuf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:620	op	`op haetae_keygen_seedbuf_rhoprime (seedbuf : byte list) : seed =`	Probability law or sampler haetaekeygenseedbufrhoprime; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:622	op	`op haetae_keygen_seedbuf_sigma (seedbuf : byte list) : crh =`	Probability law or sampler haetaekeygenseedbufsigma; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:624	op	`op haetae_keygen_seedbuf_key (seedbuf : byte list) : seed =`	Probability law or sampler haetaekeygenseedbufkey; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:626	op	`op haetae_keygen_seedbuf_layout_wf (seedbuf : byte list) : bool =`	Probability law or sampler haetaekeygenseedbuflayoutwf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:634	op	`op haetae_reference_keygen_xof_call_wf`	Total mathematical function or predicate haetaereferencekeygenxofcallwf in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:663	op	`op haetae_keygen_rhoprime (sd : seed) : seed =`	Total mathematical function or predicate haetaekeygenrhoprime in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:666	op	`op haetae_keygen_sigma (sd : seed) : crh =`	Total mathematical function or predicate haetaekeygensigma in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:669	op	`op haetae_keygen_key (sd : seed) : seed =`	Total mathematical function or predicate haetaekeygenkey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:672	op	`op haetae_keygen_counter_next (md : mode) (counter : int) : int =`	Total mathematical function or predicate haetaekeygencounternext in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:674	op	`op haetae_keygen_first_accept_counter (_ : mode) (_ : seed) : int = 0.`	Predicate/event haetaekeygenfirstacceptcounter on the game state; in probability expressions it denotes the indicator event $1_{\text{haetaekeygenfirstacceptcounter}}$. *Context:* module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:675	op	`op haetae_keygen_sampler_seed (sigma : crh) (counter : int) : seed =`	Probability law or sampler haetaekeygensamplerseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:680	op	`op public_matrix_seed (md : mode) (sd : seed) : matrix =`	Probability law or sampler publicmatrixseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:684	op	`op public_secret_vector_seed (md : mode) (sd : seed) : polyvecl =`	Probability law or sampler publicsecretvectorseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:686	op	`op public_error_vector_seed (md : mode) (sd : seed) : polyveck =`	Probability law or sampler publicerrorvectorseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:688	op	`op public_rounding_vector_seed (md : mode) (sd : seed) : polyveck =`	Probability law or sampler publicroundingvectorseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:691	op	`op haetae_keygen_a0_matrix (md : mode) (sd : seed) : matrix =`	Deterministic algebraic map or predicate haetaekeygena0matrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:693	op	`op haetae_keygen_secret_vector (md : mode) (sd : seed) : polyvecl =`	Deterministic algebraic map or predicate haetaekeygensecretvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:695	op	`op haetae_keygen_error_vector (md : mode) (sd : seed) : polyveck =`	Deterministic algebraic map or predicate haetaekeygenerrorvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:697	op	`op haetae_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck =`	Deterministic algebraic map or predicate haetaekeygenrawpublicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:703	op	`op haetae_keygen_mode23_predecompose_vector`	Deterministic algebraic map or predicate haetaekeygenmode23predecomposevector over HAETAE module/ring objects.
HAETAE_Algebra.ec:708	op	`op haetae_keygen_mode23_public_vector`	Deterministic algebraic map or predicate haetaekeygenmode23publicvector over HAETAE module/ring objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:711	op	`op haetae_keygen_mode23_rounding_lowbits`	Total mathematical function or predicate haetaekeygenmode23roundinglowbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:714	op	`op haetae_keygen_mode23_adjusted_error_vector`	Deterministic algebraic map or predicate haetaekeygenmode23adjustederrorvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:719	op	`op haetae_keygen_mode5_public_vector`	Deterministic algebraic map or predicate haetaekeygenmode5publicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:722	op	`op haetae_public_key_vector_from_relation`	Oracle-related function haetaepublickeyvectorfromrelation used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:744	op	`op haetae_keygen_public_vector (md : mode) (sd : seed) : polyveck =`	Deterministic algebraic map or predicate haetaekeygenpublicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:748	op	`op public_key_vector_seed (md : mode) (sd : seed) : polyveck =`	Probability law or sampler publickeyvectorseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:750	op	`op public_key_relation`	Total mathematical function or predicate publickeyrelation in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:753	op	`op public_key_equation_holds (md : mode) (pk : pkey) : bool =`	Total mathematical function or predicate publickeyequationholds in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:757	op	`op haetae_reference_polymatkm_expand_matA`	Oracle-related function haetaereferencepolymatkmexpandmatA used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:760	op	`op haetae_reference_polyveck_expand_vecA`	Oracle-related function haetaereferencepolyveckexpandvecA used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:763	op	`op haetae_reference_polyvecmk_expand_S_s1`	Oracle-related function haetaereferencepolyvecmkexpandSs1 used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:766	op	`op haetae_reference_polyvecmk_expand_S_s2`	Oracle-related function haetaereferencepolyvecmkexpandSs2 used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:769	op	`op haetae_reference_keygen_secret_vector`	Deterministic algebraic map or predicate haetaereferencekeygensecretvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:774	op	`op haetae_reference_keygen_error_vector`	Deterministic algebraic map or predicate haetaereferencekeygenerrorvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:779	op	`op haetae_reference_keygen_a0_matrix`	Deterministic algebraic map or predicate haetaereferencekeygena0matrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:782	op	`op haetae_reference_keygen_raw_public_vector`	Deterministic algebraic map or predicate haetaereferencekeygenrawpublicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:789	op	`op haetae_reference_keygen_mode23_predecompose_vector`	Deterministic algebraic map or predicate haetaereferencekeygenmode23predecomposevector over HAETAE module/ring objects.
HAETAE_Algebra.ec:794	op	`op haetae_reference_keygen_mode23_public_vector`	Deterministic algebraic map or predicate haetaereferencekeygenmode23publicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:798	op	`op haetae_reference_keygen_mode23_rounding_lowbits`	Total mathematical function or predicate haetaereferencekeygenmode23roundinglowbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:802	op	`op haetae_reference_keygen_mode23_adjusted_error_vector`	Deterministic algebraic map or predicate haetaereferencekeygenmode23adjustederrorvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:807	op	`op haetae_reference_keygen_mode5_public_vector`	Deterministic algebraic map or predicate haetaereferencekeygenmode5publicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:811	op	`op haetae_reference_keygen_public_vector`	Deterministic algebraic map or predicate haetaereferencekeygenpublicvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:817	op	`op packed_haetae_reference_public_key`	Total mathematical function or predicate packedhaetaereferencepublickey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:820	op	`op haetae_reference_keygen_seed_flow_wf (md : mode) (sd : seed) : bool =`	Probability law or sampler haetaereferencekeygenseedflowwf; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:841	op	`op haetae_mode23_qj_vector (md : mode) (sd : seed) : polyveck =`	Deterministic algebraic map or predicate haetaemode23qjvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:843	op	`op haetae_mode23_verification_column`	Total mathematical function or predicate haetaemode23verificationcolumn in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:846	op	`op haetae_mode5_verification_column`	Total mathematical function or predicate haetaemode5verificationcolumn in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:849	op	`op public_key_mode23_column (md : mode) (pk : pkey) : polyveck =`	Total mathematical function or predicate publickeymode23column in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:852	op	`op public_key_mode5_column (md : mode) (pk : pkey) : polyveck =`	Total mathematical function or predicate publickeymode5column in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:854	op	`op public_key_verification_column (md : mode) (pk : pkey) : polyveck =`	Total mathematical function or predicate publickeyverificationcolumn in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:858	op	`op haetae_verification_column_from_unpacked`	Oracle-related function haetaeverificationcolumnfromunpacked used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:868	op	`op haetae_verification_matrix_from_unpacked`	Oracle-related function haetaeverificationmatrixfromunpacked used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:883	op	`op packed_haetae_public_key (md : mode) (sd : seed) : pkey =`	Total mathematical function or predicate packedhaetaepublickey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:885	op	`op packed_haetae_verification_matrix (md : mode) (pk : pkey) : matrix =`	Deterministic algebraic map or predicate packedhaetaeverificationmatrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:887	op	`op packed_haetae_keygen_verification_matrix (md : mode) (sd : seed) :`	Deterministic algebraic map or predicate packedhaetaekeygenverificationmatrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:890	op	`op public_verification_matrix (md : mode) (pk : pkey) : matrix =`	Deterministic algebraic map or predicate publicverificationmatrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:892	op	`op challenge_embedding (md : mode) (ch : challenge) : polyvecl =`	Probability law or sampler challengeembedding; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:894	op	`op public_key_challenge_term`	Probability law or sampler publickeychallengeterm; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:898	op	`op reconstructed_highbits`	Total mathematical function or predicate reconstructedhighbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:902	op	`op public_key_of_secret (md : mode) (sk : skey) : pkey =`	Total mathematical function or predicate publickeyofsecret in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:904	op	`op haetae_public_key_body_bytes (md : mode) : int =`	Total mathematical function or predicate haetaepublickeybodybytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:906	op	`op haetae_public_key_bytes (md : mode) : int =`	Total mathematical function or predicate haetaepublickeybytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:908	op	`op haetae_public_key_encoding_wf (md : mode) (enc : encoded_pkey) : bool =`	Total mathematical function or predicate haetaepublickeyencodingwf in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:910	op	`op haetae_public_key_unpacked_wf (md : mode) (pk : pkey) : bool =`	Total mathematical function or predicate haetaepublickeyunpackedwf in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:912	op	`op haetae_pack_seed (sd : seed) : byte list =`	Probability law or sampler haetaepackseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:914	op	`op haetae_unpack_seed (enc : encoded_pkey) : seed =`	Probability law or sampler haetaeunpackseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:916	op	`op haetae_pack_poly_q (md : mode) (p : poly) : byte list =`	Deterministic algebraic map or predicate haetaepackpolyq over HAETAE module/ring objects.
HAETAE_Algebra.ec:920	op	`op haetae_unpack_poly_q_at`	Numerical bound or budget parameter haetaeunpackpolyqat used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:924	op	`op haetae_byte_modulus : int = 256.`	Total mathematical function or predicate haetaebytemodulus in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:925	op	`op haetae_polyq_mode23_bound : int = 32768.`	Numerical bound or budget parameter haetaepolyqmode23bound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:926	op	`op haetae_polyq_mode5_bound : int = 65536.`	Numerical bound or budget parameter haetaepolyqmode5bound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:928	op	`op haetae_byte_norm (x : int) : byte = x %% haetae_byte_modulus.`	Deterministic algebraic map or predicate haetaebytenorm over HAETAE module/ring objects.
HAETAE_Algebra.ec:930	op	`op haetae_polyq_coeff_bound (md : mode) : int =`	Numerical bound or budget parameter haetaepolyqcoeffbound used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:935	op	`op haetae_polyq_coeffs_wf (md : mode) (p : poly) : bool =`	Numerical bound or budget parameter haetaepolyqcoeffswf used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:940	op	`op haetae_polyveck_polyq_coeffs_wf`	Numerical bound or budget parameter haetaepolyveckpolyqcoeffswf used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:946	lemma	`lemma coeff_decompose_hint_high_polyq_bound md x :`	Probability bound $\Pr[\text{coeffdecomposehinthighpolyqbound}] \leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:956	lemma	`lemma poly_hint_highbits_polyq_wf md p :`	Checked theorem polyhinthighbitspolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:970	lemma	`lemma polyveck_highbits_hint_polyq_wf md xs :`	Checked theorem polyveckhighbitshintpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:984	lemma	`lemma coeff_decompose_vk_high_mode23_polyq_bound md x :`	Probability bound $\Pr[\text{coeffdecomposevkhighmode23polyqbound}] \leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:995	lemma	`lemma poly_vk_highbits_mode23_polyq_wf md p :`	Checked theorem polyvkhighbitsmode23polyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1010	lemma	`lemma polyveck_vk_highbits_mode23_polyq_wf md xs :`	Checked theorem polyveckvkhighbitsmode23polyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1025	lemma	`lemma coeff_q_bound_mode5_polyq_bound x :`	Probability bound $\Pr[\text{coeffqboundmode5polyqbound}] \leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1033	lemma	`lemma poly_q_bound_mode5_polyq_wf p :`	Probability bound $\Pr[\text{polyqboundmode5polyqwf}] \leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1045	lemma	`lemma polyveck_q_bound_mode5_polyq_wf xs :`	Probability bound $\Pr[\text{polyveckqboundmode5polyqwf}] \leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1059	op	`op haetae_pack_poly_q_mode23_byte`	Numerical bound or budget parameter haetaepackpolyqmode23byte used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1092	op	`op haetae_pack_poly_q_mode5_byte (p : poly) (j : int) : byte =`	Numerical bound or budget parameter haetaepackpolyqmode5byte used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1097	op	`op haetae_pack_poly_q_ref (md : mode) (p : poly) : byte list =`	Numerical bound or budget parameter haetaepackpolyqref used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1111	op	`op haetae_unpack_poly_q_mode23_coeff_at`	Numerical bound or budget parameter haetaeunpackpolyqmode23coeffat used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1143	op	`op haetae_unpack_poly_q_mode5_coeff_at`	Numerical bound or budget parameter haetaeunpackpolyqmode5coeffat used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1148	op	`op haetae_unpack_poly_q_ref_at`	Numerical bound or budget parameter haetaeunpackpolyqrefat used to upper-bound a probability loss in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1167	op	`op haetae_pack_polyveck_body_ref (md : mode) (b : polyveck) : byte list =`	Deterministic algebraic map or predicate haetaepackpolyveckbodyref over HAETAE module/ring objects.
HAETAE_Algebra.ec:1176	op	`op haetae_unpack_polyveck_body_ref`	Deterministic algebraic map or predicate haetaeunpackpolyveckbodyref over HAETAE module/ring objects.
HAETAE_Algebra.ec:1184	op	`op haetae_pack_vk_ref (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =`	Total mathematical function or predicate haetaepackvkref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1187	op	`op haetae_unpack_vk_public_key_ref`	Total mathematical function or predicate haetaeunpackvkpublickeyref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1191	op	`op haetae_pack_polyveck_body (md : mode) (b : polyveck) : byte list =`	Deterministic algebraic map or predicate haetaepackpolyveckbody over HAETAE module/ring objects.
HAETAE_Algebra.ec:1200	op	`op haetae_unpack_polyveck_body`	Deterministic algebraic map or predicate haetaeunpackpolyveckbody over HAETAE module/ring objects.
HAETAE_Algebra.ec:1207	op	`op haetae_pack_vk (md : mode) (b : polyveck) (sd : seed) : encoded_pkey =`	Total mathematical function or predicate haetaepackvk in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1209	op	`op haetae_unpack_vk_public_key`	Total mathematical function or predicate haetaeunpackvkpublickey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1213	op	`op haetae_unpack_vk_matrix (md : mode) (enc : encoded_pkey) : matrix =`	Deterministic algebraic map or predicate haetaeunpackvkmatrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:1216	op	`op haetae_pack_public_key_bytes (md : mode) (pk : pkey) : encoded_pkey =`	Total mathematical function or predicate haetaepackpublickeybytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1218	op	`op haetae_unpack_public_key_bytes`	Total mathematical function or predicate haetaeunpackpublickeybytes in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1221	op	`op concrete_haetae_keygen_packed_public_key`	Total mathematical function or predicate concretehaetaekeygenpackedpublickey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1224	op	`op concrete_haetae_keygen_unpacked_public_key`	Total mathematical function or predicate concretehaetaekeygenunpackedpublickey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1228	op	`op concrete_haetae_keygen_verification_matrix (md : mode) (sd : seed) :`	Deterministic algebraic map or predicate concretehaetaekeygenverificationmatrix over HAETAE module/ring objects.
HAETAE_Algebra.ec:1232	op	`op haetae_pack_public_key_bytes_ref (md : mode) (pk : pkey) :`	Total mathematical function or predicate haetaepackpublickeybytesref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1235	op	`op haetae_unpack_public_key_bytes_ref`	Total mathematical function or predicate haetaeunpackpublickeybytesref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1238	op	`op concrete_haetae_keygen_packed_public_key_ref`	Total mathematical function or predicate concretehaetaekeygenpackedpublickeyref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1241	op	`op concrete_haetae_keygen_unpacked_public_key_ref`	Total mathematical function or predicate concretehaetaekeygenunpackedpublickeyref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1245	op	`op concrete_haetae_keygen_verification_matrix_ref`	Deterministic algebraic map or predicate concretehaetaekeygenverificationmatrixref over HAETAE module/ring objects.
HAETAE_Algebra.ec:1249	op	`op concrete_haetae_reference_keygen_packed_public_key_ref`	Total mathematical function or predicate concretehaetaereferencekeygenpackedpublickeyref in the module/ring algebra, norms, and deterministic arithmetic relations.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:1253	op	`op concrete_haetae_reference_keygen_unpacked_public_key_ref`	Total mathematical function or predicate concretehaetaereferencekeygenunpackedpublickeyref in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1257	op	`op concrete_haetae_reference_keygen_verification_matrix_ref`	Deterministic algebraic map or predicate concretehaetaereferencekeygenverificationmatrixref over HAETAE module/ring objects.
HAETAE_Algebra.ec:1261	op	`op haetae_public_key_roundtrip_ok (md : mode) (pk : pkey) : bool =`	Total mathematical function or predicate haetaepublickeyroundtripok in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1264	op	`op haetae_keygen_public_key_roundtrip_ok (md : mode) (sd : seed) : bool =`	Total mathematical function or predicate haetaekeygenpublickeyroundtripok in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1267	lemma	`lemma public_key_seed_poly_wf tag src :`	Checked theorem publickeyseedpolywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1271	lemma	`lemma public_key_seed_polyvecl_wf md tag src :`	Checked theorem publickeyseedpolyveclwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1281	lemma	`lemma public_key_seed_polyveck_wf md tag src :`	Checked theorem publickeyseedpolyveckwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1291	lemma	`lemma haetae_keygen_seedbuf_bytes_gt0 :`	Checked theorem haetaekeygenseedbufbytesgt0 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1295	lemma	`lemma haetae_reference_keygen_seedbuf_bytesE :`	Checked theorem haetaereferencekeygenseedbufbytesE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1302	lemma	`lemma haetae_reference_keygen_xof_absorb_bytesE :`	Checked theorem haetaereferencekeygenxofabsorbbytesE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1306	lemma	`lemma haetae_reference_keygen_xof_squeeze_bytesE :`	Checked theorem haetaereferencekeygenxofsqueezebytesE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1311	lemma	`lemma haetae_keygen_rhoprime_offsetE :`	Checked theorem haetaekeygenrhoprimeoffsetE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1315	lemma	`lemma haetae_keygen_sigma_offsetE :`	Checked theorem haetaekeygensigmaoffsetE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1319	lemma	`lemma haetae_keygen_key_offsetE :`	Checked theorem haetaekeygenkeyoffsetE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1323	lemma	`lemma haetae_shake256_rate_bytesE :`	Checked theorem haetaeshake256ratebytesE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1329	lemma	`lemma haetae_shake256_domain_separatorE :`	Theorem haetaeshake256domainseparatorE contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Algebra.ec:1335	lemma	`lemma haetae_shake256_bytes_size input outlen :`	Checked theorem haetaeshake256bytessize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1343	lemma	`lemma haetae_xof256_squeeze_size st outlen :`	Checked theorem haetaexof256squeezesize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1352	lemma	`lemma haetae_xof256_absorb_once_squeeze_size input inlen outlen :`	Checked theorem haetaexof256absorboncesqueezesize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1361	lemma	`lemma haetae_reference_keygen_seedbuf_after_memcpy_size sd :`	Checked theorem haetaereferencekeygenseedbufaftermemcpysize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1368	lemma	`lemma haetae_keygen_xof_seedbuf_size sd :`	Checked theorem haetaekeygenxofseedbufsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1379	lemma	`lemma haetae_reference_keygen_xof256_seedbuf_size sd :`	Checked theorem haetaereferencekeygenxof256seedbufsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1388	lemma	`lemma haetae_seed_slice_size src offset len :`	Checked theorem haetaeseedslicesize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1397	lemma	`lemma haetae_reference_keygen_xof_input_size sd :`	Checked theorem haetaereferencekeygenxofinputsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1406	lemma	`lemma haetae_xof256_absorb_input_size input inlen :`	Checked theorem haetaexof256absorbinputsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1415	lemma	`lemma haetae_reference_keygen_xof_absorb_inputE sd :`	Checked theorem haetaereferencekeygenxofabsorbinputE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1425	lemma	`lemma haetae_reference_keygen_shake256_domain sd :`	Theorem haetaereferencekeygenshake256domain contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_Algebra.ec:1435	lemma	`lemma haetae_reference_keygen_xof_squeeze_bytes_lt_rate :`	Checked theorem haetaereferencekeygenxofsqueezebytesltrate in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1444	lemma	`lemma haetae_reference_keygen_xof_absorb_once_stateE sd :`	Checked theorem haetaereferencekeygenxofabsorboncestateE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1456	lemma	`lemma haetae_keygen_xof_seedbuf_incrementalE sd :`	Checked theorem haetaekeygenxofseedbufincrementalE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1466	lemma	`lemma haetae_keygen_xof_seedbuf_shake256E sd :`	Checked theorem haetaekeygenxofseedbufshake256E in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:1478	lemma	`lemma haetae_keygen_seedbuf_rhoprime_size seedbuf :`	Checked theorem haetaekeygenseedbufrhoprimesize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1486	lemma	`lemma haetae_keygen_seedbuf_sigma_size seedbuf :`	Checked theorem haetaekeygenseedbufsigmasize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1494	lemma	`lemma haetae_keygen_seedbuf_key_size seedbuf :`	Checked theorem haetaekeygenseedbufkeysize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1502	lemma	`lemma haetae_reference_keygen_xof_seedbuf_layout_wf sd :`	Checked theorem haetaereferencekeygenxofseedbuflayoutwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1516	lemma	`lemma haetae_reference_keygen_xof_call_wf_ok sd :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Algebra.ec:1537	lemma	`lemma haetae_keygen_rhoprime_ref_sliceE sd :`	Checked theorem haetaekeygenrhoprimerefsliceE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1543	lemma	`lemma haetae_keygen_sigma_ref_sliceE sd :`	Checked theorem haetaekeygensigmarefsliceE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1549	lemma	`lemma haetae_keygen_key_ref_sliceE sd :`	Checked theorem haetaekeygenkeyrefsliceE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1555	lemma	`lemma haetae_keygen_rhoprime_size sd :`	Checked theorem haetaekeygenrhoprimesize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1561	lemma	`lemma haetae_keygen_sigma_size sd :`	Checked theorem haetaekeygensigmasize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1567	lemma	`lemma haetae_keygen_key_size sd :`	Checked theorem haetaekeygenkeysize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1573	lemma	`lemma haetae_keygen_sampler_seed_size sigma counter :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:1577	lemma	`lemma haetae_keygen_counter_next_gt counter md :`	Checked theorem haetaekeygencounternextgt in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1581	lemma	`lemma public_matrix_seed_size md sd :`	Checked theorem publicmatrixseedsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1585	lemma	`lemma public_matrix_seed_rows_wf md sd :`	Checked theorem publicmatrixseedrowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1593	lemma	`lemma public_secret_vector_seed_wf md sd :`	Checked theorem publicsecretvectorseedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1597	lemma	`lemma public_error_vector_seed_wf md sd :`	Checked theorem publicerrorvectorseedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1601	lemma	`lemma public_rounding_vector_seed_wf md sd :`	Checked theorem publicroundingvectorseedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1605	lemma	`lemma haetae_reference_polymatkm_expand_matA_size md rhoprime :`	Checked theorem haetaereferencepolymatkmexpandmatAsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1611	lemma	`lemma haetae_reference_polymatkm_expand_matA_rows_wf md rhoprime :`	Checked theorem haetaereferencepolymatkmexpandmatArowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1619	lemma	`lemma haetae_reference_polyveck_expand_vecA_wf md rhoprime :`	Checked theorem haetaereferencepolyveckexpandvecAwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1626	lemma	`lemma haetae_reference_polyvecmk_expand_S_s1_wf md sigma counter :`	Checked theorem haetaereferencepolyvecmkexpandSs1wf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1634	lemma	`lemma haetae_reference_polyvecmk_expand_S_s2_wf md sigma counter :`	Checked theorem haetaereferencepolyvecmkexpandSs2wf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1642	lemma	`lemma haetae_reference_keygen_secret_vector_wf md sd :`	Checked theorem haetaereferencekeygensecretvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1649	lemma	`lemma haetae_reference_keygen_error_vector_wf md sd :`	Checked theorem haetaereferencekeygenerrorvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1656	lemma	`lemma haetae_reference_keygen_raw_public_vector_wf md sd :`	Checked theorem haetaereferencekeygenrawpublicvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1663	lemma	`lemma haetae_reference_keygen_raw_public_vector_coeffs_q_bound md sd :`	Probability bound $\Pr[\text{haetaereferencekeygenrawpublicvectorcoeffsqbound}] \le B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1671	lemma	`lemma haetae_reference_keygen_mode23_predecompose_vector_wf md sd :`	Checked theorem haetaereferencekeygenmode23predecomposevectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1679	lemma	`lemma haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound`	Probability bound $\Pr[\text{haetaereferencekeygenmode23predecomposevectorcoeffsqbound}] \le B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1688	lemma	`lemma haetae_reference_keygen_mode23_public_vector_polyq_wf md sd :`	Checked theorem haetaereferencekeygenmode23publicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1699	lemma	`lemma haetae_reference_keygen_mode23_adjusted_error_vector_wf md sd :`	Checked theorem haetaereferencekeygenmode23adjusterrorvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:1707	lemma	`lemma haetae_reference_keygen_mode5_public_vector_wf sd :`	Checked theorem haetaereferencekeygenmode5publicvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1715	lemma	`lemma haetae_reference_keygen_mode5_public_vector_coeffs_q_bound sd :`	Probability bound Pr[haetaereferencekeygenmode5publicvectorcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1723	lemma	`lemma haetae_reference_keygen_mode5_public_vector_polyq_wf sd :`	Checked theorem haetaereferencekeygenmode5publicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1732	lemma	`lemma haetae_keygen_raw_public_vector_wf md sd :`	Checked theorem haetaekeygenrawpublicvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1736	lemma	`lemma haetae_keygen_raw_public_vector_coeffs_q_bound md sd :`	Probability bound Pr[haetaekeygenrawpublicvectorcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1743	lemma	`lemma haetae_keygen_mode23_predecompose_vector_wf md sd :`	Checked theorem haetaekeygenmode23predecomposevectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1750	lemma	`lemma haetae_keygen_mode23_predecompose_vector_coeffs_q_bound md sd :`	Probability bound Pr[haetaekeygenmode23predecomposevectorcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1758	lemma	`lemma haetae_keygen_mode23_public_vector_polyq_wf md sd :`	Checked theorem haetaekeygenmode23publicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1769	lemma	`lemma haetae_keygen_mode23_rounding_lowbits_wf md sd :`	Checked theorem haetaekeygenmode23roundinglowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1776	lemma	`lemma haetae_keygen_mode23_adjusted_error_vector_wf md sd :`	Checked theorem haetaekeygenmode23adjusterrorvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1783	lemma	`lemma haetae_keygen_mode5_public_vector_wf sd :`	Checked theorem haetaekeygenmode5publicvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1790	lemma	`lemma haetae_keygen_mode5_public_vector_coeffs_q_bound sd :`	Probability bound Pr[haetaekeygenmode5publicvectorcoeffsqbound] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1798	lemma	`lemma haetae_keygen_mode5_public_vector_polyq_wf sd :`	Checked theorem haetaekeygenmode5publicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1807	lemma	`lemma haetae_public_key_vector_from_relation_polyq_wf md sd s e :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:1824	lemma	`lemma haetae_reference_keygen_public_vector_polyq_wf md sd :`	Checked theorem haetaereferencekeygenpublicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1832	lemma	`lemma haetae_keygen_public_vector_polyq_wf md sd :`	Checked theorem haetaekeygenpublicvectorpolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1839	lemma	`lemma public_key_vector_seed_wf md sd :`	Checked theorem publickeyvectorseedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1846	lemma	`lemma public_key_mode23_column_wf md pk :`	Checked theorem publickeymode23columnwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1851	lemma	`lemma haetae_verification_column_from_unpacked_wf md sd b :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:1865	lemma	`lemma public_key_verification_column_wf md pk :`	Checked theorem publickeyverificationcolumnwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1879	lemma	`lemma public_verification_matrix_size md pk :`	Checked theorem publicverificationmatrixsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1886	lemma	`lemma public_verification_matrix_rows_wf md pk :`	Checked theorem publicverificationmatrixrowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1903	lemma	`lemma challenge_embedding_wf md ch :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Algebra.ec:1916	lemma	`lemma public_key_of_secret_wf md sk :`	Checked theorem publickeyofsecretwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1920	lemma	`lemma haetae_pack_seed_size sd :`	Checked theorem haetaepackseedsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1924	lemma	`lemma haetae_unpack_seed_size enc :`	Checked theorem haetaeunpackseedsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1928	lemma	`lemma haetae_unpack_seed_pack sd :`	Checked theorem haetaeunpackseedpack in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1942	lemma	`lemma haetae_unpack_seed_pack_cat sd tail :`	Checked theorem haetaeunpackseedpackcat in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1955	lemma	`lemma haetae_polyq_coeff_bound_gt0 md :`	Probability bound Pr[haetaepolyqcoeffboundgt0] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1962	lemma	`lemma haetae_byte_norm_bounds x :`	Probability bound Pr[haetaebytenormbounds] $\leq B$ or loss inequality controlling a bad event in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:1966	lemma	`lemma haetae_byte_norm_small x :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Algebra.ec:1970	lemma	`lemma haetae_byte_norm_idempotent x :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:1974	lemma	`lemma haetae_unpack15_coeff0 c0 c1 :`	Checked theorem haetaeunpack15coeff0 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:1982	lemma	`lemma haetae_unpack15_coeff1 c0 c1 c2 :`	Checked theorem haetaeunpack15coeff1 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:1992	lemma	`lemma haetae_unpack15_coeff2 c1 c2 c3 :`	Checked theorem haetaeunpack15coeff2 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2002	lemma	`lemma haetae_unpack15_coeff3 c2 c3 c4 :`	Checked theorem haetaeunpack15coeff3 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2012	lemma	`lemma haetae_unpack15_coeff4 c3 c4 c5 :`	Checked theorem haetaeunpack15coeff4 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2022	lemma	`lemma haetae_unpack15_coeff5 c4 c5 c6 :`	Checked theorem haetaeunpack15coeff5 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2032	lemma	`lemma haetae_unpack15_coeff6 c5 c6 c7 :`	Checked theorem haetaeunpack15coeff6 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2042	lemma	`lemma haetae_unpack15_coeff7 c6 c7 :`	Checked theorem haetaeunpack15coeff7 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2050	lemma	`lemma haetae_pack_poly_q_ref_size md p :`	Checked theorem haetaepackpolyqrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2057	lemma	`lemma haetae_unpack_poly_q_ref_at_wf md enc offset :`	Checked theorem haetaeunpackpolyqrefatwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2064	lemma	`lemma haetae_pack_polyveck_body_ref_size md b :`	Checked theorem haetaepackpolyveckbodyrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2072	lemma	`lemma haetae_unpack_polyveck_body_ref_wf md enc offset :`	Checked theorem haetaeunpackpolyveckbodyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2082	lemma	`lemma haetae_pack_vk_ref_size md b sd :`	Checked theorem haetaepackvkrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2091	lemma	`lemma haetae_unpack_vk_public_key_ref_wf md enc :`	Checked theorem haetaeunpackvkpublickeyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2100	lemma	`lemma nth_cat_size_plus ['a] (d : 'a) (xs ys : 'a list) j :`	Checked theorem nthcatsizeplus in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2105	lemma	`lemma haetae_pack_polyveck_body_ref_nth md b row k :`	Checked theorem haetaepackpolyveckbodyrefnth in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2121	lemma	`lemma haetae_pack_polyveck_body_ref_mode5_nth b row k :`	Checked theorem haetaepackpolyveckbodyrefmode5nth in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2131	lemma	`lemma haetae_unpack_poly_q_mode23_coeff_at_cat prefix bytes blk ofs :`	Checked theorem haetaeunpackpolyqmode23coeffatcat in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2142	lemma	`lemma haetae_polyq_mode23_coeff_rng md p j :`	Checked theorem haetaepolyqmode23coeffrng in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2155	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs0 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2182	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs1 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2211	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs2 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2240	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs3 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2269	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs4 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2298	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs5 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2327	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs6 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2356	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 md p blk :`	Checked theorem haetaeunpackpolyqrefpackmode23coeffofs7 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2383	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_coeff md p i :`	Checked theorem haetaeunpackpolyqrefpackmode23coeff in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2446	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode23_at md prefix p :`	Checked theorem haetaeunpackpolyqrefpackmode23at in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2479	lemma	`lemma haetae_unpack_poly_q_mode23_coeff_at_body md prefix b row blk ofs :`	Checked theorem haetaeunpackpolyqmode23coeffatbody in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:2495	lemma	`lemma haetae_unpack_polyveck_body_ref_pack_mode23_coeff md prefix b row i :`	Checked theorem haetaeunpackpolyveckbodyrefpackmode23coeff in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2518	lemma	`lemma haetae_unpack_polyveck_body_ref_pack_mode23 md prefix b :`	Checked theorem haetaeunpackpolyveckbodyrefpackmode23 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2567	lemma	`lemma haetae_pack_vk_ref_roundtrip_mode23 md sd b :`	Checked theorem haetaepackvkrefroundtripmode23 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2585	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode5_coeff p i :`	Checked theorem haetaeunpackpolyqrefpackmode5coeff in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2603	lemma	`lemma haetae_unpack_poly_q_ref_pack_mode5_at prefix p :`	Checked theorem haetaeunpackpolyqrefpackmode5at in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2628	lemma	`lemma haetae_unpack_polyveck_body_ref_pack_mode5_coeff prefix b row i :`	Checked theorem haetaeunpackpolyveckbodyrefpackmode5coeff in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2684	lemma	`lemma haetae_unpack_polyveck_body_ref_pack_mode5 prefix b :`	Checked theorem haetaeunpackpolyveckbodyrefpackmode5 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2715	lemma	`lemma haetae_pack_vk_ref_roundtrip_mode5 sd b :`	Checked theorem haetaepackvkrefroundtripmode5 in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2732	lemma	`lemma haetae_public_key_ref_roundtrip md (pk : pkey) :`	Checked theorem haetaepublickeyrefroundtrip in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2755	lemma	`lemma haetae_keygen_public_key_ref_roundtrip md sd :`	Checked theorem haetaekeygenpublickeyrefroundtrip in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2771	lemma	`lemma haetae_pack_polyveck_body_size md b :`	Checked theorem haetaepackpolyveckbodysize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2778	lemma	`lemma haetae_pack_poly_q_size md p :`	Checked theorem haetaepackpolyqsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2782	lemma	`lemma haetae_unpack_poly_q_at_wf md enc offset :`	Checked theorem haetaeunpackpolyqatwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2786	lemma	`lemma haetae_unpack_poly_q_pack_at md prefix p :`	Checked theorem haetaeunpackpolyqpackat in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2803	lemma	`lemma haetae_unpack_polyveck_body_wf md enc offset :`	Checked theorem haetaeunpackpolyveckbodywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2813	lemma	`lemma haetae_body_index_div md row col :`	Checked theorem haetaebodyindexdiv in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2820	lemma	`lemma haetae_body_index_mod md row col :`	Checked theorem haetaebodyindexmod in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2827	lemma	`lemma haetae_body_index_lt md row col :`	Checked theorem haetaebodyindexlt in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2837	lemma	`lemma haetae_unpack_polyveck_body_pack_at md prefix b :`	Checked theorem haetaeunpackpolyveckbodypackat in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2860	lemma	`lemma haetae_unpack_polyveck_body_pack md sd b :`	Checked theorem haetaeunpackpolyveckbodypack in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2872	lemma	`lemma haetae_public_key_roundtrip md pk :`	Checked theorem haetaepublickeyroundtrip in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2886	lemma	`lemma haetae_pack_public_key_bytes_size md pk :`	Checked theorem haetaepackpublickeybytessize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2894	lemma	`lemma haetae_pack_public_key_bytes_wf md pk :`	Checked theorem haetaepackpublickeybyteswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2900	lemma	`lemma haetae_pack_public_key_bytes_ref_size md pk :`	Checked theorem haetaepackpublickeybytesrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2907	lemma	`lemma haetae_pack_public_key_bytes_ref_wf md pk :`	Checked theorem haetaepackpublickeybytesrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2915	lemma	`lemma haetae_unpack_public_key_bytes_wf md enc :`	Checked theorem haetaeunpackpublickeybyteswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2926	lemma	`lemma haetae_unpack_public_key_bytes_ref_wf md enc :`	Checked theorem haetaeunpackpublickeybytesrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2934	lemma	`lemma concrete_haetae_keygen_packed_public_key_wf md sd :`	Checked theorem concretehaetaekeygenpackedpublickeywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2942	lemma	`lemma concrete_haetae_keygen_unpacked_public_key_wf md sd :`	Checked theorem concretehaetaekeygenunpackedpublickeywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:2950	lemma	`lemma concrete_haetae_keygen_packed_public_key_ref_wf md sd :`	Checked theorem concretehaetaekeygenpackedpublickeyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2958	lemma	`lemma concrete_haetae_keygen_unpacked_public_key_ref_wf md sd :`	Checked theorem concretehaetaekeygenunpackedpublickeyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2966	lemma	`lemma packed_haetae_reference_public_key_wf md sd :`	Checked theorem packedhaetaereferencepublickeywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2974	lemma	`lemma packed_haetae_reference_public_key_polyq_wf md sd :`	Checked theorem packedhaetaereferencepublickeypolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2982	lemma	`lemma packed_haetae_reference_public_key_unpacked_wf md sd :`	Checked theorem packedhaetaereferencepublickeyunpackedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:2994	lemma	`lemma concrete_haetae_reference_keygen_packed_public_key_ref_wf md sd :`	Checked theorem concretehaetaereferencekeygenpackedpublickeyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3002	lemma	`lemma concrete_haetae_reference_keygen_unpacked_public_key_ref_wf md sd :`	Checked theorem concretehaetaereferencekeygenunpackedpublickeyrefwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3010	lemma	`lemma haetae_reference_keygen_public_key_ref_roundtrip md sd :`	Checked theorem haetaereferencekeygenpublickeyrefroundtrip in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3022	lemma	`lemma public_key_of_secret_relation md sk :`	Checked theorem publickeyofsecretrelation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3033	lemma	`lemma public_key_of_secret_equation_holds md sk :`	Checked theorem publickeyofsecretequationholds in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3039	op	`op valid_mode : mode -> bool = fun _ => true.`	Predicate/event validmode on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{validmode}}$. *Context:* module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3040	op	`op valid_keypair (md : mode) (pk : pkey) (sk : skey) : bool =`	Predicate/event validkeypair on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{validkeypair}}$. *Context:* module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3042	op	`op valid_signature (md : mode) (sig : signature) : bool =`	Predicate/event validsignature on the game state; in probability expressions it denotes the indicator event $1_{\mathsf{validsignature}}$. *Context:* module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3051	op	`op sig_commitment_highbits (sig : signature) : polyveck = sig.‘1.`	Total mathematical function or predicate sigcommitmenthighbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3052	op	`op sig_commitment_lowbits (sig : signature) : poly = sig.‘2.`	Total mathematical function or predicate sigcommitmentlowbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3053	op	`op sig_message_hash (sig : signature) : crh = sig.‘3.`	Oracle-related function sigmessagehash used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:3054	op	`op sig_challenge (sig : signature) : challenge = sig.‘4.`	Probability law or sampler sigchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3055	op	`op sig_token_value (sig : signature) : sig_token = sig.‘5.`	Total mathematical function or predicate sigtokenvalue in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3056	op	`op sig_response (sig : signature) : polyvecl = (sig_token_value sig).‘1.`	Total mathematical function or predicate sigresponse in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3057	op	`op sig_response_aux (sig : signature) : polyveck = (sig_token_value sig).‘2.`	Total mathematical function or predicate sigresponseaux in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3058	op	`op sig_hint (sig : signature) : hint_t = (sig_token_value sig).‘3.`	Total mathematical function or predicate sighint in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3060	op	`op encode_poly (p : poly) : byte list = p.`	Deterministic algebraic map or predicate encodepoly over HAETAE module/ring objects.
HAETAE_Algebra.ec:3061	op	`op encode_polyveck (xs : polyveck) : byte list = flatten xs.`	Deterministic algebraic map or predicate encodepolyveck over HAETAE module/ring objects.
HAETAE_Algebra.ec:3062	op	`op encode_pkey (pk : pkey) : byte list = pk.‘1 ++ encode_polyveck pk.‘2.`	Total mathematical function or predicate encodepkey in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3064	op	`op deterministic_poly (tag : int) (src : byte list) : poly =`	Deterministic algebraic map or predicate deterministicpoly over HAETAE module/ring objects.
HAETAE_Algebra.ec:3066	op	`op deterministic_polyveck (md : mode) (src : byte list) : polyveck =`	Deterministic algebraic map or predicate deterministicpolyveck over HAETAE module/ring objects.
HAETAE_Algebra.ec:3070	op	`op deterministic_unit_coeff (tag : int) (src : byte list) (i : int) :`	Deterministic algebraic map or predicate deterministicunitcoeff over HAETAE module/ring objects.
HAETAE_Algebra.ec:3073	op	`op deterministic_unit_poly (tag : int) (src : byte list) : poly =`	Deterministic algebraic map or predicate deterministicunitpoly over HAETAE module/ring objects.
HAETAE_Algebra.ec:3075	op	`op deterministic_unit_polyveck`	Deterministic algebraic map or predicate deterministicunitpolyveck over HAETAE module/ring objects.
HAETAE_Algebra.ec:3080	op	`op deterministic_unit_polyvecl`	Deterministic algebraic map or predicate deterministicunitpolyvecl over HAETAE module/ring objects.
HAETAE_Algebra.ec:3085	op	`op commitment_source`	Total mathematical function or predicate commitmentsource in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3089	op	`op commitment_raw :`	Total mathematical function or predicate commitmentraw in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3093	op	`op response_source`	Total mathematical function or predicate responsesource in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3097	op	`op response_aux_vector :`	Deterministic algebraic map or predicate responseauxvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:3101	op	`op signing_sample_y1 :`	Probability law or sampler signingsampley1; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3105	op	`op signing_sample_y2 :`	Probability law or sampler signingsampley2; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3109	op	`op signing_entropy_token_of_coins (coins : random_coins) : int =`	Probability law or sampler signingentropytokenofcoins; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3118	op	`op commitment_lowbits :`	Total mathematical function or predicate commitmentlowbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3127	op	`op message_hash : pkey -> context -> message -> crh =`	Oracle-related function messagehash used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Algebra.ec:3129	op	`op challenge_source (highbits : polyveck) (lowbits : poly) (mu : crh) :`	Probability law or sampler challengesource; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3132	op	`op challenge_from_seed (md : mode) (src : byte list) : challenge =`	Probability law or sampler challengefromseed; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3138	op	`op challenge_hash : mode -> polyveck -> poly -> crh -> challenge =`	Probability law or sampler challengehash; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3141	op	`op commitment_challenge :`	Probability law or sampler commitmentchallenge; mathematically a distribution on the corresponding HAETAE coins/challenge space.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:3148	op	`op commitment_highbits :`	Total mathematical function or predicate commitmenthighbits in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3156	op	`op response_vector :`	Deterministic algebraic map or predicate responsevector over HAETAE module/ring objects.
HAETAE_Algebra.ec:3160	op	`op hint_vector :`	Deterministic algebraic map or predicate hintvector over HAETAE module/ring objects.
HAETAE_Algebra.ec:3164	op	`op signature_token :`	Total mathematical function or predicate signaturetoken in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3171	lemma	`lemma deterministic_poly_wf tag src :`	Checked theorem deterministicpolywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3175	lemma	`lemma deterministic_polyveck_wf md src :`	Checked theorem deterministicpolyveckwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3185	lemma	`lemma deterministic_unit_poly_wf tag src :`	Checked theorem deterministicunitpolywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3189	lemma	`lemma deterministic_unit_poly_unit tag src :`	Checked theorem deterministicunitpolyunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3200	lemma	`lemma deterministic_unit_polyveck_wf md tag src :`	Checked theorem deterministicunitpolyveckwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3210	lemma	`lemma deterministic_unit_polyveck_unit md tag src :`	Checked theorem deterministicunitpolyveckunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3220	lemma	`lemma deterministic_unit_polyvecl_wf md tag src :`	Checked theorem deterministicunitpolyveclwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3230	lemma	`lemma deterministic_unit_polyvecl_unit md tag src :`	Checked theorem deterministicunitpolyveclunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3240	lemma	`lemma commitment_raw_wf md sk m ctx coins :`	Checked theorem commitmentrawwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3246	lemma	`lemma commitment_highbits_wf md sk m ctx coins :`	Checked theorem commitmenthighbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3253	lemma	`lemma commitment_lowbits_wf md sk m ctx coins :`	Checked theorem commitmentlowbitswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3260	lemma	`lemma commitment_lowbits_entropy_token md sk m ctx coins :`	Checked theorem commitmentlowbitsentropytoken in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3270	lemma	`lemma challenge_from_seed_wf md src :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3282	lemma	`lemma challenge_from_seed_sparse md src :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3297	lemma	`lemma challenge_hash_wf md w1 w0 mu :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3301	lemma	`lemma challenge_hash_sparse md w1 w0 mu :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3305	lemma	`lemma response_vector_wf md sk m ctx coins :`	Checked theorem responsevectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3311	lemma	`lemma response_vector_unit md sk m ctx coins :`	Checked theorem responsevectorunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3317	lemma	`lemma response_aux_vector_wf md sk m ctx coins :`	Checked theorem responseauxvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3323	lemma	`lemma response_aux_vector_unit md sk m ctx coins :`	Checked theorem responseauxvectorunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3329	lemma	`lemma signing_sample_y1_wf md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:3335	lemma	`lemma signing_sample_y1_unit md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:3341	lemma	`lemma signing_sample_y2_wf md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:3347	lemma	`lemma signing_sample_y2_unit md sk m ctx coins :`	Distributional theorem identifying the implemented sampler with the paper distribution, possibly up to rejection or exact-hyperball loss.
HAETAE_Algebra.ec:3353	lemma	`lemma hint_vector_wf md sk m ctx coins :`	Checked theorem hintvectorwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3359	lemma	`lemma hint_vector_unit md sk m ctx coins :`	Checked theorem hintvectorunit in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3365	lemma	`lemma message_hash_size pk ctx m :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3369	lemma	`lemma challenge_hash_size md w1 w0 mu :`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Algebra.ec:3373	op	`op keygen_internal (md :  mode) (sd :  seed) :  pkey * skey =`	Total mathematical function or predicate keygeninternal in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3375	op	`op sign_internal :`	Total mathematical function or predicate signinternal in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3386	op	`op verify_challenge_consistent`	Probability law or sampler verifychallengeconsistent; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Algebra.ec:3397	op	`op verify_reconstructed_highbits`	Total mathematical function or predicate verifyreconstructedhighbits in the module/ring algebra, norms, and deterministic arithmetic relations.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:3402	op	`op verify_public_equation`	Total mathematical function or predicate verifypublicequation in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3407	lemma	`lemma keygen_internal_valid md sd :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:3411	lemma	`lemma public_key_of_keygen md sd :`	Checked theorem publickeyofkeygen in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3416	lemma	`lemma public_key_vector_seed_keygen_equation md sd :`	Checked theorem publickeyvectorseedkeygenequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3423	lemma	`lemma public_key_vector_seed_mode23_keygen_equation md sd :`	Checked theorem publickeyvectorseedmode23keygenequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3443	lemma	`lemma public_key_vector_seed_mode5_keygen_equation sd :`	Checked theorem publickeyvectorseedmode5keygenequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3459	lemma	`lemma haetae_keygen_raw_public_vector_keygen_equation md sd :`	Checked theorem haetaekeygenrawpublicvectorkeygenequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3468	lemma	`lemma haetae_reference_keygen_raw_public_vector_equation md sd :`	Checked theorem haetaereferencekeygenrawpublicvectorequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3481	lemma	`lemma haetae_reference_keygen_mode23_public_vector_equation md sd :`	Checked theorem haetaereferencekeygenmode23publicvectorequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3503	lemma	`lemma haetae_reference_keygen_mode23_adjusted_error_equation md sd :`	Checked theorem haetaereferencekeygenmode23adjustederrorequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3520	lemma	`lemma haetae_reference_keygen_mode5_public_vector_equation sd :`	Checked theorem haetaereferencekeygenmode5publicvectorequation in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3536	lemma	`lemma packed_haetae_reference_public_key_seedE md sd :`	Checked theorem packedhaetaereferencepublickeyseedE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3541	lemma	`lemma packed_haetae_reference_public_key_bodyE md sd :`	Checked theorem packedhaetaereferencepublickeybodyE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3546	lemma	`lemma haetae_reference_keygen_seed_flow_wf_ok md sd :`	Checked theorem haetaereferencekeygenseedflowwfok in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3557	lemma	`lemma packed_haetae_public_key_wf md sd :`	Checked theorem packedhaetaepublickeywf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3564	lemma	`lemma packed_haetae_public_key_polyq_wf md sd :`	Checked theorem packedhaetaepublickeypolyqwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3571	lemma	`lemma packed_haetae_public_key_unpacked_wf md sd :`	Checked theorem packedhaetaepublickeyunpackedwf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3579	lemma	`lemma haetae_keygen_public_key_roundtrip md sd :`	Checked theorem haetaekeygenpublickeyroundtrip in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3589	lemma	`lemma public_key_of_secret_packed md sk :`	Checked theorem publickeyofsecretpacked in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3596	lemma	`lemma keygen_public_key_packed md sd :`	Checked theorem keygenpublickeypacked in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3603	lemma	`lemma public_verification_matrix_packedE md pk :`	Checked theorem publicverificationmatrixpackedE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3607	lemma	`lemma packed_haetae_keygen_verification_matrixE md sd :`	Checked theorem packedhaetaekeygenverificationmatrixE in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3617	lemma	`lemma honest_public_verification_matrix_eq_packed_keygen md sd :`	Checked theorem honestpublicverificationmatrixeqpackedkeygen in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3627	lemma	`lemma concrete_keygen_public_verification_matrix_correct md sd :`	Checked theorem concretekeygenpublicverificationmatrixcorrect in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3644	lemma	`lemma concrete_keygen_public_verification_matrix_ref_correct md sd :`	Checked theorem concretekeygenpublicverificationmatrixrefcorrect in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3661	lemma	`lemma concrete_reference_keygen_public_verification_matrix_ref_correct md sd :`	Checked theorem concretereferencekeygenpublicverificationmatrixrefcorrect in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3673	lemma	`lemma concrete_keygen_public_verification_matrix_size md sd :`	Checked theorem concretekeygenpublicverificationmatrixsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3681	lemma	`lemma concrete_keygen_public_verification_matrix_ref_size md sd :`	Checked theorem concretekeygenpublicverificationmatrixrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3689	lemma	`lemma concrete_reference_keygen_public_verification_matrix_ref_size md sd :`	Checked theorem concretereferencekeygenpublicverificationmatrixrefsize in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3698	lemma	`lemma concrete_keygen_public_verification_matrix_rows_wf md sd :`	Checked theorem concretekeygenpublicverificationmatrixrowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Algebra.ec:3718	lemma	`lemma concrete_keygen_public_verification_matrix_ref_rows_wf md sd :`	Checked theorem concretekeygenpublicverificationmatrixrefrowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3739	lemma	`lemma concrete_reference_keygen_public_verification_matrix_ref_rows_wf md sd :`	Checked theorem concretereferencekeygenpublicverificationmatrixrefrowswf in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Algebra.ec:3760	lemma	`lemma valid_signature_sign_internal md sk m ctx coins :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:3769	lemma	`lemma verify_challenge_consistent_sign_internal md sk m ctx coins :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Algebra.ec:3777	op	`op coeff_norm_sq (x : coeff) : int = x * x.`	Deterministic algebraic map or predicate coeffnormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3778	op	`op poly_norm_sq (p : poly) : int =`	Deterministic algebraic map or predicate polynormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3782	op	`op polyvecl_norm_sq (md : mode) (xs : polyvecl) : int =`	Deterministic algebraic map or predicate polyveclnormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3786	op	`op polyveck_norm_sq (md : mode) (xs : polyveck) : int =`	Deterministic algebraic map or predicate polyvecknormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3790	op	`op signature_response_norm_sq (md : mode) (sig : signature) : int =`	Deterministic algebraic map or predicate signatureresponsenormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3793	op	`op signature_verify_norm_sq (md : mode) (sig : signature) : int =`	Deterministic algebraic map or predicate signatureverifynormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3797	op	`op secret_norm_sq (sk : skey) : int = sk.‘2.`	Deterministic algebraic map or predicate secretnormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3798	op	`op response_norm_sq (md : mode) (sig : signature) : int =`	Deterministic algebraic map or predicate responsenormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3800	op	`op verify_norm_sq (md : mode) (sig : signature) : int =`	Deterministic algebraic map or predicate verifynormsq over HAETAE module/ring objects.
HAETAE_Algebra.ec:3803	op	`op secret_norm_ok (md : mode) (sk : skey) : bool =`	Deterministic algebraic map or predicate secretnormok over HAETAE module/ring objects.
HAETAE_Algebra.ec:3805	op	`op response_norm_ok (md : mode) (sig : signature) : bool =`	Deterministic algebraic map or predicate responsenormok over HAETAE module/ring objects.
HAETAE_Algebra.ec:3807	op	`op verify_norm_ok (md : mode) (sig : signature) : bool =`	Deterministic algebraic map or predicate verifynormok over HAETAE module/ring objects.
HAETAE_Algebra.ec:3810	op	`op verify_internal :`	Total mathematical function or predicate verifyinternal in the module/ring algebra, norms, and deterministic arithmetic relations.
HAETAE_Algebra.ec:3819	lemma	`lemma secret_norm_ok_current md sd :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:3823	lemma	`lemma response_norm_ok_current md sk m ctx coins :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:3843	lemma	`lemma verify_norm_ok_current md sk m ctx coins :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Algebra.ec:3868	lemma	`lemma verify_internal_sign_internal md sk m ctx coins :`	Checked theorem verifyinternalsigninternal in the module/ring algebra, norms, and deterministic arithmetic relations; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:5	op	`op seedbytes : int = 32.`	Probability law or sampler seedbytes; mathematically a distribution on the corresponding HAETAE coins/challenge space.
HAETAE_Params.ec:6	op	`op crhbytes : int = 64.`	Total mathematical function or predicate crhbytes in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:7	op	`op n : int = 256.`	Total mathematical function or predicate n in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:8	op	`op q : int = 64513.`	Total mathematical function or predicate q in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:9	op	`op dq : int = 2 * q.`	Total mathematical function or predicate dq in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:11	type	`type mode = [ Mode2 | Mode3 | Mode5 ].`	Carrier set mode used in the HAETAE parameter constants and admissibility side conditions; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Params.ec:13	op	`op mode_k (md : mode) : int =`	Total mathematical function or predicate modek in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:18	op	`op mode_l (md : mode) : int =`	Total mathematical function or predicate model in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:23	op	`op mode_tau (md : mode) : int =`	Total mathematical function or predicate modetau in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:28	op	`op mode_b0sq (md : mode) : int =`	Total mathematical function or predicate modeb0sq in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:33	op	`op mode_b1sq (md : mode) : int =`	Total mathematical function or predicate modeb1sq in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:38	op	`op mode_b2sq (md : mode) : int =`	Total mathematical function or predicate modeb2sq in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:43	op	`op mode_ln (_ : mode) : int = 8192.`	Total mathematical function or predicate modeln in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:45	op	`op mode_alpha_hint (md : mode) : int =`	Total mathematical function or predicate modealphahint in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:50	op	`op mode_m (md : mode) : int = mode_l md - 1.`	Total mathematical function or predicate modem in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:52	op	`op mode_crypto_bytes (md : mode) : int =`	Total mathematical function or predicate modecryptobytes in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:57	op	`op mode_polyq_packedbytes (md : mode) : int =`	Numerical bound or budget parameter modepolyqpackedbytes used to upper-bound a probability loss in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:62	op	`op mode_publickeybytes (md : mode) : int =`	Total mathematical function or predicate modepublickeybytes in the HAETAE parameter constants and admissibility side conditions.
HAETAE_Params.ec:65	lemma	`lemma seedbytes_gt0 : 0 < seedbytes by rewrite /seedbytes.`	Checked theorem seedbytesgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:66	lemma	`lemma crhbytes_gt0 : 0 < crhbytes by rewrite /crhbytes.`	Checked theorem crhbytesgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:67	lemma	`lemma n_gt0 : 0 < n by rewrite /n.`	Checked theorem ngt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:68	lemma	`lemma q_gt0 : 0 < q by rewrite /q.`	Checked theorem qgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:69	lemma	`lemma dqE : dq = 129026 by rewrite /dq /q.`	Checked theorem dqE in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Params.ec:71	lemma	`lemma mode_k_gt0 md : 0 < mode_k md.`	Checked theorem modekgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:74	lemma	`lemma mode_l_gt0 md : 0 < mode_l md.`	Checked theorem modelgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:77	lemma	`lemma mode_tau_gt0 md : 0 < mode_tau md.`	Checked theorem modetaugt0 in the HAETAE parameter constants and admisssibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:80	lemma	`lemma mode_tau_le_n md : mode_tau md <= n.`	Checked theorem modetaulen in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:83	lemma	`lemma mode_b0sq_gt0 md : 0 < mode_b0sq md.`	Checked theorem modeb0sqgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:86	lemma	`lemma mode_b1sq_gt0 md : 0 < mode_b1sq md.`	Checked theorem modeb1sqgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:89	lemma	`lemma mode_b2sq_gt0 md : 0 < mode_b2sq md.`	Checked theorem modeb2sqgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:92	lemma	`lemma mode_ln_gt0 md : 0 < mode_ln md.`	Checked theorem modelngt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:95	lemma	`lemma mode_alpha_hint_gt0 md : 0 < mode_alpha_hint md.`	Checked theorem modealphahintgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:98	lemma	`lemma mode_m_gt0 md : 0 < mode_m md.`	Checked theorem modemgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:101	lemma	`lemma mode_crypto_bytes_gt0 md : 0 < mode_crypto_bytes md.`	Checked theorem modecryptobytesgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:104	lemma	`lemma mode_polyq_packedbytes_gt0 md : 0 < mode_polyq_packedbytes md.`	Checked theorem modepolyqpackedbytesgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:107	lemma	`lemma n_le_mode_polyq_packedbytes md : n <= mode_polyq_packedbytes md.`	Checked theorem nlemodepolyqpackedbytes in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Params.ec:110	lemma	`lemma mode_publickeybytes_gt0 md : 0 < mode_publickeybytes md.`	Checked theorem modepublickeybytesgt0 in the HAETAE parameter constants and admissibility side conditions; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:8	type	`type fips_byte = int.`	Carrier set fipsbyte used in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; EasyCrypt equality is the mathematical equality on this set.
HAETAE_FIPS202.ec:9	type	`type fips202_state = fips_byte list.`	Carrier set fips202state used in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; EasyCrypt equality is the mathematical equality on this set.
HAETAE_FIPS202.ec:11	op	`op fips202_state_bytes : int = 200.`	Total mathematical function or predicate fips202statebytes in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface.
HAETAE_FIPS202.ec:12	op	`op shake256_rate_bytes : int = 136.`	Oracle-related function shake256ratebytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:13	op	`op shake256_capacity_bytes : int = 64.`	Oracle-related function shake256capacitybytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:14	op	`op shake256_domain_separator : fips_byte = 31.`	Oracle-related function shake256domainseparator used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:15	op	`op fips202_final_padding_byte : fips_byte = 128.`	Total mathematical function or predicate fips202finalpaddingbyte in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface.
HAETAE_FIPS202.ec:17	op	`op fips202_keccak_f1600 (st : fips202_state) : fips202_state =`	Oracle-related function fips202keccakf1600 used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:20	op	`op shake256_absorb_once_short_domain`	Oracle-related function shake256absorbonceshortdomain used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:24	op	`op shake256_absorb_once_short_block_byte`	Oracle-related function shake256absorbonceshortblockbyte used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:33	op	`op shake256_absorb_once_short_state`	Oracle-related function shake256absorbonceshortstate used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:42	op	`op shake256_absorb_once`	Oracle-related function shake256absorbonce used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:47	op	`op shake256_squeeze (st : fips202_state) (outlen : int) : fips_byte list =`	Oracle-related function shake256squeeze used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:50	op	`op shake256`	Oracle-related function shake256 used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_FIPS202.ec:54	lemma	`lemma fips202_state_bytesE :`	Checked theorem fips202statebytesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:58	lemma	`lemma shake256_rate_bytesE :`	Checked theorem shake256ratebytesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:62	lemma	`lemma shake256_capacity_bytesE :`	Checked theorem shake256capacitybytesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:66	lemma	`lemma shake256_domain_separatorE :`	Theorem shake256domainseparatorE contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_FIPS202.ec:70	lemma	`lemma fips202_final_padding_byteE :`	Theorem fips202finalpaddingbyteE contributing to the final HAETAE EUF-CMA advantage bound $\mathrm{Adv}^{\text{euf-cma}}_{\text{HAETAE}}(\mathcal{A})$.
HAETAE_FIPS202.ec:74	lemma	`lemma shake256_rate_capacity_state_bytesE :`	Checked theorem shake256ratecapacitystatebytesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_FIPS202.ec:81	lemma	`lemma shake256_absorb_once_short_state_size input inlen :`	Checked theorem shake256absorbonceshortstatesize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:88	lemma	`lemma shake256_squeeze_size st outlen :`	Checked theorem shake256squeezesize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_FIPS202.ec:97	lemma	`lemma shake256_size input inlen outlen :`	Checked theorem shake256size in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec:5	type	`type keccak_byte = int.`	Carrier set keccakbyte used in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Keccak1600.ec:6	type	`type keccak_lane = bool list.`	Carrier set keccaklane used in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Keccak1600.ec:7	type	`type keccak_state = keccak_lane list.`	Carrier set keccakstate used in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; EasyCrypt equality is the mathematical equality on this set.
HAETAE_Keccak1600.ec:9	op	`op keccak_lane_bits : int = 64.`	Oracle-related function keccaklanebits used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:10	op	`op keccak_state_lanes : int = 25.`	Oracle-related function keccakstatelanes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:11	op	`op keccak_state_bytes : int = 200.`	Oracle-related function keccakstatebytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:13	op	`op keccak_byte_norm (b : keccak_byte) : keccak_byte = b %% 256.`	Oracle-related function keccakbytenorm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:14	op	`op keccak_byte_bit (b : keccak_byte) (i : int) : bool =`	Oracle-related function keccakbytebit used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:16	op	`op keccak_word_norm (w : int) : int = w %% (2 ^ keccak_lane_bits).`	Oracle-related function keccakwordnorm used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:17	op	`op keccak_word_bit (w : int) (i : int) : bool =`	Oracle-related function keccakwordbit used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:20	op	`op keccak_lane_zero : keccak_lane = nseq keccak_lane_bits false.`	Oracle-related function keccaklanezero used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:21	op	`op keccak_lane_bit (lane : keccak_lane) (i : int) : bool =`	Oracle-related function keccaklanebit used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:23	op	`op keccak_lane_wf (lane : keccak_lane) : bool =`	Oracle-related function keccaklanewf used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:26	op	`op keccak_lane_xor (a b : keccak_lane) : keccak_lane =`	Oracle-related function keccaklanexor used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:30	op	`op keccak_lane_and (a b : keccak_lane) : keccak_lane =`	Oracle-related function keccaklaneand used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:34	op	`op keccak_lane_not (a : keccak_lane) : keccak_lane =`	Oracle-related function keccaklanenot used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:38	op	`op keccak_lane_rotl (a : keccak_lane) (offset : int) : keccak_lane =`	Oracle-related function keccaklanerotl used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:43	op	`op keccak_lane_of_int (x : int) : keccak_lane =`	Oracle-related function keccaklaneofint used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:46	op	`op keccak_lane_to_byte (lane : keccak_lane) (byte_index : int) : keccak_byte =`	Oracle-related function keccaklanetobyte used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:55	op	`op keccak_bytes_to_lane (bs : keccak_byte list) (lane_index : int) :`	Oracle-related function keccakbytestolane used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:64	op	`op keccak_lanes_of_bytes (bs : keccak_byte list) : keccak_state =`	Oracle-related function keccaklanesofbytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:67	op	`op keccak_bytes_of_lanes (st : keccak_state) : keccak_byte list =`	Oracle-related function keccakbytesoflanes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:75	op	`op keccak_lane_index (x y : int) : int =`	Oracle-related function keccaklaneindex used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:77	op	`op keccak_state_lane (st : keccak_state) (x y : int) : keccak_lane =`	Oracle-related function keccakstatelane used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:80	op	`op keccak_rho_offsets : int list =`	Oracle-related function keccakrhooffsets used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:87	op	`op keccak_round_constants : int list =`	Oracle-related function keccakroundconstants used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:113	op	`op keccak_theta_c (st : keccak_state) (x : int) : keccak_lane =`	Oracle-related function keccakthetac used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:124	op	`op keccak_theta_d (st : keccak_state) (x : int) : keccak_lane =`	Oracle-related function keccakthetad used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:129	op	`op keccak_theta (st : keccak_state) : keccak_state =`	Oracle-related function keccaktheta used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:139	op	`op keccak_rho_pi (st : keccak_state) : keccak_state =`	Oracle-related function keccakrhopi used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:151	op	`op keccak_chi (st : keccak_state) : keccak_state =`	Oracle-related function keccakchi used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:163	op	`op keccak_iota (st : keccak_state) (rc : keccak_lane) : keccak_state =`	Oracle-related function keccakiota used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:171	op	`op keccak_round (st : keccak_state) (rc : keccak_lane) : keccak_state =`	Oracle-related function keccakround used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:174	op	`op keccak_f1600_lanes (st : keccak_state) : keccak_state =`	Oracle-related function keccakf1600lanes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:180	op	`op keccak_f1600_bytes (st : keccak_byte list) : keccak_byte list =`	Oracle-related function keccakf1600bytes used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec:183	lemma	`lemma keccak_lane_bitsE :`	Checked theorem keccaklanebitsE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec:187	lemma	`lemma keccak_state_lanesE :`	Checked theorem keccakstatelanesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Keccak1600.ec: 191	lemma	`lemma keccak_state_bytesE :`	Checked theorem keccakstatebytesE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 195	lemma	`lemma keccak_lane_zero_wf :`	Checked theorem keccaklanezerowf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 199	lemma	`lemma keccak_lane_xor_wf a b :`	Checked theorem keccaklanexorwf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 203	lemma	`lemma keccak_lane_and_wf a b :`	Checked theorem keccaklaneandwf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 207	lemma	`lemma keccak_lane_not_wf a :`	Checked theorem keccaklanenotwf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 211	lemma	`lemma keccak_lane_rotl_wf a offset :`	Checked theorem keccaklanerotlwf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 215	lemma	`lemma keccak_lane_of_int_wf x :`	Checked theorem keccaklaneofintwf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 219	lemma	`lemma keccak_lane_mod_range x :`	Deterministic side-condition lemma establishing algebraic validity, support membership, or parameter admissibility.
HAETAE_Keccak1600.ec: 223	lemma	`lemma keccak_word_modulus_pos :`	Checked theorem keccakwordmoduluspos in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 227	lemma	`lemma keccak_word_norm_small w :`	Program-logic theorem: a Hoare/phoare/losslessness judgment proving termination and postcondition preservation for the corresponding procedure.
HAETAE_Keccak1600.ec: 235	lemma	`lemma keccak_pow2_0E :`	Checked theorem keccakpow20E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 239	lemma	`lemma keccak_pow2_1E :`	Checked theorem keccakpow21E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 243	lemma	`lemma keccak_pow2_2E :`	Checked theorem keccakpow22E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 247	lemma	`lemma keccak_pow2_3E :`	Checked theorem keccakpow23E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 254	lemma	`lemma keccak_pow2_4E :`	Checked theorem keccakpow24E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 261	lemma	`lemma keccak_pow2_5E :`	Checked theorem keccakpow25E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 268	lemma	`lemma keccak_pow2_6E :`	Checked theorem keccakpow26E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 275	lemma	`lemma keccak_pow2_7E :`	Checked theorem keccakpow27E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 282	lemma	`lemma keccak_bytes_to_lane_wf bs lane_index :`	Checked theorem keccakbytestolanewf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 286	lemma	`lemma keccak_rho_offsets_size :`	Checked theorem keccakrhooffsetssize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 290	lemma	`lemma keccak_round_constants_size :`	Checked theorem keccakroundconstantssize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 294	lemma	`lemma keccak_lanes_of_bytes_size bs :`	Checked theorem keccaklanesofbytessize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 298	lemma	`lemma keccak_lanes_of_bytes_lane_wf bs lane :`	Checked theorem keccaklanesofbyteslanewf in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 308	lemma	`lemma keccak_lanes_of_bytes_bitE bs lane bit :`	Checked theorem keccaklanesofbytesbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 324	lemma	`lemma keccak_lane_xor_bitE a b bit :`	Checked theorem keccaklanexorbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 334	lemma	`lemma keccak_lane_and_bitE a b bit :`	Checked theorem keccaklaneandbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 344	lemma	`lemma keccak_lane_not_bitE a bit :`	Checked theorem keccaklanenotbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 354	lemma	`lemma keccak_lane_rotl_bitE a offset bit :`	Checked theorem keccaklanerotlbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 364	lemma	`lemma keccak_lane_of_int_bitE x bit :`	Checked theorem keccaklaneofintbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 374	lemma	`lemma keccak_state_lane_bitE st x y bit :`	Checked theorem keccakstatelanebitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
HAETAE_Keccak1600.ec: 381	op	`op keccak_theta_c_bit_index_value`	Oracle-related function keccakthetacbitindexvalue used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec: 394	op	`op keccak_theta_d_bit_index_value`	Oracle-related function keccakthetadbitindexvalue used to form or interpret a random-oracle query in the lazy-ROM proof.
HAETAE_Keccak1600.ec: 402	lemma	`lemma keccak_theta_c_bitE st x bit :`	Checked theorem keccakthetacbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 415	lemma	`lemma keccak_theta_c_bit_indexE st x bit :`	Checked theorem keccakthetacbitindexE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 425	lemma	`lemma keccak_theta_d_bitE st x bit :`	Checked theorem keccakthetadbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 439	lemma	`lemma keccak_theta_d_bit_indexE st x bit :`	Checked theorem keccakthetadbitindexE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 452	lemma	`lemma keccak_theta_bitE st lane bit :`	Checked theorem keccakthetabitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 469	lemma	`lemma keccak_theta_bit_indexE st lane bit :`	Checked theorem keccakthetabitindexE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 484	lemma	`lemma keccak_rho_pi_bitE st lane bit :`	Checked theorem keccakrhopibitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 505	lemma	`lemma keccak_rho_pi_bit_indexE st lane bit :`	Checked theorem keccakrhopibitindexE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 524	lemma	`lemma keccak_chi_bitE st lane bit :`	Checked theorem keccakchibitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 545	lemma	`lemma keccak_chi_bit_indexE st lane bit :`	Checked theorem keccakchibitindexE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 566	lemma	`lemma keccak_iota_bitE st rc lane bit :`	Checked theorem keccakiotabitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 585	lemma	`lemma keccak_round_bitE st rc lane bit :`	Checked theorem keccakroundbitE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 611	lemma	`lemma keccak_bytes_of_lanes_size st :`	Checked theorem keccakbytesoflanessize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 615	lemma	`lemma keccak_bytes_of_lanes_byteE st i :`	Checked theorem keccakbytesoflanesbyteE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 626	lemma	`lemma keccak_lane_to_byte_bitsE lane byte_index :`	Checked theorem keccaklanetobytebitsE in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 650	lemma	`lemma keccak_lane_to_byte_from_bitsE`	Lazy-ROM invariant theorem: oracle maps/logs preserve consistency, freshness, or programming equivalence for $\mathcal{H}$.
HAETAE_Keccak1600.ec: 676	lemma	`lemma keccak_lane_to_byte_70E lane byte_index :`	Checked theorem keccaklanetobyte70E in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.
HAETAE_Keccak1600.ec: 693	lemma	`lemma keccak_f1600_bytes_size st :`	Checked theorem keccakf1600bytessize in the deterministic SHAKE/FIPS-202 support for the modeled random-oracle interface; mathematically it is a universally quantified implication or probability inequality over the stated objects.